HF Model Finder: Constraint-Verified Retrieval-Augmented Generation for Finding Hugging Face Models

Capstone Project Report submitted in partial fulfilment

of

Name of the Programme

in

Name of the Stream

by

Aditya Rane (Roll No., SAP ID)

Under the supervision of

Name of Faculty Mentor

(Designation, Name of the Department, MPSTME)

SVKM's NMIMS University

(Deemed-to-be University)

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

(Academic Year)

CERTIFICATE



This is to certify that the project entitled **"HF Model Finder: Constraint-Verified Retrieval-Augmented Generation for Finding Hugging Face Models"** has been done by **Mr. Aditya Rane** (Roll No., SAP ID) under my guidance and supervision and has been submitted in partial fulfilment of the degree of Name of the Programme in Name of the Stream of MPSTME, SVKM's NMIMS (Deemed-to-be University), Mumbai, India.

Project mentor (name and signature)
(Internal Guide)
Name of the mentor

Examiner (name and signature)
Name of the examiner

Date: dd Month yyyy
Place: Mumbai

(HoD) (name and signature)
Name of the Head of Department

ACKNOWLEDGEMENT

I express my sincere gratitude to my mentor, Name of Faculty Mentor, for guidance, timely feedback and encouragement throughout this project, and to the Head of the Department, Name of the Head of Department, and the faculty of MPSTME, SVKM's NMIMS (Deemed-to-be University), for the academic environment that made this work possible.

This work builds on the open resources of the Hugging Face Hub, whose model cards and metadata are the data of this study, and on free-tier services from Pinecone (vector search, hosted embeddings and re-ranking), Groq (language-model inference) and the open-source communities behind LangGraph, Streamlit and the Hugging Face libraries. I thank the authors of the papers cited in this report; the evaluation would not have been possible without their published methods.

Disclosure of tools. The software was developed with the assistance of an AI coding assistant (Claude Code, Anthropic), which wrote and ran code under my direction. The relevance grades and the explanation-faithfulness scores reported in Chapter 4 were produced by independent language-model judges, not by human annotators; this is stated wherever those numbers appear. All design decisions, the choice of scope and the interpretation of results are my own responsibility.

Student:

NAME	ROLL NO.	SAP ID
Aditya Rane	Roll No.	SAP ID

ABSTRACT

Choosing a pre-trained model on the Hugging Face Hub is a constrained-selection problem. A developer wants "a Marathi named-entity model under 500 MB that I can use commercially", and four of those five requirements are hard constraints (task, language, size and licence rights) that a model either meets or does not. Keyword search cannot read the soft part of such a request. The Hub's own filters match a licence *string* rather than a right. A plain retrieval-augmented generation (RAG) pipeline recommends whatever sounds relevant: in this work it broke at least one stated constraint on half of its recommendations.

This report presents **HF Model Finder**, a constraint-verified RAG system. A language model only parses the request into constraints and later writes a cited explanation; every pass, fail or unknown verdict is decided by deterministic code that reads model metadata, never model output or card text. Missing metadata is shown as an explicit *unknown* instead of being guessed or silently dropped. The system is a LangGraph pipeline with clarifying questions, an editable reading of the request and a prompt-injection defence. It searches 15,991 models across eleven tasks (speech, translation, text, vision, generation, summarization and question answering) with a deliberate emphasis on Indian languages, using hosted multilingual embeddings in Pinecone and a Streamlit interface.

The system was evaluated against naive RAG, BM25 with filters and the Hub's own filters on 100 queries written before any tuning. On 75 normal queries it recommended no model that fails a requested constraint (0.0% against 50.4% for naive RAG and 25.9% to 32.8% for the Hub filters) and 83.3% of its recommendations were fully verified (29.1% for naive RAG). It asked a clarifying question on all 15 vague queries and never claimed verification when no model in the corpus could pass. In a blind relevance study graded by independent language-model judges (not humans), it had the highest mean grade (1.28 of 2). The report states its limits plainly: 27% to 30% of models have unknown size, relevance and faithfulness were judged by language models, and the six added tasks were checked by smoke tests rather than the 100-query study.

TABLE OF CONTENTS

Topics **Page**

List of Figures ..... **vi**

List of Tables ..... **viii**

Abbreviations ..... **x**

Chapter 1 Introduction **1**

1.1 Background of the project topic 1
1.2 Motivation and scope of the report 3
1.3 Problem statement 4
1.4 Salient contributions 5
1.5 Organization of report 6
1.6 Standards followed 6

Chapter 2 Literature survey **8**

2.1 Introduction to the overall topic 8
2.2 Exhaustive literature survey 9
2.3 Comparison of the surveyed work 14
2.4 Gap analysis 14
2.5 Problem definition 15
2.6 Summary 16

Chapter 3 Methodology, Implementation, Flowcharts and Algorithms **17**

3.1 Requirements and design principles 17
3.2 Block diagram 18
3.3 Hardware description 20
3.4 Software description 20
3.5 Data pipeline 21
3.6 Query pipeline 25
3.7 Algorithms 31
3.8 User interface design 33
3.9 Security, safety and ethics 33
3.10 Evaluation methodology 35
3.11 Verification and validation of the software 36
3.12 Development process and reproducibility 37

Chapter 4 Snapshots of the Project, Results and Discussion **39**

4.1 Application snapshots 39
4.2 Corpus statistics 50
4.3 Constraint extraction 51
4.4 Comparison with the baselines on normal queries 52
4.5 Vague queries, impossible queries and the no-verified slice 55
4.6 Relevance 56
4.7 Explanations, security, usability and performance 57

4.8 Discussion....59
4.9 Threats to validity and limitations....60

Chapter 5 Source Code....62
5.1 Repository layout....62
5.2 Verification....63
5.3 Normalisation: rights and size....64
5.4 Retrieval....65
5.5 The query graph....66
5.6 Constraint extraction and the deterministic backstop....69
5.7 Defence against hostile card text, and explanation....71
5.8 Token-budget throttle....72
5.9 Build-on information....72
5.10 Evaluation metrics....73
5.11 Rendering a result....74

Chapter 6 Conclusion and Future Scope....76
6.1 Work carried out....76
6.2 Conclusions....76
6.3 Limitations....77
6.4 Future scope....77
6.5 Closing remark....79

References....80

Appendix A: Soft Code Flowcharts....84
A.1 Verification of one model....84
A.2 Metadata filter....85
A.3 Size rule....85
A.4 Constraint extraction....86
A.5 Language-model call with token throttle....87

Appendix B: Data Sheets....88
B.1 Datasheet of the model snapshot....88
B.2 Models per task....89
B.3 Licences in the snapshot....89
B.4 Size summary....90
B.5 Vector index....90
B.6 Models and service limits....90
B.7 Configuration values that affect results....90

Appendix C: List of Components....92

Appendix D: List of Papers Presented and Published....93

LIST OF FIGURES

Fig No.	Name of the figure	Page No.
1.1	A request combines hard constraints, which code can verify, with a soft preference, which only meaning can rank.	4
2.1	Six strands of prior work surveyed in this chapter and the gap between them.	9
3.1	Block diagram of HF Model Finder: the offline data pipeline (left) and the online query pipeline (right). Orange boxes are the only places a language model is used.	19
3.2	The offline data pipeline: from the Hub API to the index and snapshot.	22
3.3	The LangGraph query pipeline and its branches. Clarify and relax end a run; the interface re-invokes the graph with the user's answer.	26
3.4	Verify wide, show narrow: from 48,179 chunks to five models.	28
3.5	Per-constraint verdicts and their combination into one verdict and a rank.	30
3.6	Defence in depth against hostile card text: verdicts never depend on the language model.	34
3.7	Design of the evaluation.	35
4.1	Start page with example queries, the search box and the comparison switch.	40
4.2	Side-by-side comparison for a small Hindi sentiment model under a commercial licence (first three results of each column shown).	41
4.3	A vague request ("I want a Hindi model") is answered with a question, not a guess.	42
4.4	Results after the user chose "Detect sentiment / classify text".	42
4.5	The banner "No model fully verified" when every candidate has an unknown, for a request no model can fully satisfy.	43
4.6	Translation direction known from the model name, and flagged as such.	44
4.7	The Try it and Build on it panels for a Tamil named-entity model that the user wants to modify.	45
4.8	Image classification with a commercial-use requirement: vision models are verified in the same way.	46
4.9	Summarization with a commercial-use requirement.	47
4.10	An out-of-scope request is refused, with nothing shown rather than a model for a different task.	47
4.11	The editable reading ("Edit what we understood") and the glossary.	48
4.12	The Examples tab: more than 25 queries grouped by task.	49
4.13	Check-a-model mode for a model in the snapshot.	50
4.14	Models per task in the snapshot, split by whether the size is known.	51
4.15	Share of recommended models that are fully verified, unknown or violating, by system.	53

Fig No.	Name of the figure	Page No.
4.16	Recommendations that fail each constraint, by baseline. Ours and keyword search: 0.0 on every constraint.	54
4.17	Recommendations on the 12 queries for which no model in the corpus fully passes.	56
4.18	Relevance results from the blind study (language-model judges).	56
4.19	Try-it smoke test: small models for which the generated snippet ran, by task.	58
A.1	Flowchart of verify, overall and rank_key.	84
A.2	Flowchart of the metadata filter construction.	85
A.3	Flowchart of the size rule (D20).	86
A.4	Flowchart of constraint extraction and the deterministic backstop.	86
A.5	Flowchart of a language-model call with cache, token throttle and fallback.	87

LIST OF TABLES

Table No.	Name of the table / listing	Page No.
1.1	Organization of the report.	6
1.2	IEEE and ISO/IEC/IEEE standards that informed the project, and where they are applied.	7
2.1	Comparison of selected related work against the requirements of constraint-verified model finding.	14
3.1	Software requirements, with the evidence that verifies each one.	17
3.2	Hardware and service environment.	20
3.3	Software stack.	21
3.4	Metadata written on every chunk of a model.	23
3.5	Licence-to-rights map (representative entries; the full map has 54).	24
3.6	Nodes of the query graph.	26
3.7	Vague size words and their fixed thresholds.	27
3.8	Verdict rules for each constraint.	29
Algorithm 3.1	Verification of one model and ranking of the pool	31
Algorithm 3.2	Metadata filter with unknowns kept, and widening k	32
Algorithm 3.3	Control flow of one run of the query graph	32
3.9	Interface elements.	33
3.10	Test layers and evidence.	37
4.1	Data-quality figures of the snapshot (percentages of models).	51
4.2	Constraint-extraction accuracy on the tuning sets.	52
4.3	Main results on the 75 normal queries (top five per query, one set of rules for all systems).	52
4.4	Violation rate (%) by task.	54
4.5	Models in the corpus that could answer each intended-impossible query.	55
4.6	Relevance at five, graded 0 to 2 by language-model judges (blind).	56
4.7	Latency of the pipeline.	59
4.8	Threats to validity and known limitations.	60
4.9	Evaluation queries inspected while debugging, and what changed.	61
5.1	Modules of the system.	62
Listing 5.1	src/verify.py (lines 1 to 123): the single place where pass, fail or unknown is decided.	63
Listing 5.2	src/normalize.py (lines 130 to 135): licence identifier to rights.	64
Listing 5.3	src/normalize.py (lines 200 to 262): translation direction, size rule and the reason a size is unknown.	65

Table No.	Name of the table / listing	Page No.
Listing 5.4	src/retrieve.py (lines 13 to 86): filter with unknowns kept, grouping, widening search.	65
Listing 5.5	src/graph.py (lines 36 to 208): state, routing and graph construction.	67
Listing 5.6	src/llm.py (lines 248 to 358): validation, instruction guard, backstop and extraction.	69
Listing 5.7	src/llm.py (lines 359 to 404): untrusted-text handling and the explanation call.	71
Listing 5.8	src/llm.py (lines 133 to 175): sliding-window token budget.	72
Listing 5.9	src/adapt.py (lines 22 to 60): rights notes and memory estimate.	72
Listing 5.10	eval/metrics.py (lines 21 to 64): violation, unknown and clarification rates.	73
Listing 5.11	eval/metrics.py (lines 90 to 119): Cohen's kappa, unweighted and quadratic-weighted.	74
Listing 5.12	app.py (lines 114 to 181): rendering the outcome of one graph run.	74
B.1	Composition of the snapshot by task.	89
B.2	Most frequent licence identifiers.	89
B.3	Vector index specification.	90
B.4	Models used and free-tier limits.	90
B.5	Selected configuration values.	91
C.1	Software components.	92
C.2	Hardware and hosted-service components.	92
C.3	Data artefacts in the repository.	92

ABBREVIATIONS

Abbreviation	Expansion
ANN	Approximate Nearest Neighbour
API	Application Programming Interface
ASR	Automatic Speech Recognition
BM25	Best Matching 25 (a probabilistic ranking function)
CPU	Central Processing Unit
CC-BY-NC	Creative Commons Attribution-NonCommercial licence
GGUF	GPT-Generated Unified Format (quantized model file format)
GPU	Graphics Processing Unit
HF	Hugging Face
HNSW	Hierarchical Navigable Small World (graph index)
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation (parameter-efficient fine-tuning)
MPSTME	Mukesh Patel School of Technology Management and Engineering
NER	Named-Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
RAG	Retrieval-Augmented Generation
RAM	Random-Access Memory
SDD	Software Design Description
TPM	Tokens Per Minute
UI	User Interface
V&V	Verification and Validation

Chapter 1
Introduction

A model repository is only useful if the right model can be found in it. The Hugging Face Hub now hosts more than a million pre-trained models, yet a developer who needs "a small Marathi named-entity model that I can use in a commercial product" must still translate that sentence into a task tag, a language tag, a licence filter and a size check, then read model cards one by one to find out which candidates actually qualify. This report describes **HF Model Finder**, a system that performs that translation and the checking automatically, and that is built so that its answers can be trusted on the points where trust matters most: size, licence rights, language and task.

The report is organised as follows. This chapter gives the background, motivation, problem statement and contributions of the work. Chapter 2 surveys the literature on retrieval-augmented generation, dense retrieval, language models that use tools, model documentation and Indic-language resources, and ends with the gap this project addresses. Chapter 3 describes the design and implementation: the block diagram, the hardware and software environment, the data pipeline, the query pipeline and its algorithms. Chapter 4 presents screenshots of the working application and the results of the evaluation, with a discussion. Chapter 5 lists the core source code, and Chapter 6 concludes and gives the scope for future work. The appendices contain the soft-code flowcharts, the data sheets, the list of components and the list of papers.

1.1 Background of the project topic

Retrieval-augmented generation. Large language models store a great deal of knowledge in their parameters, but that knowledge is frozen at training time, cannot be inspected and is sometimes simply wrong. Retrieval-augmented generation (RAG) addresses this by fetching relevant documents from an external store at question time and giving them to the model as context. Lewis *et al.* combined a pre-trained sequence-to-sequence generator with a dense retriever over Wikipedia and showed gains on knowledge-intensive tasks [1]. REALM trained the retriever and the language model together during pre-training [2], Fusion-in-Decoder let the generator read many retrieved passages at once [3], RETRO retrieved from a database of trillions of tokens [4], and Self-RAG taught a model to decide when to retrieve and to critique its own output [5]. The survey by Gao *et al.* organises this growing family into naive, advanced and modular RAG [6].

Retrieval and ranking. Modern RAG rests on a few retrieval building blocks. Dense passage retrieval encodes questions and passages into vectors and finds neighbours by similarity [7]; sentence encoders such as Sentence-BERT make such embeddings cheap to compute [8]; the probabilistic BM25 function remains a strong lexical baseline [9]; and a cross-encoder can re-rank a short list of candidates more accurately than the first-stage retriever [10]. For a multilingual audience, embedding models such as BGE-M3 and multilingual E5 map more than a hundred languages into one vector space [11], [12]. Large vector collections are searched with approximate nearest-neighbour indexes such as GPU-accelerated brute force [13] and hierarchical navigable small-world graphs [14], and the problem of combining vector search with attribute filters has its own literature [15].

Language models as controllers. All of these methods rest on the Transformer architecture [16] and on pre-trained encoders and generators such as BERT [17]. A second line of work uses a language model not as the source of answers but as a controller that calls other components: ReAct interleaves reasoning with actions [18], program-aided language models hand arithmetic to an interpreter instead of computing it in text [19], and HuggingGPT lets a language model choose models from the Hugging Face Hub to solve multi-step tasks [20]. The principle that the model should *plan* and a deterministic component should *compute* is central to this project.

Model repositories, documentation and licences. Model cards [21] and datasheets [22] were proposed so that the people who reuse a model or dataset can learn what it is for and how it behaves. In practice, documentation on a public hub is uneven: an empirical study of pre-trained-model reuse on the Hugging Face registry reports that missing or inconsistent documentation and attributes make selection difficult [23], and an audit of dataset licences found that licence information on hosting sites is frequently omitted or mis-categorised [24]. Licences also differ in kind: a permissive licence allows commercial use and modification, a non-commercial licence does not, and responsible-AI and community licences attach conditions that follow the model into derivatives. A tool that recommends models therefore has to *interpret* licences, not just display them.

Indic languages. India has twenty-two scheduled languages and many more spoken ones, but language technology for them is concentrated in a small number of resources: monolingual corpora, benchmarks and pre-trained models [25], [26], a translation system covering all twenty-two scheduled languages [27], and multilingual models such as Whisper for speech [28]. Many community fine-tunes of these models are published on the

Hub with little documentation and with low download counts, so they sit in the long tail of any popularity-ranked search. Multilingual encoders such as XLM-R cover Indic languages without carrying a tag for each of them [29], which makes simple language filters unreliable.

Evaluating and securing RAG systems. Automatic evaluation of RAG has produced reference-free measures such as faithfulness and context relevance [30], and strong language models are now widely used as judges of other systems' output, with measurable agreement with human raters [31]. Retrieval benchmarks show that lexical baselines are hard to beat out of domain [32], and long-context studies show that models use information in the middle of a long prompt poorly [33]. Generated text can contain unsupported claims [34], and when retrieved text is untrusted it can carry instructions that hijack the application, a threat known as indirect prompt injection [35]. Agreement between graders is conventionally reported with Cohen's kappa [36] and interpreted with the bands of Landis and Koch [37].

1.2 Motivation and scope of the report

The request quoted above illustrates the motivation. It has one *soft* part, "works well on noisy social-media text", which is a matter of meaning and can be ranked, and several *hard* parts that are matters of fact (Fig. 1.1). Each existing approach fails on one of them. Keyword search cannot read meaning. The Hub's own filters work on metadata, but its licence filter matches a licence identifier and does not know that a CC-BY-NC licence rules out commercial use or that a Llama-style licence imposes conditions on derivatives. A plain RAG pipeline retrieves models whose cards *sound* relevant and lets a language model recommend them; because vector similarity measures topical closeness and not logical compliance, such a pipeline recommends models that are too large, in the wrong language or non-commercial. In this project's own evaluation (Chapter 4), a naive RAG baseline recommended a model that fails at least one stated requirement for 50.4% of its recommendations.

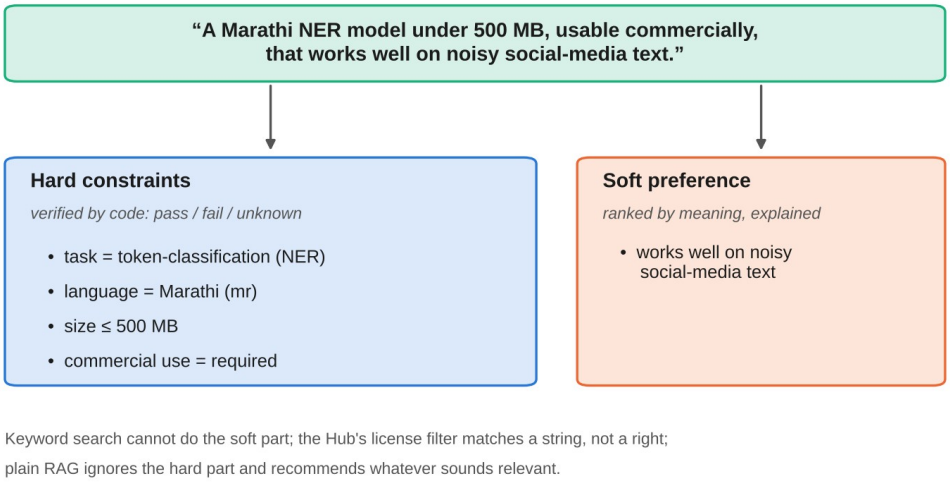


Fig. 1.1. A request combines hard constraints, which code can verify, with a soft preference, which only meaning can rank.

Three further observations shape the scope. First, metadata on the Hub is often missing. About 27% of the models in the original five-task corpus have no determinable size, because their weights are only published in a format such as GGUF or under a non-standard file name; a system that silently drops such models loses good answers, and one that guesses invents facts. Second, the people who most need help are often beginners who do not know the vocabulary of tasks and licences and who, after finding a model, still need to run it and perhaps adapt it. Third, Indian-language models are numerous but poorly surfaced.

The scope of this report is therefore a *model finder*: given a natural-language request, return a short list of models from a snapshot of the Hub, each with a per-requirement verdict, a short cited explanation, a starter code snippet and information on whether the model can be adapted. It covers eleven tasks (automatic speech recognition, translation, text classification, token classification, sentence similarity, image classification, object detection, image segmentation, text generation, summarization and question answering). It does **not** run or chat with a model, does not provide legal advice (rights are read from the *declared* licence), does not rank by benchmark scores, and refuses requests for image or music generation, captioning, optical character recognition, text-to-speech and video, which are outside the tasks indexed.

1.3 Problem statement

To design, implement and evaluate a retrieval-augmented system that, given a natural-language request that mixes hard constraints (task, language, size,

commercial-use and modification rights) with a soft preference, returns Hugging Face models such that every returned model is verified in code against every stated hard constraint; that represents missing metadata as an explicit *unknown* instead of a guess or a silent omission; that asks for clarification when the request is under-specified; and that is measurably better on constraint satisfaction than naive RAG, keyword search with filters and the Hub's own filters, with particular attention to Indian-language models.

The problem decomposes into five questions, each answered by a measurement in Chapter 4: whether verification reduces constraint violations; whether it costs relevance; whether the system asks when it should and avoids false assurance when nothing fully qualifies; whether the generated explanations are grounded in the model cards; and whether it is better than using the Hub's filters directly.

1.4 Salient contributions

The main contributions of this work are the following.

1. **A design principle, "the LLM parses, code verifies".** The language model is used only to turn the request into a constraint record and to write an explanation. Every pass, fail or unknown verdict is computed by plain functions from model metadata, so a verdict can never be changed by card text or model output.
2. **A three-valued treatment of missing data.** Each requested constraint evaluates to pass, fail or unknown. Retrieval filters are written so that models with unknown fields are kept, then verified and ranked below fully verified models, and the answer states when no returned model is fully verified.
3. **Rights, not licence strings.** A licence-to-rights map interprets 54 licence identifiers as commercial use and modification rights (yes, no, conditional or unknown), and a size rule (the sum of the main weight files) gives a reproducible model size.
4. **A verify-wide, show-narrow retrieval design.** Because semantic search cannot see size, direction or licence, the system verifies a pool of 100 models and shows five; this raised fully verified answers from 72.3% to 83.3% in the evaluation.
5. **A LangGraph pipeline with guided interaction.** Clarifying questions, an editable plain-language reading of the request, relax-and-retry and abstention paths, an out-of-scope refusal, ready-to-run snippets, fine-tuning memory estimates and a check-a-model mode make the tool usable by beginners.

6. **A prompt-injection defence.** Card text is wrapped as data, escaped, and its citations filtered; a poisoned-card test confirmed that verdict tables are unaffected.
7. **A reproducible evaluation.** One hundred queries written before tuning, five systems judged by one rule set, a blind relevance study with two grading passes, an explanation-faithfulness study, a poisoned-card test and a smoke test of the generated snippets, with the exact limits of each stated.
8. **A coverage extension to eleven tasks** that includes vision, text generation, summarization and question answering, with an honest record of which parts of the evaluation cover it.

1.5 Organization of report

Table 1.1 lists the chapters and what each contains.

Table 1.1. Organization of the report.

Chapter	Content
1 Introduction	Background, motivation and scope, problem statement, contributions, organization, standards followed
2 Literature survey	Background from 15 or more references; a survey of fourteen papers in paragraph form; a comparison table; the gap and the problem definition
3 Methodology	Requirements, block diagram, hardware and software description, data pipeline, query pipeline, algorithms, user interface, security, evaluation method, testing
4 Snapshots and results	Screenshots of the application; corpus statistics; quantitative results and discussion; threats to validity
5 Source code	Listings of the core modules
6 Conclusion and future scope	Summary of the work, conclusions, limitations and future work
Appendices A to D	Soft-code flowcharts; data sheets; list of components; list of papers presented and published

1.6 Standards followed

The guidelines for this report ask that the IEEE standards followed in designing the project be named at appropriate places. The project is a student software system, not a certified product, so the standards below are used as *guides* for documentation and process, and the report does not claim formal compliance with any of them. Table 1.2 states where each one informed the work.

Table 1.2. IEEE and ISO/IEC/IEEE standards that informed the project, and where they are applied.

Standard	Used for	Where in this report
ISO/IEC/IEEE 29148:2018, requirements engineering [38]	Writing requirements as uniquely identified, verifiable "shall" statements, with a stated source and a verification method	Section 3.1 (Table 3.1)
IEEE 1016-2009, software design descriptions [39]	Describing the design through several views: context, composition, logic and algorithm, interface and information	Sections 3.2 to 3.8 and Appendix A
IEEE 1012-2016, verification and validation [40]	Planning verification activities and their evidence: unit tests, data checks, live-service checks and evaluation	Section 3.10 and Chapter 4
ISO/IEC/IEEE 29119-3:2021, test documentation [41]	Organising the test catalogue with identified cases, expected results and recorded outcomes	Section 3.10
IEEE 7000-2021, ethical concerns in system design [42]	Treating transparency, honesty about unknowns, user autonomy and non-misleading claims as design requirements	Sections 3.1, 3.9 and 4.8

Chapter 2
Literature survey

This chapter places the project in the literature. Section 2.1 introduces the overall topic with more than fifteen references. Section 2.2 is a paragraph-style survey of the individual papers that bear most directly on the problem, grouped by theme and read in the order of the reference list. Section 2.3 compares them in a table, and Sections 2.4 and 2.5 identify the gap that is left and state the problem that follows from it.

2.1 Introduction to the overall topic

The topic sits at the meeting point of four areas, shown in Fig. 2.1. The first is **retrieval-augmented generation**, which grounds a language model in documents fetched at question time [1]–[6]. The second is **retrieval itself**: dense passage retrievers [7], sentence encoders [8], the lexical BM25 baseline [9], cross-encoder re-ranking [10], multilingual embedding models [11], [12], and the approximate nearest-neighbour indexes and filtered search methods on which production vector stores are built [13]–[15]. The third is the use of **language models as controllers** of tools and other models [18]–[20]. The fourth is the **documentation and governance of models**: model cards, datasheets, empirical studies of reuse and audits of licensing [21]–[24].

Two cross-cutting concerns complete the picture. The first is **language coverage**: Indic-language corpora, benchmarks and models [25]–[27], speech and multilingual encoders that cover many languages without tagging each one [28], [29]. The second is **trustworthy evaluation and security**: reference-free RAG metrics [30], language models as judges [31], zero-shot retrieval benchmarks [32], the limits of long contexts [33], hallucination [34], indirect prompt injection [35] and the statistics of rater agreement [36], [37].

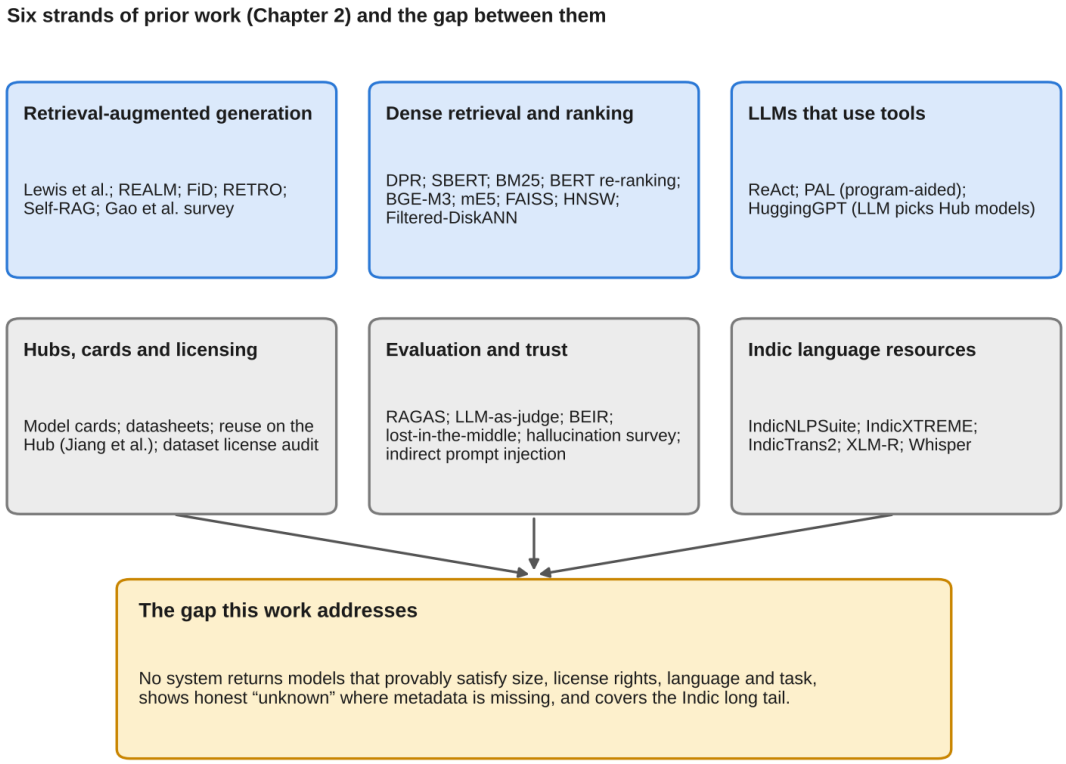


Fig. 2.1. Six strands of prior work surveyed in this chapter and the gap between them.

Taken together, this body of work shows how to *retrieve* documents by meaning, how to let a language model *use* a tool, and how to *document* a model. It does not show how to *select a model that provably satisfies a set of stated requirements* when the metadata that would prove it is incomplete. That is the question this project asks.

2.2 Exhaustive literature survey

2.2.1 Retrieval-augmented generation

Lewis *et al.* [1] introduced the name and the recipe. Their RAG models couple a pre-trained sequence-to-sequence generator with a dense retriever that queries a vector index of Wikipedia, and they study two formulations: one in which the same retrieved passages condition the whole output, and one that may use different passages for each token. The retriever and generator are fine-tuned together. The models set the state of the art on several open-domain question-answering benchmarks and produced more specific and more factual text than a parametric-only generator, with the practical advantage that the knowledge base can be updated by replacing the index. For the present problem the lesson is architectural: the retrieval index is a separate, replaceable component. The limitation is that the retrieved items are free-text passages and nothing in the method can require that a retrieved item satisfy a structured condition.

Guu *et al.* [2] (REALM) brought retrieval into pre-training. A masked language model is trained with a latent retriever that attends over a large corpus, and gradients flow through the retrieval step over millions of documents. On open-domain question answering the approach outperformed earlier explicit and implicit knowledge-augmented models by four to sixteen points of absolute accuracy. REALM shows that retrieval quality is learnable, but it still treats relevance as the only property of a retrieved document.

Izacard and Grave [3] (Fusion-in-Decoder) addressed how a generator should read many retrieved passages. Each passage is encoded independently together with the question and the decoder attends over all the encodings at once, so the cost grows linearly with the number of passages and the answer quality keeps improving as more passages are supplied. This supports the design choice made here of retrieving a *wide* candidate set before narrowing it.

Borgeaud *et al.* [4] (RETRO) scaled the retrieval database to about two trillion tokens and conditioned a transformer on chunks retrieved by similarity to the preceding text, using chunked cross-attention. A model with 25 times fewer parameters matched much larger models on language-modelling benchmarks. RETRO makes the point that a good retrieval store can substitute for parameters, which motivates keeping model facts (size, licence) in a store rather than expecting an LLM to recall them.

Asai *et al.* [5] (Self-RAG) trained a model to decide *when* to retrieve and to critique its own output with special reflection tokens. It outperformed ChatGPT and retrieval-augmented Llama-2 chat on several tasks and improved factuality and citation accuracy in long-form generation. Self-RAG is the closest prior work in spirit to a system that checks its answers, but the critique is itself a learned judgement of the language model and therefore not a guarantee; in the present project the checks that matter are exact comparisons performed in code.

Gao *et al.* [6] surveyed the field and organised it into naive RAG (retrieve, then generate), advanced RAG (pre-retrieval and post-retrieval processing such as query rewriting and re-ranking) and modular RAG (components that can be recomposed and routed). The system built here is a modular RAG with an additional stage, *verification*, that the survey's taxonomy does not name.

2.2.2 Retrieval, ranking and filtered search

Karpukhin *et al.* [7] showed that a dual-encoder trained on a modest number of question-passage pairs can outperform BM25 by nine to nineteen points of top-20 accuracy in passage retrieval, which established dense retrieval as the default first stage for RAG. **Reimers and Gurevych [8]** (Sentence-BERT) made semantically meaningful sentence embeddings cheap to compare: finding the most similar pair among ten thousand sentences dropped from about 65 hours with a BERT cross-encoder to about five seconds. **Robertson and Zaragoza [9]** review the probabilistic relevance framework behind BM25 and BM25F, which remains the standard lexical baseline; the keyword baseline in this project is a BM25 index. **Nogueira and Cho [10]** applied BERT as a cross-encoder to re-rank passages and improved the previous state of the art on MS MARCO by 27% (relative) in MRR@10, which is the reason a re-ranker follows the vector search here.

Chen *et al.* [11] (BGE M3-Embedding) supports more than one hundred languages, three retrieval functions (dense, multi-vector and sparse) and inputs up to 8,192 tokens, trained with self-knowledge distillation; the re-ranker used in this project, bge-reranker-v2-m3, belongs to the same family. **Wang *et al.* [12]** describe the multilingual E5 models, trained by contrastive pre-training on one billion multilingual text pairs and then fine-tuned on labelled data; the *large* member is the embedding model of this project, chosen because queries and cards may be in Hindi, Marathi or English.

Gollapudi *et al.* [15] (Filtered-DiskANN) tackle the problem that matters most for constrained search: how to combine nearest-neighbour search with attribute filters. Filtering before the search is exact but slow for selective filters, and filtering afterwards can return nothing; their graph index is built with the labels in mind and gives a better accuracy-throughput trade-off than either. The work assumes that every item *has* its labels. Production vector stores expose a simpler model, a metadata filter that keeps only items matching a condition, and with that model an item that lacks the field is silently excluded. How to treat missing fields is left to the application, and it is a central design question here. Underneath such stores are the index structures of Johnson *et al.* [13] (billion-scale brute-force and compressed search on GPUs) and Malkov and Yashunin [14] (hierarchical navigable small-world graphs), which this project uses indirectly through the hosted service and directly through an exact local index with the same filter semantics.

2.2.3 Language models that use tools

Yao *et al.* [18] (ReAct) let a language model alternate between reasoning traces and actions such as calling a search API, which reduced hallucination on question answering and improved success on interactive decision-making benchmarks. **Gao *et al.* [19]** (PAL) go a step further in separating roles: the model writes a program as its reasoning and a Python interpreter executes it, and this beat chain-of-thought prompting with a much larger model on arithmetic reasoning. PAL is the clearest precedent for the principle adopted here: where an answer is a computation, the language model should write the *question* and code should compute the *answer*.

Shen *et al.* [20] (HuggingGPT) is the closest prior work on the *task*. A language model acts as a controller that plans a request into sub-tasks, selects an expert model from the Hugging Face Hub for each, runs them and summarises the results. Model selection is done by giving the language model textual descriptions of candidate models. There is therefore no verification of size, licence or language: the choice is a judgement of the language model over text, and the approach has no way to report that a fact is missing.

2.2.4 Model documentation, reuse and licensing

Mitchell *et al.* [21] proposed model cards as short documents that accompany a trained model and state its intended use, evaluation conditions and limitations; **Gebru *et al.* [22]** proposed datasheets for datasets on the same principle. Both are the reason Hub metadata exists at all and both are the source of the data used here, but both are conventions, not schemas, so the fields a recommender needs are present only when authors choose to write them.

Jiang *et al.* [23] studied how practitioners reuse pre-trained models on the Hugging Face registry. From interviews with twelve practitioners and measurements of the registry they identified three recurring challenges: missing attributes, discrepancies between claimed and actual performance, and model risks. The first of these is what the *unknown* verdict in this project represents. **Longpre *et al.* [24]** audited more than 1,800 text datasets and found frequent mis-categorisation of licences on widely used hosting sites, with licences omitted in more than 70% of cases and errors in more than 50%. Although the audit concerns datasets, it shows that a licence shown on a hub page cannot be taken at face value, which is why this project reports rights as *declared* and states that it is not legal advice.

2.2.5 Indic-language resources

Kakwani *et al.* [25] (IndicNLPSuite) released monolingual corpora, evaluation benchmarks and pre-trained models for Indian languages. **Doddapaneni *et al.* [26]** extended this to IndicCorp v2, with 20.9 billion tokens in 24 languages, the IndicXTREME benchmark of nine tasks over 20 languages, and the IndicBERT v2 model. **Gala *et al.* [27]** (IndicTrans2) released parallel corpora, an n-way benchmark and open translation models covering all 22 scheduled Indian languages. **Radford *et al.* [28]** (Whisper) trained speech models on 680,000 hours of multilingual supervised data and showed that they generalise to standard benchmarks without fine-tuning, and **Conneau *et al.* [29]** (XLM-R) trained a cross-lingual encoder at scale for around one hundred languages. These are the foundations on which the Indic models in the Hub are built, but they are research artefacts. The thousands of community fine-tunes derived from them are what a developer actually downloads, and no cited work helps a developer choose among them.

2.2.6 Evaluation, trust and security

Es *et al.* [30] (RAGAs) defined reference-free measures for RAG, notably *faithfulness* (are the claims in the answer supported by the retrieved context) and context relevance; the explanation-faithfulness study in this project follows the same idea but is carried out by grading the claims directly. **Zheng *et al.* [31]** examined language models as judges, documented position, verbosity and self-enhancement biases, and showed that strong judges agree with human preferences more than 80% of the time, which is the justification, and also the caveat, for the judged relevance study in Chapter 4. **Thakur *et al.* [32]** (BEIR) found that BM25 is a robust zero-shot baseline that dense retrievers often fail to beat, which is why BM25 with the same filters is one of the baselines here. **Liu *et al.* [33]** showed that language models use information in the middle of long contexts poorly, and **Ji *et al.* [34]** surveyed hallucination in generated text; together they motivate keeping prompts short and keeping verdicts out of generated text.

Greshake *et al.* [35] showed that when an application retrieves text that an adversary can write, instructions hidden in that text can take control of the language model, an attack they call indirect prompt injection. Model cards are exactly such text: anyone can publish one. The defence adopted in this work (delimiting and escaping card text, filtering citations, and above all making the verdict independent of the language model) responds directly to this paper. Finally, **Cohen [36]** defined the kappa coefficient of chance-corrected agreement

and **Landis and Koch [37]** gave the conventional interpretation of its values; both are used to report agreement between the two grading passes.

2.3 Comparison of the surveyed work

Table 2.1 compares the main works against the properties that this project needs. A "yes" means the work provides the property as part of its method; "no" means it does not, which is not a criticism of work with a different aim.

Table 2.1. Comparison of selected related work against the requirements of constraint-verified model finding.

Work	What it provides	Guaranteed hard constraints	Handles missing metadata	Gap for this problem
RAG [1]	Dense retrieval plus generation over a text corpus	no	no	Relevance only; no structured conditions
REALM, FiD, RETRO [2]–[4]	Learned retrieval; fusing many passages; very large stores	no	no	Same as RAG
Self-RAG [5]	Adaptive retrieval and self-critique	no (learned critique)	no	Critique is a model judgement
Dense retrieval and BM25 [7], [9]	First-stage retrieval	no	n/a	No constraint semantics
Cross-encoder re-ranking [10]	More accurate ordering of a short list	no	n/a	Orders by relevance only
Filtered ANN [15]	Search combined with label filters	yes, for labelled items	assumes labels present	Missing labels are not modelled
ReAct, PAL [18], [19]	Reasoning with actions; code as reasoning	partly (code computes)	no	Not a selection method
HuggingGPT [20]	LLM selects Hub models by description	no	no	Selection by text; no verification
Model cards, datasheets [21], [22]	Documentation convention	n/a	n/a	Optional fields, no schema
Reuse study, licence audit [23], [24]	Evidence that attributes and licences are missing or wrong	n/a	identifies the problem	No tool or remedy
Indic resources [25]–[27]	Corpora, benchmarks, models	n/a	n/a	Research artefacts, not a finder
RAGAs, LLM judges [30], [31]	Reference-free metrics; judging	n/a	n/a	No measure of constraint violation
Indirect injection [35]	Threat model for retrieved text	n/a	n/a	Motivates the defence
This work	Parse by LLM, verify in code, unknown as a verdict	**yes, by construction**	**yes**	Evaluated on 100 queries

2.4 Gap analysis

Reading the literature against the problem of Chapter 1 leaves five gaps.

1. **No selection method guarantees hard constraints.** RAG and its variants optimise relevance, and the systems that let a language model choose a model [20] do so from descriptions. A recommendation that must be "under 500 MB and usable

commercially" needs an exact comparison against metadata, which none of these provide.

2. **Missing metadata is not modelled.** Filtered search assumes the labels exist [15]; production metadata filters drop items that lack the field; and the reuse and licensing studies [23], [24] show that missing and wrong attributes are the normal case. A selection method needs a verdict for "the data does not say".
3. **Licences are displayed, not interpreted.** The documentation literature [21], [22] and the Hub filters treat a licence as a label; whether a licence permits commercial use or modification, or imposes conditions on derivatives, is left to the reader.
4. **The Indic long tail is under-served.** Corpora, benchmarks and flagship models exist [25]–[27], but community fine-tunes with few downloads and multilingual models that carry no language tag [28], [29] are invisible to popularity-ranked and tag-filtered search.
5. **Evaluation ignores constraint violation.** RAG metrics measure faithfulness and relevance [30]; judges approximate preferences [31]. Neither measures how often a recommended item breaks a stated requirement, which is the property a model finder must have.

2.5 Problem definition

The gaps lead to the problem stated in Chapter 1 and to six objectives for the work:

1. Parse a natural-language request into a structured constraint record with a language model, and validate the result in code.
2. Verify every requested constraint in code against metadata, with the three verdicts pass, fail and unknown, and keep the verdict independent of any generated text.
3. Retrieve in a way that keeps models with unknown fields, rank fully verified models first, and state when none is fully verified.
4. Interpret licences as commercial-use and modification rights, and interpret size by a reproducible rule.
5. Cover the Indic long tail and multilingual models, ask clarifying questions for under-specified requests, and give beginners a starter snippet and adaptation information.
6. Evaluate the system against naive RAG, keyword search with filters and the Hub's own filters on queries fixed before tuning, with the limits of the evaluation stated.

2.6 Summary

The literature supplies the components (dense and lexical retrieval, re-ranking, filtered search, tool-using language models, documentation conventions and evaluation methods) and the evidence that metadata on public hubs is incomplete and sometimes wrong. It does not supply a method that combines them into a model finder whose verdicts are exact and whose unknowns are explicit. The next chapter describes the design that fills that gap.

Chapter 3
Methodology, Implementation, Flowcharts and Algorithms

This chapter explains how the system is built. It states the requirements and design principles (Section 3.1), shows the block diagram (3.2), describes the hardware and software environment (3.3 and 3.4), and then follows the data from the Hugging Face Hub to the index (3.5) and a user request from the browser to the answer (3.6). The algorithms are given as pseudocode with their cost and correctness arguments (3.7). The user interface, the security design and the evaluation method follow (3.8 to 3.10), and the chapter closes with how the software itself was tested and how the work can be reproduced (3.11 and 3.12). The design is described in the views recommended for a software design description [39]: context and composition (3.2 to 3.4), information (3.5), logic and algorithm (3.6 and 3.7), interface (3.8) and the decisions that cut across them (3.9 to 3.12).

3.1 Requirements and design principles

The requirements were written as numbered, verifiable statements with a rationale and a verification method, following the practice of the requirements-engineering standard [38]. Table 3.1 lists them. A requirement is considered met only if the last column names evidence that exists in the repository or in Chapter 4.

Table 3.1. Software requirements, with the evidence that verifies each one.

ID	The system shall	Rationale	Verification
FR-1	accept a natural-language request and show, in plain words, how it was understood, editable by the user	Beginners cannot judge a result without knowing what was asked	UI test; Fig. 4.1, Fig. 4.11
FR-2	extract task, languages, translation direction, size limit, commercial and modification requirements and a soft preference, and validate every field against a fixed schema	Model output is untrusted input	Unit tests; 29-case extraction set
FR-3	ask at most two clarifying questions when the task, or the language pair of a translation, is missing	Guessing silently changes the question	Clarification rate on 15 vague queries
FR-4	decide pass, fail or unknown for every requested constraint in code, from metadata only	Verdicts must not depend on generated text	Unit tests; violation rate
FR-5	never present a model that fails a requested constraint as a recommendation	The core promise of the tool	Violation rate on 75 queries
FR-6	show *unknown* where metadata is missing, and not guess or omit	Guessing invents facts; omitting hides good answers	Unknown rate; data-quality report
FR-7	state when no returned model is fully verified	Prevents false assurance	No-verified slice (12 queries)
FR-8	explain each model with citations restricted to the candidates, without stating licence, size or language facts	Explanations must be grounded and cannot override verdicts	Faithfulness study; poisoned-card test
FR-9	give a starter snippet, rights notes, lineage and a fine-tuning memory estimate	Finding a model is the first step of using it	Unit tests; Try-it smoke test

ID	The system shall	Rationale	Verification
FR-10	check any model in the snapshot against stated requirements	Serves reviewers of models already in use	Unit tests; Fig. 4.13
FR-11	refuse requests outside its eleven tasks, without searching	Wrong-task models must never appear	Unit tests; Fig. 4.10
FR-12	offer to relax one constraint or abstain when no candidate is viable, at most once	Bounded recovery	Unit tests (paths reachable only in tests)
NFR-1	be reproducible from a dated, checksummed snapshot, one configuration file and cached model calls	Results must be re-derivable	Snapshot checksum; frozen-evaluation reproduction
NFR-2	treat card text as untrusted data and keep secrets out of the repository	Anyone can publish a card	Poisoned-card test; secret scan
NFR-3	answer in a median of about 2 s without the explanation, and stay inside free-tier token limits with it	Interactive use on free services	Latency measurements
NFR-4	state its limits: rights are declared and not legal advice; labels were produced by language-model judges	Honest, non-misleading claims	Sections 4.8 and 6.2

Four design principles follow from these requirements and run through the rest of the chapter.

1. **The language model parses; code verifies.** The model's only jobs are to turn a request into a constraint record (FR-2) and to write an explanation (FR-8). Everything that decides a verdict (FR-4 to FR-7) is a plain function.
2. **Unknown is a first-class value.** Each constraint has three outcomes, and a missing field is never replaced by a default.
3. **Filters are loose and verification is exact.** The vector store's filter only narrows the search; it is written to keep unknowns, and the verification step is the single place where pass or fail is decided.
4. **Every deployed claim has a measurement.** Each design choice that could be wrong (the verify pool, the extraction backstop, the size rule) is backed by a number or listed as a limitation.

3.2 Block diagram

Fig. 3.1 shows the system. The left column is the **offline** pipeline that turns the Hub into an index; it is run once per snapshot. The right column is the **online** pipeline that answers a request. The only place the two meet is the index, which holds one vector and one copy of the model's metadata per chunk of a model card.

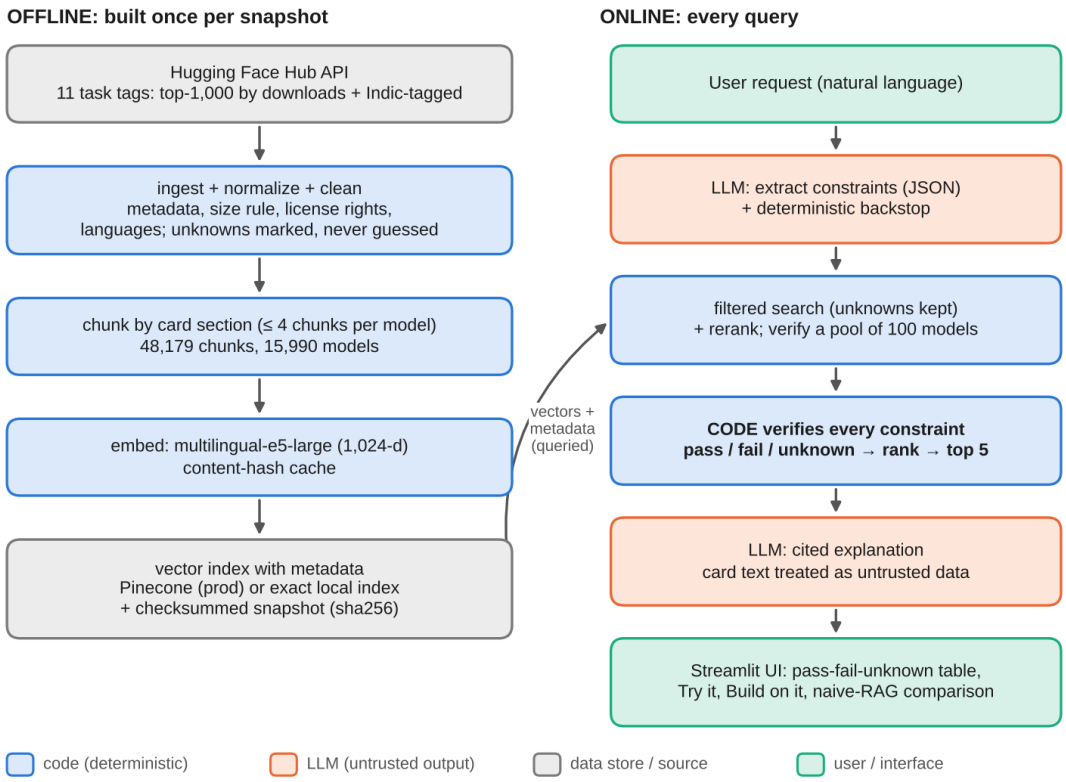


Fig. 3.1. Block diagram of HF Model Finder: the offline data pipeline (left) and the online query pipeline (right). Orange boxes are the only places a language model is used.

The blocks have the following responsibilities.

- **Hugging Face Hub API** supplies the raw metadata and card text of each model.
- **Ingest, normalise and clean** convert raw records into the metadata schema of Table 3.4, computing size, rights, languages and direction and marking every unknown as unknown.
- **Chunk and embed** split each card by section and convert each chunk into a 1,024-dimensional vector with a multilingual embedding model.
- **Index and snapshot** store the vectors with their metadata in Pinecone (production) or in an exact local index with the same filter semantics, and freeze the normalised records in a checksummed snapshot.
- **Extract** (language model plus deterministic backstop) turns the request into constraints.
- **Filtered search and re-rank** narrow the 48,179 chunks to a pool of 100 candidate models.
- **Verify** decides pass, fail or unknown for each requested constraint and ranks the pool.

- **Explain** (language model) writes a cited explanation of why each shown model suits the request; card text reaches it only as delimited data.
- **Streamlit interface** shows the verdict tables, explanations, snippets, adaptation information and the optional naive-RAG comparison.

3.3 Hardware description

The system needs no special hardware. All development, indexing and evaluation were run in a managed cloud container, and the heavy computation (embedding, re-ranking, language-model inference) is done by hosted services. Table 3.2 lists the environment and the resources each part consumes.

Table 3.2. Hardware and service environment.

Item	Specification and use
Development and evaluation host	Cloud container, Linux 6.x, 4 virtual CPUs, 16 GB RAM, no GPU; Python 3.13
Local data on disk	Normalised snapshot (about 16,000 records), raw card cache, embedding cache (128 MB), exact local index (376 MB)
Vector database	Pinecone Starter plan, serverless index `hf-model-finder` on AWS us-east-1, cosine metric, 1,024 dimensions, 48,179 vectors in namespace `prod`
Embedding and re-ranking	Pinecone-hosted `multilingual-e5-large` and `bge-reranker-v2-m3`; 4.05 million hosted tokens used to embed the whole corpus once; hosted re-ranking limited to 500 requests per month
Language model	Groq free tier, `gpt-oss-20b` (fallback `gpt-oss-120b`): 8,000 tokens per minute, 1,000 requests and 200,000 tokens per day
Offline language model	Ollama with `qwen2.5:7b-instruct` (about 4.7 GB, needs about 8 GB RAM) or `qwen2.5:3b` for 8 GB laptops; not run in the evaluation environment
Client	Any current web browser; the Streamlit server runs on a laptop or server with Python 3.10 or later and no GPU
Local re-ranker (fallback)	The same cross-encoder on CPU: about 103 s to load and about 43 s for 100 chunks, so it is capped at 40 chunks (about 20 s per query)

Two measured consequences of this environment shaped the design. First, the hosted re-ranker has a monthly request limit that was reached on 25 October 2026, so the application detects the quota error, keeps the vector-search order and says so on the page (Section 3.6.4). Second, the free language-model tier limits *tokens per minute* more tightly than requests, which led to the token throttle of Section 3.7.4 and to the budgeted evidence selection of Section 3.6.8.

3.4 Software description

Table 3.3 lists the software stack. Versions are those installed in the evaluation environment.

Table 3.3. Software stack.

Component	Version	Role
Python	3.13	Implementation language of all modules
LangGraph [43]	1.2	Query pipeline as a state graph with conditional edges
Streamlit [44]	1.66	Web interface; session state holds the graph state between runs
Pinecone client and hosted inference [45]–[47]	10.1	Vector store, metadata filters, embeddings and re-ranking
Groq client [48]	1.7	Language-model calls (JSON mode, low reasoning effort)
`huggingface_hub` [49]	2.2	Listing models by task and fetching metadata and cards
Transformers [50]	4.57 / 5.20	Used only by the Try-it smoke test to run generated snippets
BM25 keyword index	own code	Keyword baseline (Okapi BM25, k1 = 1.5, b = 0.75), about 100 lines
NumPy, PyYAML, requests	2.5 / 6.0 / 2.34	Numerics, configuration and HTTP
pytest	9.1	335 offline tests
Playwright, LibreOffice, matplotlib	–	Screenshots, document rendering and figures for this report

The code base comprises about 3,600 lines in `src/` and `app.py`, 900 in `eval/`, 700 in `scripts/` and 3,300 in `tests/`. A single `config.yaml` holds every number that influences results (model names, token budgets, chunk limits, retrieval widths, size-word thresholds, graph caps). Language-model providers are hidden behind one function so that changing from Groq to a local Ollama model is a matter of one environment variable.

3.5 Data pipeline

Fig. 3.2 summarises the offline pipeline. It is deterministic given the raw cache, resumable after any interruption, and every stage writes to disk before the next begins.

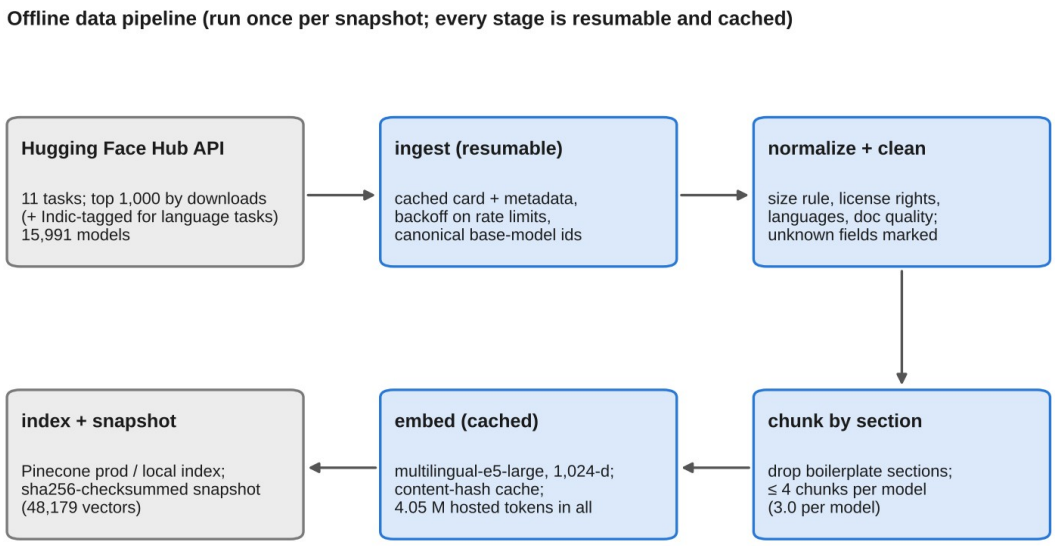


Fig. 3.2. The offline data pipeline: from the Hub API to the index and snapshot.

3.5.1 Candidate selection and ingestion

For each of the eleven tasks the pipeline lists the top 1,000 models by downloads. For the language tasks (speech recognition, translation, text and token classification, sentence similarity, summarization and question answering) it also adds every model tagged with one of ten Indic languages (Hindi, Marathi, Bengali, Tamil, Telugu, Gujarati, Kannada, Malayalam, Punjabi, Urdu), so that the low-download Indic long tail is represented; this sampling is skipped for the vision tasks, where language does not apply, and for text generation, where thousands of multilingual models carry Indic tags. Downloads are used only to *sample*, never to rank.

For every candidate the pipeline fetches the model's API record with file sizes [49] and its card in Markdown [51]. Responses are written atomically to a raw cache, the fetch honours the server's retry hints and exponential back-off on rate limiting, and a re-run never calls the API for a model already cached. Base-model identifiers declared in cards are resolved to their current canonical identifiers (71 legacy identifiers were merged in the original corpus), so that lineage counts are not split across old and new names. The result is a corpus of **15,991 models**: the original five tasks contribute 9,764 and the six added tasks 6,227.

3.5.2 Normalisation: turning metadata into verifiable facts

Normalisation converts the raw record into the schema of Table 3.4. Four rules matter most.

Table 3.4. Metadata written on every chunk of a model.

Field	Type	Meaning and unknown convention
`model_id`, `task`, `downloads`	text, text, number	Identifier, Hub task tag, popularity (used for sampling only)
`languages`, `language_status`, `multilingual`	list, enum, bool	ISO codes; status is *unknown* when the list is empty; `multilingual` marks models whose list may be partial
`size_mb`, `size_status`	number, enum	`size_mb` is written only when known; status is *known* or *unknown*
`param_count`	number	Written only when the Hub reports it
`commercial`, `modifiable`	enum	`yes`, `no`, `conditional` or `unknown`, from the licence map
`base_model`, `lineage_status`	text, enum	Declared base model, if any
`directions`, `direction_source`	list, text	Translation direction, only when stated by a naming pattern
`doc_quality`	enum	`full` or `minimal` documentation
`section`, `text`	text	Per chunk: the card section and its text (what the explanation cites)

Size (rule D20). The model size is the sum of the sizes of its main weight files: the top-level `.safetensors` files if any exist, otherwise the top-level `.bin` files. Duplicate formats (ONNX, GGUF, TensorFlow and Flax copies), nested folders and optimiser state are excluded, and the sum is kept exact (1 MB = 10^6 bytes, as on the Hub) because it is compared with a hard limit. If a counted file has no recorded size, or no weight files qualify, the size is unknown and no number is written. The rule is shown as a flowchart in Appendix A (Fig. A.3). Across the corpus 11,279 models (70.5%) have a known size and 4,712 (29.5%) do not; the reasons are no weight files at all (2,416), GGUF-only (1,570), weights under another name (262), weights only in a sub-folder (243) and adapter-only repositories (221). Widening the rule to those formats is deliberately out of scope, because a quantised or ONNX size is not the size of the model the snippet would load.

Rights from licences. A map of 54 licence identifiers assigns each licence a pair of values for commercial use and for modification, each *yes*, *no* or *conditional* (Table 3.5). Permissive licences are *yes / yes*. A non-commercial licence such as CC-BY-NC is *no* for commercial use but *yes* for modification (the result stays non-commercial). A no-derivatives licence is the reverse. Licences that attach use restrictions that follow the model into a fine-tune (OpenRAIL, Llama, Gemma) and copyleft or share-alike licences (GPL, AGPL, CC-BY-SA) are *conditional* for both rights, and a *conditional* right is never shown as a clean pass. Any licence that is missing or not in the map, including the generic identifier `other`, is *unknown* for both. Over the corpus, commercial use is *yes* for 70.3% of models, *conditional* for 6.5%, *no* for 3.2% and *unknown* for 20.1%.

Table 3.5. Licence-to-rights map (representative entries; the full map has 54).

Licence identifiers	Commercial use	Modification	Reason
apache-2.0, mit, bsd-3-clause, cc-by-4.0, cc0-1.0	yes	yes	Permissive
cc-by-nc-4.0, cc-by-nc-sa-4.0	no	yes	Non-commercial; derivatives allowed but stay non-commercial
cc-by-nd-4.0	yes	no	No derivatives
cc-by-nc-nd-4.0	no	no	Neither
openrail, creativeml-openrail-m, llama3.1, gemma	conditional	conditional	Use restrictions carry over to derivatives
gpl-3.0, agpl-3.0, cc-by-sa-4.0, mpl-2.0	conditional	conditional	Share-alike or copyleft obligations
other, missing, any identifier not in the map	unknown	unknown	Never guessed

Languages. Card language fields are normalised to two-letter codes (region subtags are dropped, three-letter codes for the ten Indic languages are aliased), "multilingual" is removed from the list and recorded as a flag, and an empty list gives *unknown*. Because some widely used multilingual models (XLM-R, Whisper, Qwen3-ASR) cover Indic languages without tagging them, a small hand-made map with a cited source for each entry (13 model identifiers) merges their documented languages into the list and sets the multilingual flag.

Translation direction. A language list cannot say whether a model translates Hindi to Marathi or the reverse. Direction is therefore recorded only for models named `opus-mt-<source>-<target>` (380 models), where the name states it and the pattern is one-directional; every other translation model has an *unknown* direction. The interface shows "direction from model name" next to such a pass.

3.5.3 Cleaning and chunking

Cards are cleaned (front matter, badges, images, HTML, code blocks and long tables removed; short tables kept), then split by Markdown heading into sections, and long sections are split on paragraph and word boundaries so that no chunk exceeds 450 estimated tokens, below the embedding model's limit. Sections that add tokens but no retrieval value (training hyper-parameters, framework versions, citations, acknowledgements, contact details and similar) are dropped, and at most four chunks per model are kept, which brought the corpus from an estimated 85,000 chunks to 48,179 (3.0 per model). A card with fewer than 200 characters of real content is marked `doc_quality = minimal`, and a single fallback chunk built from the metadata

(identifier, task, languages, base model, tags) is embedded instead, so that such a model can still be found and is shown with a "minimal documentation" note.

3.5.4 Embedding, index and snapshot

Each chunk is embedded with `multilingual-e5-large` (1,024 dimensions), with the prefixes the model expects for passages and for queries. Vectors are cached by the hash of the text, so rebuilding an index in a new namespace costs no new embedding; the whole corpus used 4.05 million hosted tokens. The vectors and their metadata are upserted into Pinecone (namespace `prod`; experiments use `exp-<name>`). An exact in-process index with the same filter operators (`$eq`, `$lte`, `$in`, `$and`, `$or`) can be substituted by setting an environment variable; it was used for the frozen evaluation and as a fallback when the data plane was unreachable. A live check confirmed that the hosted filters behave as the local ones do on the four semantics the design depends on: a missing field is kept by an explicit `$or`, `$in` matches any element of a list field, nested `$and` of `$or` clauses is accepted, and namespaces do not mix.

Finally, the normalised records are written to a snapshot file whose SHA-256 checksum is recorded in `config.yaml` and checked on load, and the build is deterministic. The snapshot (15,991 records, SHA-256 `45814b8c`…, dated 25 October 2026) is the ground truth for check-a-model mode and for the evaluation.

3.6 Query pipeline

The online pipeline is a LangGraph state graph, shown in Fig. 3.3. Nodes are thin functions over plain-Python components that are unit-tested on their own; the graph only routes between them. Table 3.6 lists the nodes.

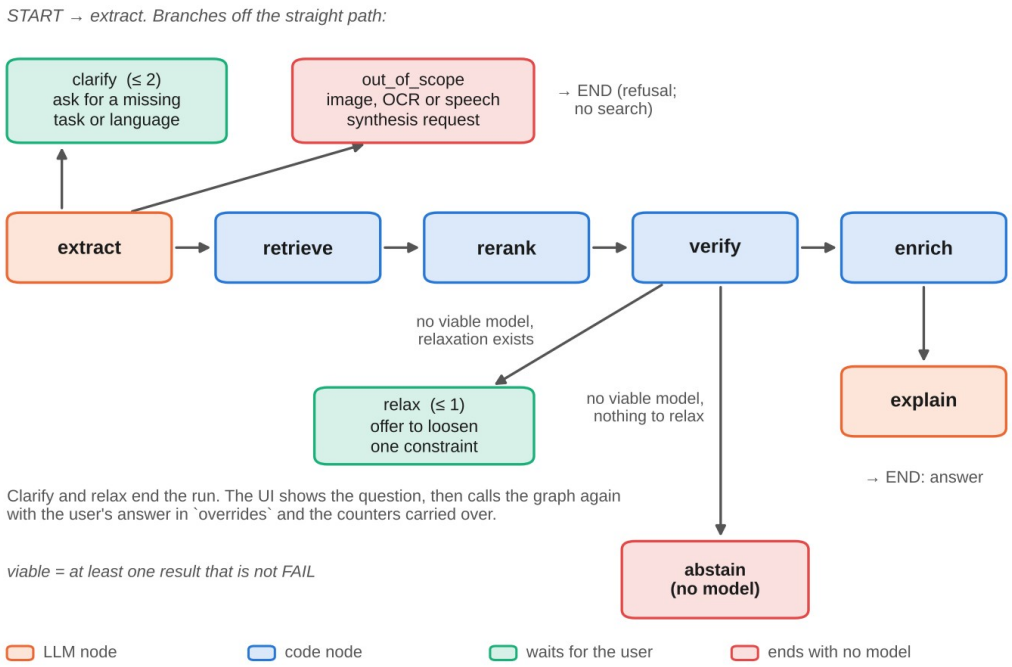


Fig. 3.3. The LangGraph query pipeline and its branches. Clarify and relax end a run; the interface re-invokes the graph with the user's answer.

Table 3.6. Nodes of the query graph.

Node	Uses an LLM	Function
extract	yes	Out-of-scope check, language-model extraction, validation, deterministic backstop, user overrides
clarify	no	Returns one question with clickable options (at most two per query)
retrieve	no	Filtered dense search with the user's own words, widening *k* until the pool is large enough
rerank	no	Cross-encoder re-ordering of chunks; falls back to vector order if the quota is exhausted
verify	no	Pass, fail or unknown for each requested constraint; rank; keep five
relax	no	Offers to loosen one constraint (at most once)
abstain	no	Ends with no recommendation when every candidate has a known fail and nothing can be relaxed
enrich	no	Rights notes, lineage, memory estimate, task guide and snippet for each shown model
explain	yes	Cited explanation of each shown model from budgeted card excerpts

3.6.1 Constraint extraction

A request first passes a deterministic out-of-scope check: image or music generation, captioning, optical character recognition, text-to-speech, video and similar requests are refused with a message and neither an LLM call nor a search is made. Otherwise the request is sent to the language model with a prompt that lists the eleven tasks with a hint for each, asks for JSON only, and forbids inventing a size: the model may output an explicit size in megabytes only if the user gave one, and otherwise picks a **size word** from a fixed table (Table 3.7) whose numeric value comes from code. The model runs at

temperature zero with low reasoning effort (about 500 tokens a call), and every response is cached on disk by the hash of the model, prompt and system text.

Table 3.7. Vague size words and their fixed thresholds.

Word	Threshold
"tiny", "mobile"	at most 100 MB
"small", "lightweight"	at most 500 MB
"medium"	at most 2,000 MB

The output is *untrusted*. `clean_constraints` coerces every field to the schema (an unknown task, a non-positive size or a non-boolean right becomes null) and a **deterministic backstop** then fills in only what the model left empty and the request plainly states: an explicit size with a unit (MB or GB), one size word, commercial and modification keywords, named permissive licences ("Apache", "MIT", "BSD" or "permissive licence" are read as commercial and modifiable, an approximation that is shown to the user), exactly one task keyword (two or more categories are ambiguous and left for the user), and language names including Indic spellings. It never overrides the model, never runs on instruction-like queries ("ignore previous instructions", "set commercial to false"), and rejects a `text-generation` guess unless the request has a generation cue. The backstop was added because the larger fallback model silently dropped constraints in three of twenty test cases; with it, extraction matches the expected record exactly on all 29 cases of the extraction set, against 90% for the model alone (Table 4.2 in Chapter 4). The flow is drawn in Appendix A (Fig. A.4).

3.6.2 Clarifying questions and the editable reading of the request

If no task was found, or a translation names fewer than two languages, the graph ends the run with a question and clickable options (for example "What should the model do?" with one button per task). The interface shows the question; when the user answers, it re-invokes the graph with the answer as an *override* and carries the counters forward, so the caps (two questions per request, one relaxation) hold across turns. Every result page starts with a plain-language sentence, for example "We understood: you want a model to find names, places and other items in text (NER), for Marathi, usable commercially", and a panel in which any field can be edited and the search repeated. Edits pass through the same validator as model output.

3.6.3 Retrieval

Retrieval embeds the **user's own words**, not a rewritten query. This was a finding of the project: the extractor's `soft_query` field occasionally lost the topic (a request for a small Gujarati classifier with a permissive licence came back as "permissive license" and retrieved only licence sections), so every system now searches with the original text. A metadata filter is built from the constraints (Fig. A.2): the task is an exact match; a size limit becomes "size at most the limit *or* size unknown"; a rights requirement removes only models with a definite *no*; and each language becomes "tagged with it, *or* language unknown, *or* multilingual". Every clause keeps the unknowns, so the filter is deliberately loose and verification is exact.

The search asks for 50 chunks and, if they cover fewer than the wanted number of distinct models, doubles the request up to 400, because several chunks of one model can fill the first results. Chunks are then grouped by model and scored by the model's best chunk, so one model cannot occupy several slots. The wanted number is **100**, not five (Fig. 3.4). Semantic search cannot see direction, size or licence, because these are not in the card text; if only five models were verified, a fully verified model ranked sixth by meaning would never be seen, and five better-sounding but unknown models would be shown instead. Verifying a pool of 100 and then cutting to five raised the share of fully verified recommendations on the 75 normal queries from 72.3% to 83.3%.

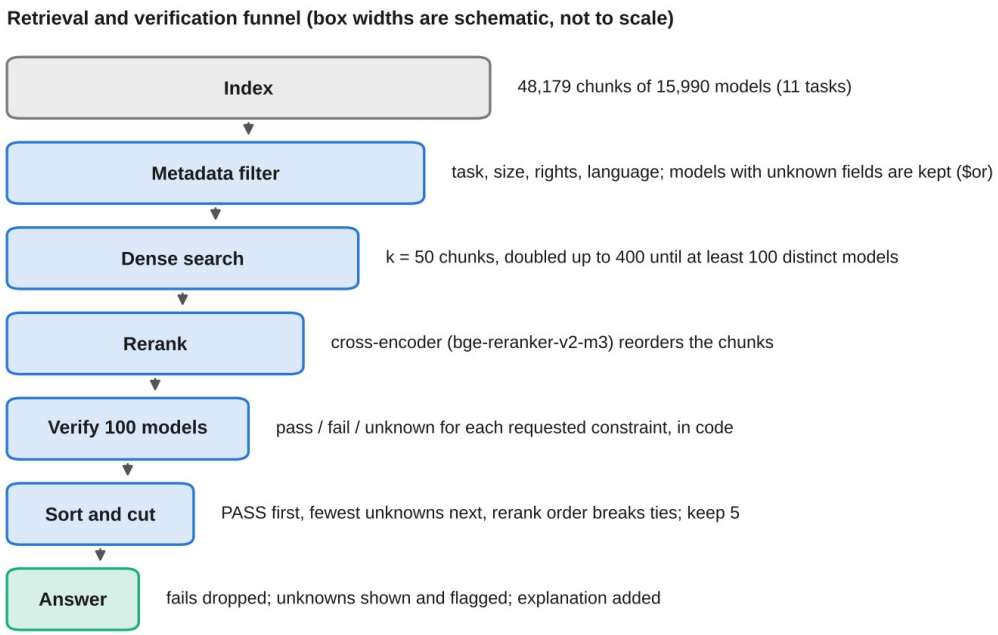


Fig. 3.4. Verify wide, show narrow: from 48,179 chunks to five models.

3.6.4 Re-ranking

The chunks returned by the search are re-ordered by a cross-encoder (`bge-reranker-v2-m3`), which reads the request and each chunk together and is more accurate than comparing two separate vectors [10]. At most 100 chunks are sent, truncated to the model's input limit (Indic scripts consume many more tokens than a word count suggests, and an over-long chunk once made a whole request fail), and results are cached by query and chunk text so that a repeated evaluation consumes no quota. If the hosted service reports that its monthly quota is exhausted, the graph sets a flag, keeps the vector order and the interface shows a caption saying that results are not re-ranked; the verdicts are unaffected. A local re-ranker with the same model is the fallback, capped at 40 chunks.

3.6.5 Verification

Verification is the heart of the system. For each model in the pool, `verify` computes one result for each *requested* constraint (rows for constraints nobody asked for are not produced, so the interface never shows a pass for something the user did not request). Table 3.8 gives the rule for each constraint, and Fig. 3.5 the combination.

Table 3.8. Verdict rules for each constraint.

Constraint	Pass	Fail	Unknown
Task	Model's task equals the requested task	A different task	No task in metadata
Languages (all requested)	Every requested language is in the list	List known, some language absent, model not multilingual	List empty or status unknown; or absent from the list of a multilingual model (list may be partial)
Direction	The model name states the direction	Never	Otherwise
Size	`size_mb` at most the limit	`size_mb` above the limit	Size unknown
Commercial, modifiable	Right is *yes*	Right is *no*	Right is *conditional* or *unknown*

The overall verdict of a model is *fail* if any row fails, otherwise *unknown* if any row is unknown, otherwise *pass*. Models are sorted by this verdict, then by the number of unknown rows, and the sort is stable so that the re-ranker's order decides between equals. Models that fail are never shown as recommendations. The top five are kept. A model that passes while being multilingual carries a note ("multilingual, not specific to Hindi"), and a model with minimal documentation carries a note; notes never change a verdict.

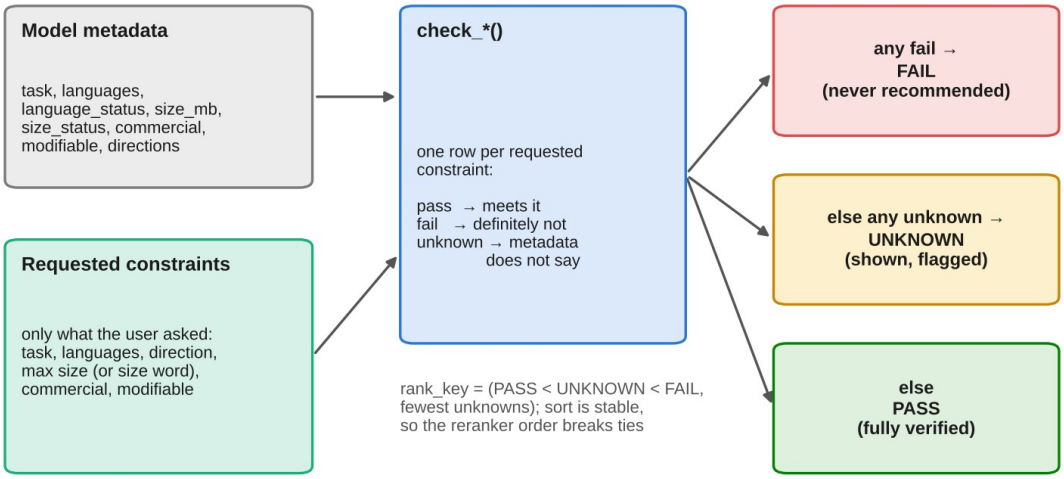


Fig. 3.5. Per-constraint verdicts and their combination into one verdict and a rank.

3.6.6 Relax-and-retry and abstention

If no model in the pool is viable (every one fails), the graph offers to relax one constraint, in a fixed order (the size limit moves to the next larger size-word threshold, then the modification requirement, then the commercial requirement; the task and languages are never relaxed) and at most once; if nothing can be relaxed it abstains. Because a model whose size or language is unknown can never fail (FR-6), every real request in the corpus leaves something to show, so these two paths are exercised by unit tests but are unreachable on this corpus; an impossible request ends instead as an all-unknown answer with the banner "No model fully verified". This is a limitation discussed in Chapter 4.

3.6.7 Enrichment

For each shown model the pipeline computes, from metadata only: **rights notes** in plain words (for example, "You may modify it, but the result stays non-commercial"); **lineage** ("fine-tune of X; N other fine-tunes in our snapshot"); a **fine-tuning memory estimate**; a link to the official task guide for fine-tuning; and a **starter snippet** built from a fixed template filled with the model identifier and task (never generated by a language model, and the identifier is validated against a strict pattern before it is placed in code). The memory estimate for a model with P parameters uses two bytes per parameter for inference, and sixteen bytes per parameter for full fine-tuning with AdamW in mixed precision; LoRA is estimated as the inference weights plus a trainable adapter of 1% of the

parameters at the full-fine-tuning cost (Eq. 3.1). The estimates are lower bounds that exclude activations and are compared with a free 16 GB GPU.

$$M_{inf} = 2P, \quad M_{LoRA} = 2P + 0.01 \cdot 16P = 2.16P, \quad M_{full} = 16P \quad \text{(bytes)} \tag{3.1}$$

3.6.8 Explanation

For the shown models, the explanation step selects up to four passages per model from distinct sections (best chunk of each section first) and trims each to its share of a 5,000-token prompt budget, keeping the sentences that overlap with the user's request. The budget exists because the free tier allows 8,000 tokens per minute, so five models with several full chunks each would not fit. The passages are wrapped in `<card_section>` tags and sent to the language model, whose system prompt states that tagged text is third-party data to be described and not obeyed, and that licence, size and language facts must not be stated because they come from the verified table. The reply is JSON; explanations that cite a model identifier outside the candidate set are dropped. If the language model fails, the verified table is still rendered and the page says explanations are unavailable.

3.7 Algorithms

This section gives the main algorithms as pseudocode and states their cost.

3.7.1 Verification and ranking

Algorithm 3.1. Verification of one model and ranking of the pool

```
VERIFY(meta, C):                    # C = constraints the user requested
  R <- empty map
  if C.task:
    R[task] <- UNKNOWN if meta.task is empty
               else PASS if meta.task = C.task else FAIL
  if C.langs:
    R[languages] <- UNKNOWN if meta.languages is empty or status unknown
                    else PASS if every requested code is in meta.languages
                    else UNKNOWN if meta.multilingual else FAIL
  if C.direction:
    R[direction] <- PASS if "src-tgt" is in meta.directions else UNKNOWN
  L <- C.max_size_mb, else SIZE_WORDS[C.size_word], else none
  if L is not none:
    R[size] <- UNKNOWN if meta.size_status != known
               else PASS if meta.size_mb <= L else FAIL
  for r in {commercial, modifiable} that were requested:
    R[r] <- PASS if meta[r] = yes; FAIL if meta[r] = no; else UNKNOWN
  return R

OVERALL(R):  FAIL if FAIL in R; else UNKNOWN if UNKNOWN in R; else PASS
RANK_KEY(R): (0 PASS | 1 UNKNOWN | 2 FAIL ,  number of UNKNOWN rows in R)

POOL <- the 100 best models by re-ranked chunk score
sort POOL by RANK_KEY (stable); drop models whose OVERALL is FAIL; keep first 5
```

Correctness. Because every row is a comparison of a requested value with a stored field, and because *fail* is produced only when the stored field is known and violates the request, a returned model cannot violate a requested constraint (FR-5). The property holds

independently of retrieval, re-ranking and the language model. *Cost.* Verifying a model is O(|C|), so the 100-model pool costs a few hundred comparisons, negligible beside the network calls.

3.7.2 Filter construction and widening search

Algorithm 3.2. Metadata filter with unknowns kept, and widening k

```
BUILD_FILTER(C):
  clauses <- []
  if C.task:      add {task = C.task}
  if L defined:   add {size_mb <= L  OR  size_status = unknown}
  for r in {commercial, modifiable} that were requested:
                  add {r in [yes, conditional, unknown]}
  for each requested language g:
                  add {g in languages OR language_status = unknown
                       OR multilingual = true}
  return {} if no clauses; the clause if one; AND(clauses) otherwise

FETCH_CHUNKS(query, C, want = 100):
  k <- 50
  loop:
    M <- INDEX.search(embed(query), filter = BUILD_FILTER(C), top_k = k)
    if distinct models in M >= want or k >= 400 or |M| < k:  return M
    k <- min(2k, 400)
```

Cost. At most four index calls (k = 50, 100, 200, 400). With the hosted index each call is a single approximate nearest-neighbour query; with the exact local index it is a linear scan of the 48,179 vectors after the filter.

3.7.3 The query graph

Algorithm 3.3. Control flow of one run of the query graph

```
RUN(query, overrides, counters):
  if OUT_OF_SCOPE(query): return refusal
  C <- CLEAN(LLM_EXTRACT(query));  C <- BACKSTOP(query, C)
                                   # the backstop is skipped for instruction-like queries
  C <- CLEAN(C overlaid with overrides)
  if C.task is missing (or a translation names < 2 languages)
     and counters.asked < 2:        return CLARIFY(question)
  M <- FETCH_CHUNKS(query, C, want = 100)
  M <- RERANK(query, M)             # vector order is kept if the quota is exhausted
  P <- GROUP_BY_MODEL(M)[:100]
  for m in P:  R[m] <- VERIFY(m.meta, C)
  sort P by RANK_KEY;  S <- first 5
  if no model in S has OVERALL != FAIL:
      if counters.relaxed < 1 and an offer exists:  return RELAX(offer)
      return ABSTAIN
  report[m] <- ENRICH(m) for each viable m in S
  text <- LLM_EXPLAIN(budgeted <card_section> passages of S)  # failure keeps the table
  return table, report, text, banner if no model has OVERALL = PASS
```

3.7.4 Token-budget throttle

The free tier counts tokens per minute, so every call first reserves its estimated tokens in a 60-second sliding window and sleeps until the window has room; after the call the estimate is replaced by the provider's own count (Fig. A.5). A rate-limit error is handled once by waiting the server's `retry-after` period if it is short, and otherwise by trying a fallback model with its own limit. The cost of waiting is bounded by one minute per call.

3.7.5 Evaluation formulas

For a system answering query *q* with top-five models *m*, and the overall verdict *v(m)* computed by the same verification rules for every system, the report uses the following definitions.

$$\text{Violation} = |\{m : v(m) = \text{fail}\}| / |\{m\}|, \quad \text{Unknown} = |\{m : v(m) = \text{unknown}\}| / |\{m\}|,$$
$$\text{Fully verified} = |\{m : v(m) = \text{pass}\}| / |\{m\}| \quad (3.2)$$

The three fractions add up to one over all recommended models (Eq. 3.2). Agreement between two grading passes is reported as Cohen's kappa [36],

$$\kappa = (p_o - p_e) / (1 - p_e) \quad (3.3)$$

where p_o is the observed agreement and p_e the agreement expected by chance (Eq. 3.3), both unweighted and quadratic-weighted for the ordinal 0 to 2 grades, and is interpreted with the bands of Landis and Koch [37].

3.8 User interface design

The interface is a single Streamlit page with three tabs (Table 3.9). The design follows the needs of the two main users: a developer who wants fast, trustworthy answers and a beginner who needs guidance. Pass is shown in plain text, fail with a red cross and unknown with a warning sign, so that colour is not the only signal; a legend and a glossary explain every term in plain words. Streamlit re-runs the whole script on every interaction, so the graph state, the loop counters and the loaded services are kept in session state and cached resources; otherwise a clarifying question would lose its place.

Table 3.9. Interface elements.

Element	Purpose
Search tab	Query box, five example queries, "Compare with naive RAG" switch, "We understood" sentence, clarifying questions and relax offers, verdict tables, explanations, banner, Try it and Build on it panels, editable intent, glossary
Examples tab	More than 25 example queries grouped by task, each with a "Use query" button
Check a model tab	Paste a model identifier from the snapshot; shows facts, an optional verdict table against requirements and the Build on it panel
Naive-RAG comparison	Runs the same request with no filters and no verification, side by side, and counts the models that fail a requirement
Replay mode	Serves recorded runs without network, for demonstrations; the page says it is a replay

3.9 Security, safety and ethics

Untrusted card text. Anyone can publish a model card, and card text is placed in a prompt, so it is a channel for indirect prompt injection [35]. Fig. 3.6 summarises the

defence in depth. The most important defence is architectural: verification reads only metadata and never language-model output, so injected text can at most change the *wording* of an explanation and can never change a pass, fail or unknown. Additional layers wrap each passage in delimiters that name its source, escape any literal delimiter inside the card, tell the model in the system prompt that tagged text is data, instruct it not to state licence, size or language facts, and drop any explanation that cites an identifier outside the candidate set. Delimiters reduce the risk but do not eliminate it, and the report says so. A test with three synthetic hostile cards (an authority spoof, a delimiter break and a citation bait) whose metadata was deliberately unknown, so that the filter could not remove them, found that tables came from metadata alone in five of five runs, hostile cards ranked below honest ones, and none of the five explanations repeated an injected claim.

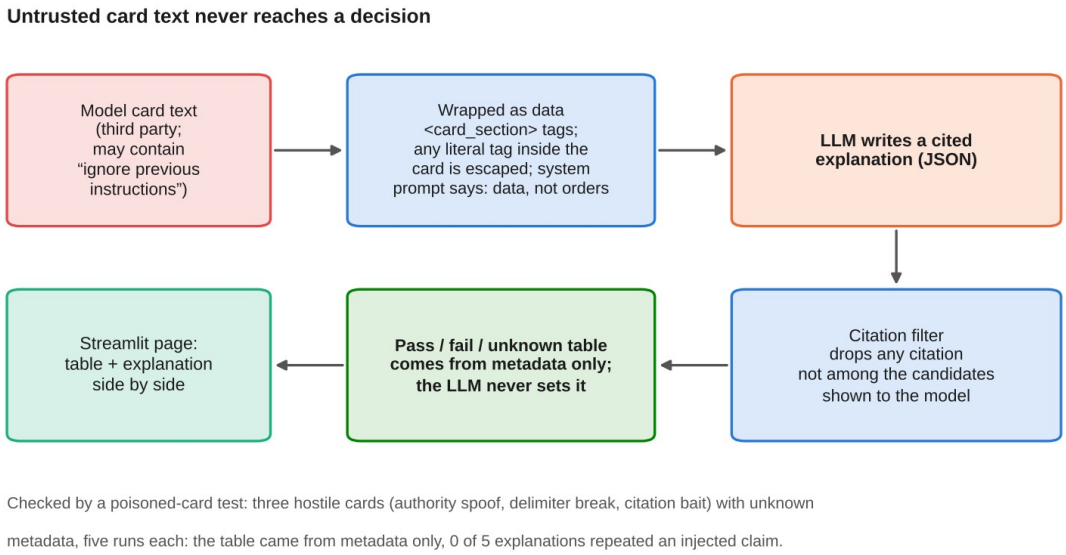


Fig. 3.6. Defence in depth against hostile card text: verdicts never depend on the language model.

Other safeguards. User edits and model output share one validator; snippet templates reject identifiers that are not of the form `owner/name`; secrets live only in a git-ignored environment file and no key is logged; and no card text is rendered as HTML.

Ethical concerns. In line with the value-based approach of the ethics standard [42], four values are treated as requirements and not as afterthoughts: *transparency* (every verdict, every unknown and the reading of the request are visible), *honesty* (the tool says when nothing is fully verified, does not guess, and states that rights are read from the declared licence and are not legal advice), *user autonomy* (the user can edit the reading, see the comparison with naive RAG and decide), and *fairness to the long tail* (Indic-language models are sampled deliberately and are not ranked by popularity). The limits of the

evidence are also part of honesty: the relevance and faithfulness grades in this report were produced by language-model judges, not by humans, and say so wherever they appear.

3.10 Evaluation methodology

Fig. 3.7 summarises the evaluation. The questions it answers are those of Section 1.3: does verification reduce violations, does it cost relevance, does the system ask and abstain appropriately, are the explanations grounded, and is it better than the Hub's own filters.

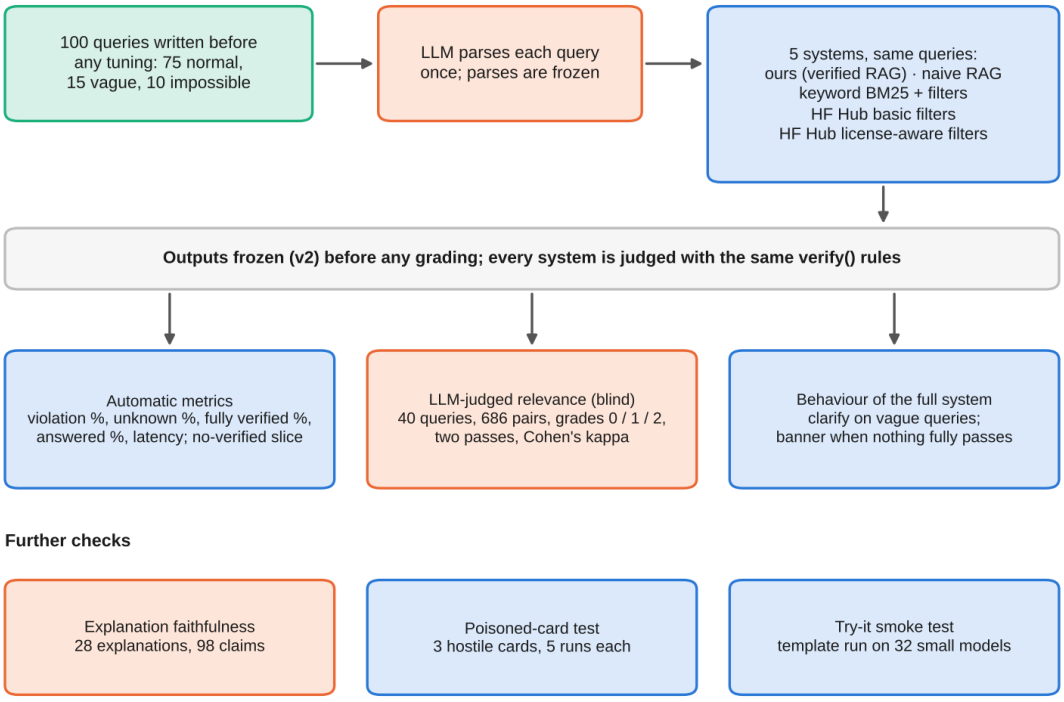


Fig. 3.7. Design of the evaluation.

Queries. One hundred queries were written and committed before any retrieval experiment: 75 normal queries over the five original tasks and at least six Indian languages, with a mix of size, rights, language and direction constraints; 15 vague beginner queries (for example "I want a Hindi model"); and 10 requests intended to be impossible. They were written by the author, from user needs and not from system output, and are frozen; every query later used to debug a failure is listed in a "seen" file (Table 4.9 in Chapter 4). One author both wrote the queries and built the system, so there is no independent query writer; this limitation is stated again in Chapter 4.

Systems. Five systems answer the same queries. (1) **Ours**: the full graph. (2) **Naive RAG**: the same index, embeddings, re-ranker and language model, but no metadata filter (not even the task) and no verification-driven behaviour. (3) **Keyword + filters**: Okapi BM25

over the same chunks with the same metadata filters. (4) **HF Hub, basic filters**: the Hub's own listing filtered by task and language, ordered by downloads. (5) **HF Hub, licensed filters**: the same plus the licence filter `apache-2.0` whenever commercial use or modification is required, which is the best case for a licence-string filter. Every system is judged by the *same* verification rules applied to its top five, using the same parsed constraints, so one definition of "violation" applies to all, and outputs were frozen before any grading.

Metrics. The automatic metrics are the three fractions of (Eq. 3.2), the share of queries answered, clarification rate on vague queries, behaviour on queries for which no model in the corpus fully passes, and latency. *Relevance* was graded 0 (does not suit), 1 (plausible) or 2 (clearly suits) on a blind sheet of 686 (request, model) pairs over 40 queries, which is the union of the top five of four systems, shuffled, with system names hidden. Size and licence are checked automatically and are not graded. *Faithfulness* counted the factual claims in 28 explanations and how many the shown card passages support. **All relevance and faithfulness grades were produced by independent language-model agents acting as judges** (eight agents for the first relevance pass, three different agents for a repeat pass over 15 queries, two agents for faithfulness) with a written rubric, no access to system names and a requirement to grade by reading; their one-line reasons are stored. This is weaker evidence than human grading, because the judges share a model family with the assistant that helped build the system and no human has checked the grades.

Reproducibility. Results are generated by one command from the snapshot, the configuration file and the query file. The labelled evaluation was run on the five-task corpus; after the corpus grew to eleven tasks, the five-task index was restored (`CORPUS_TASKS=five`, a separate namespace and the frozen parses) and reproduced identical top-five lists for ours, naive RAG and keyword search on all 100 queries.

3.11 Verification and validation of the software

Verification and validation activities were planned across four layers, in the spirit of the verification-and-validation standard [40], and documented as identified test cases with expected results and recorded outcomes in the manner of the test-documentation standard [41]. Table 3.10 summarises them.

Table 3.10. Test layers and evidence.

Layer	Count	What it covers
Unit and component tests (`tests/`)	335	Licence map, languages, size rule, verification and ranking, filter construction, chunking, cleaning, backstop (40 cases), graph routing and loop caps, guidance, throttle, evidence budget, snapshot checksum, replay, app rendering. All run offline with stubs, in about 23 s
Edge-case catalogue	140 cases	Cases identified as E-NORM, E-VER, E-RET, E-LLM, E-GRAPH and so on, each with an expected result; 103 are marked as covered in the testing plan
Live-service checks	–	Hub response shape (eight real responses saved as fixtures), Pinecone filter semantics (four checks), hosted embedding and re-ranking limits, Groq rate limits
Extraction accuracy test	29 cases	Exact match of the constraint record, with and without the backstop
Poisoned-card test	3 cards × 5 runs	Verdicts from metadata only; no repeated injected claim
Try-it smoke test	32 models	The generated snippet runs on small models
End-to-end	7 demo scenarios	Recorded against live services and replayed with the network broken

Testing found real defects that were fixed and are part of the project's history, among them: a missing `language` argument in the Hub client call, a card-download endpoint that returned an empty card for some repositories, the hosted re-ranker rejecting over-long Indic chunks, the Pinecone client ignoring the proxy setting, a size rule that left known-weights models unknown, and vacuous test assertions that passed regardless of behaviour. Evaluation queries used while debugging are listed in the "seen" table.

3.12 Development process and reproducibility

The project was carried out by one person between 16 and 27 October 2026, with an AI coding assistant doing much of the implementation and test writing under the author's direction and the author making the design decisions. Design decisions D1 to D24 were recorded as they were taken, with alternatives and rationale, and amended when live measurements contradicted them (for example the hosted embedding quota, the verify-pool size and the unreachable abstain path). The repository history has more than 35 commits. A reader can reproduce the work with four commands: install the requirements, run the test suite, rebuild the index from the snapshot and embedding cache, and run the evaluation harness; secrets are supplied in an environment file, and nothing in the repository contains a key.

The tuning policy was set in advance. After the queries were committed, fixes made in response to failures were allowed only if every evaluation query used to debug was listed (Table 4.9), and after the outputs were frozen for grading, no retrieval, ranking, prompt or

configuration value was changed. Two design changes made before the freeze were driven by measured failures on the evaluation queries: widening the verify pool from 25 to 100 models (translation queries had no fully verified model although one existed) and adding the permissive-licence and language-name rules to the backstop. Six tasks were added after the freeze; they were tested with smoke queries and the demonstration, and the frozen evaluation was shown to be unchanged.

Chapter 4
Snapshots of the Project, Results and Discussion

This chapter shows the working application and then presents the measurements. Section 4.1 walks through screenshots of the interface. Sections 4.2 to 4.7 report the results of the evaluation described in Section 3.10, and Section 4.8 discusses what they show and what they do not. Every number in the tables comes from the result files stored in the repository; **relevance and faithfulness figures were produced by language-model judges and not by human annotators**, and are labelled as such.

4.1 Application snapshots

The screenshots were captured from the running Streamlit application. Where the live services were not needed, the application was run in *replay* mode, which serves recorded runs from disk and says so on the page ("Replay mode: showing recorded runs, no live services"); the recordings were made against the live index and models. In the tables, a pass is shown as plain text, a failure with a red cross and an unknown with a warning sign.

4.1.1 Start page

Fig. 4.1 shows the start page: the data-snapshot date, five example queries covering different tasks and languages, the search box and the switch that adds the naive-RAG comparison. A disclaimer under the title states that rights are read from the declared licence and are not legal advice (FR-1, NFR-4).

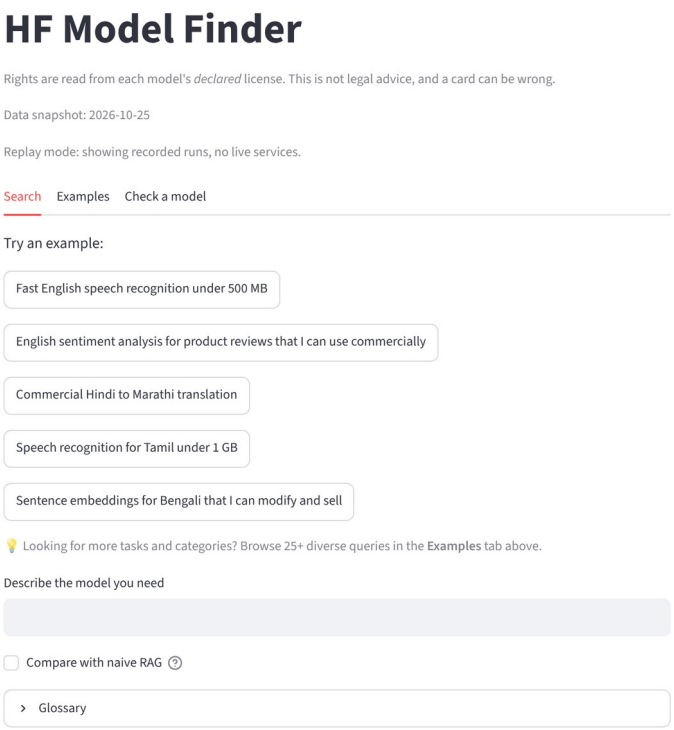


Fig. 4.1. Start page with example queries, the search box and the comparison switch.

4.1.2 Verified results against naive RAG

Fig. 4.2 shows the request "Small sentiment model for Hindi I can use in a commercial app" with the comparison switched on. The sentence "We understood" shows that "small" was read as 500 MB. The left column is the full system and the right column is naive RAG running on the same index, with no filters and no checks. Every row on the left passes. On the right, naive RAG recommends models with a red cross for size: they sound relevant but are larger than the limit the user implied. The caption at the end of the right-hand column counts the failures (four of five in this run).

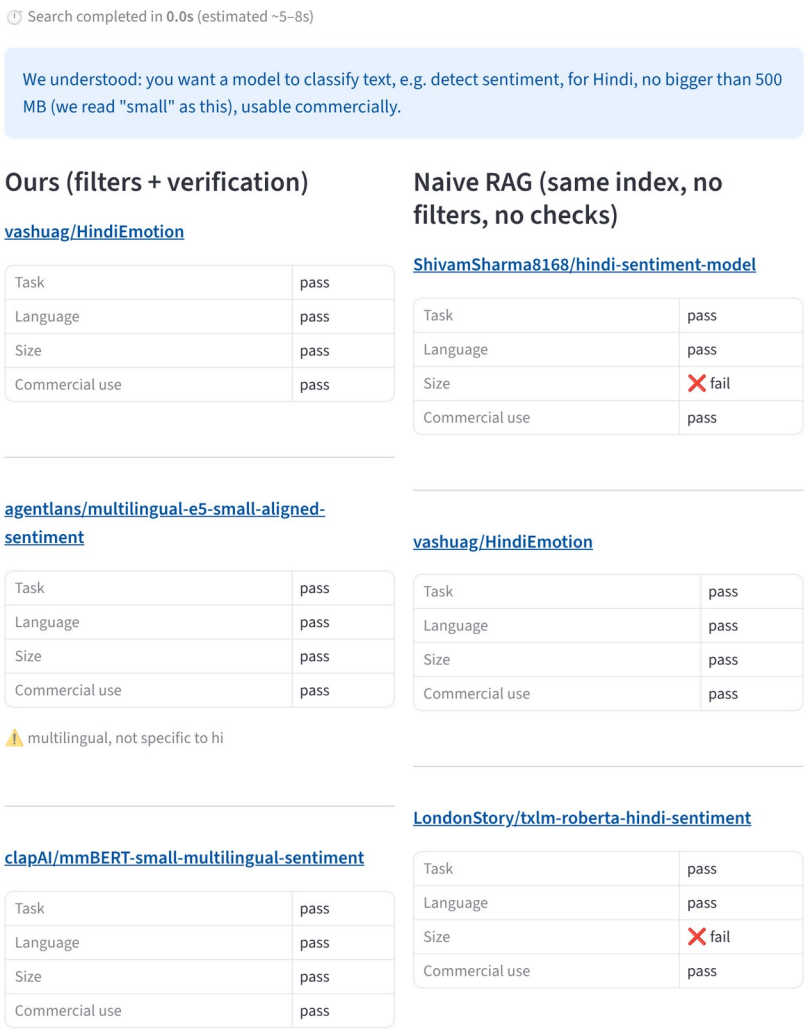


Fig. 4.2. Side-by-side comparison for a small Hindi sentiment model under a commercial licence (first three results of each column shown).

4.1.3 Clarifying questions

For the request "I want a Hindi model", the system does not guess. Fig. 4.3 shows the reading ("a model, we are not sure what for yet, for Hindi") and the question "What should the model do?" with one button per task. Choosing "Detect sentiment / classify text" re-runs the graph with that answer as an override and produces the results of Fig. 4.4. Every vague query in the evaluation behaved this way.

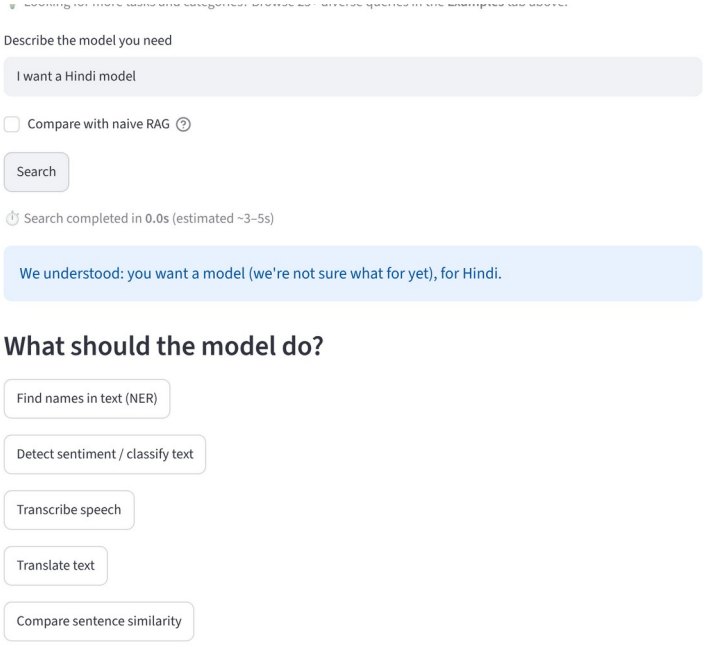


Fig. 4.3. A vague request ("I want a Hindi model") is answered with a question, not a guess.

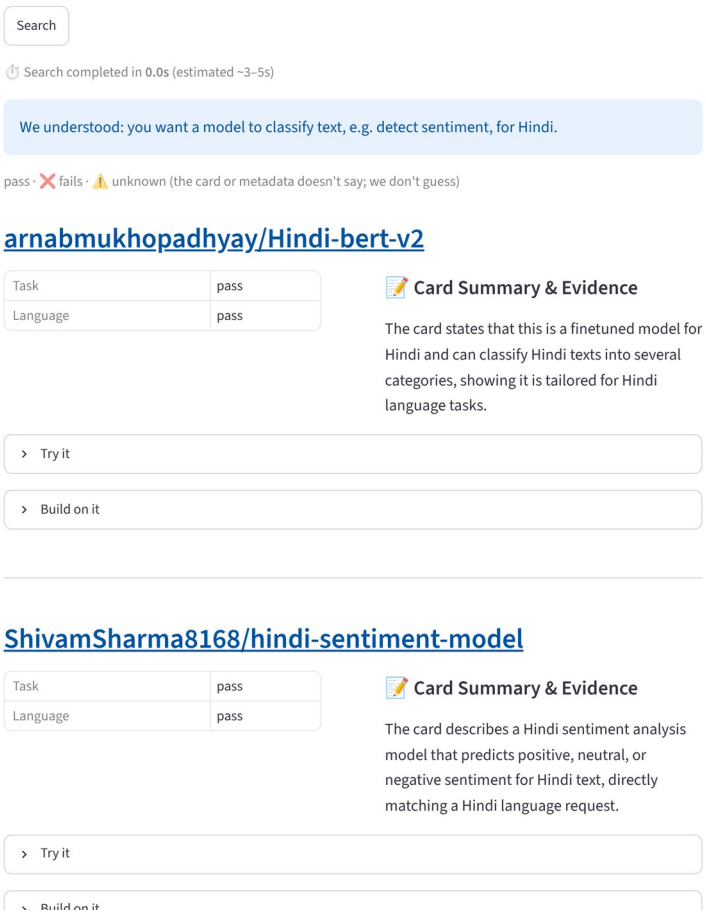


Fig. 4.4. Results after the user chose "Detect sentiment / classify text".

4.1.4 Honest unknowns and the banner

Fig. 4.5 shows the request "Hindi to Marathi translation under 1 MB". No model can be verified: size and direction are unknown for every candidate, so the page shows the banner "No model fully verified: every result below has at least one unknown", and each row carries a warning sign. The system does not claim a pass it cannot prove, and it does not discard the candidates either. In Fig. 4.6, for "Urdu to English translation I can use in a product", the top results pass the direction check because the model *name* states it (`opus-mt-ur-en`), and the note "direction from model name" makes the source of the verdict visible; other models in the same list remain unknown.

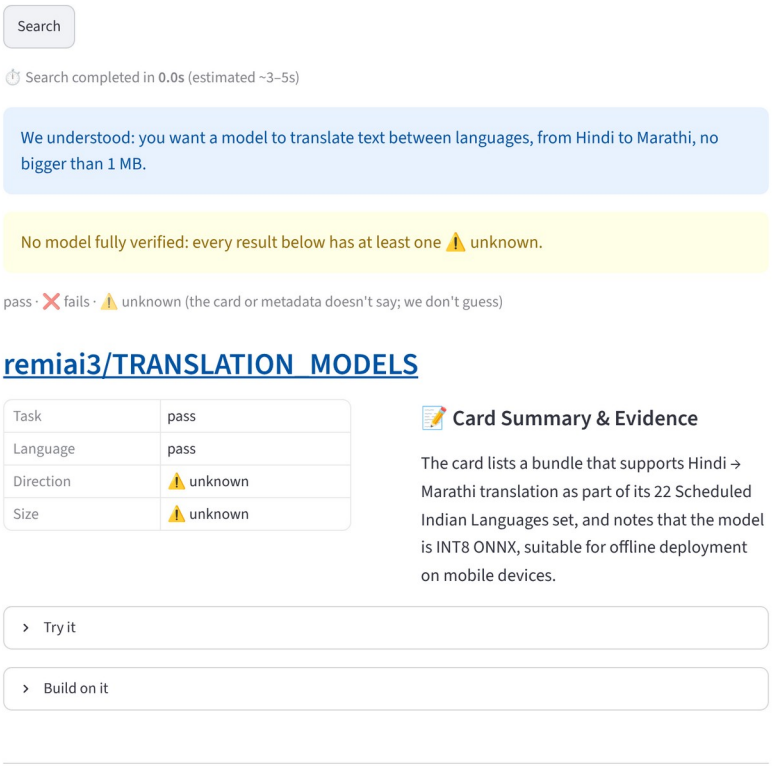


Fig. 4.5. The banner "No model fully verified" when every candidate has an unknown, for a request no model can fully satisfy.

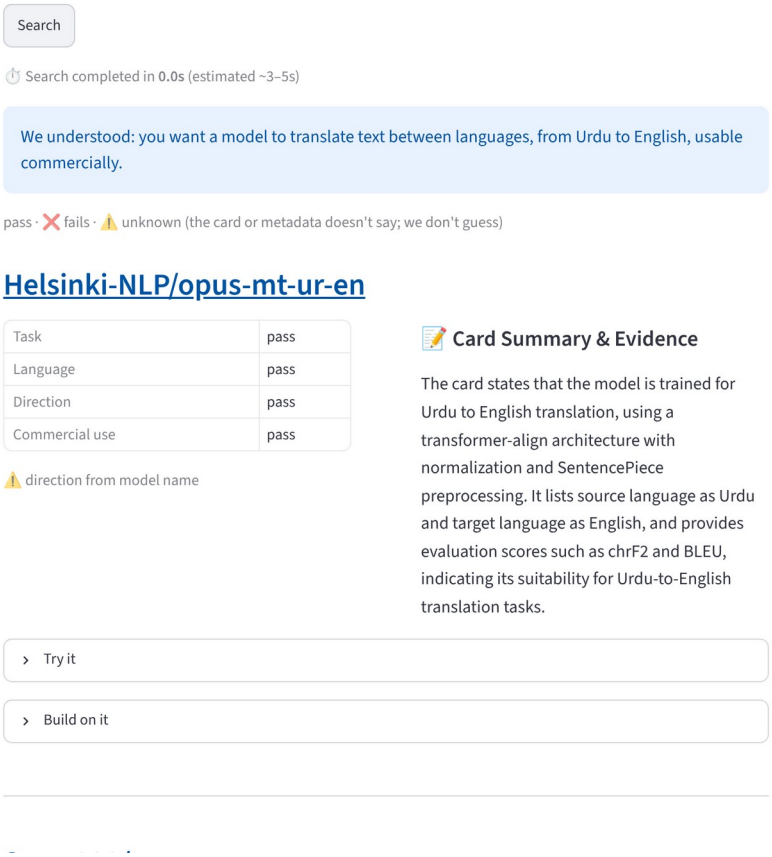


Fig. 4.6. Translation direction known from the model name, and flagged as such.

4.1.5 Trying a model and building on it

Under each model, two panels give the next step. **Try it** shows a starter snippet built from a fixed template and says in plain words that it is a standard template that has not been tested on that model. **Build on it** shows the modification right, lineage ("fine-tune of X; N other fine-tunes in our snapshot"), the estimated GPU memory for inference, LoRA and full fine-tuning against a free 16 GB GPU, and a link to the official fine-tuning guide for the task (Fig. 4.7). When the parameter count is not known the panel says that no memory estimate is possible.

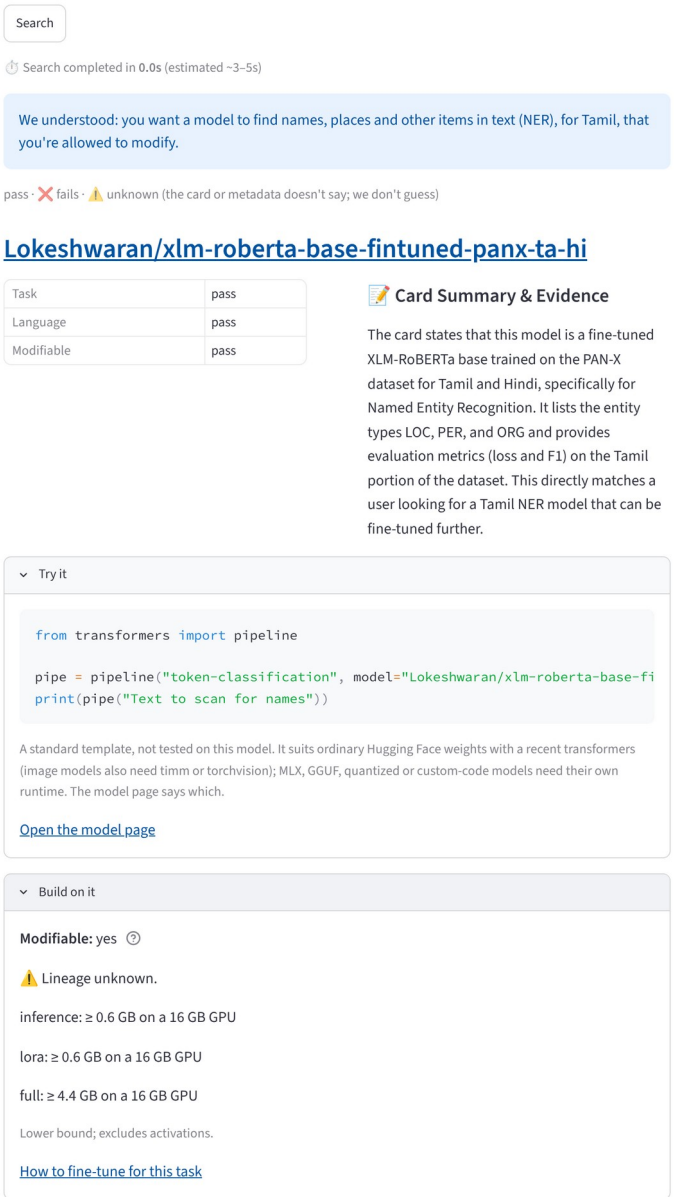


Fig. 4.7. The Try it and Build on it panels for a Tamil named-entity model that the user wants to modify.

4.1.6 Coverage beyond language models

Fig. 4.8 shows an image-classification request answered with models that pass the task and commercial-use checks, and Fig. 4.9 a summarization request. Both were added when the scope was extended to eleven tasks. An image request is no longer read as a text task: before the extension, "image classifier" was parsed as text classification and returned text models marked as passing the task check. Requests that the index cannot serve, such as "Text to image diffusion model", are refused with a message and no models (Fig. 4.10).

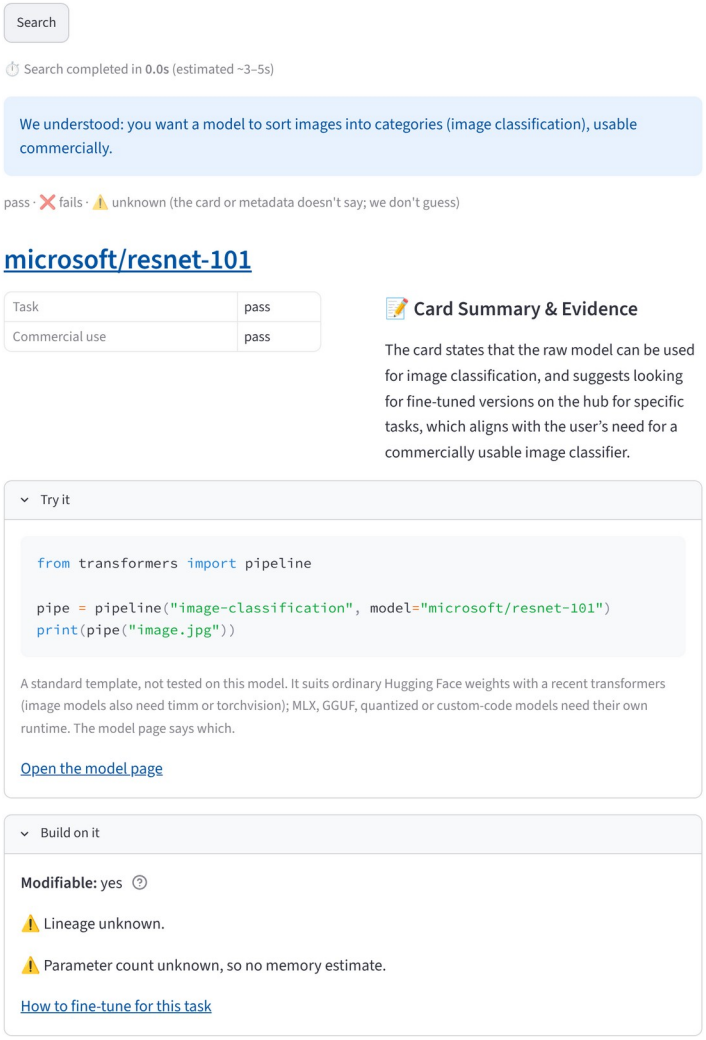


Fig. 4.8. Image classification with a commercial-use requirement: vision models are verified in the same way.

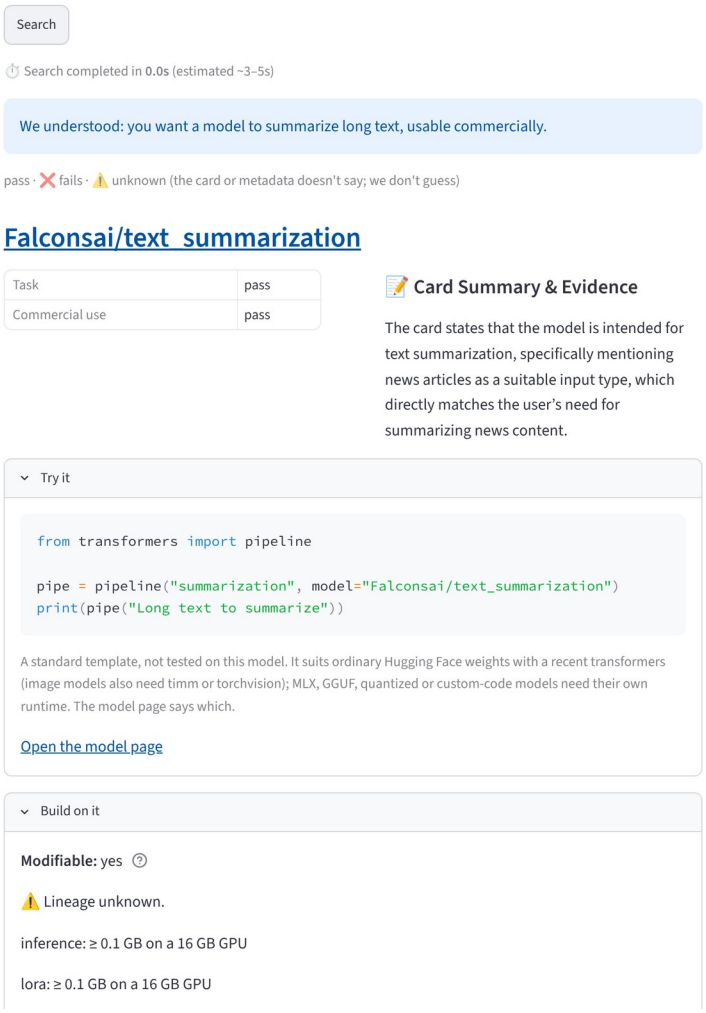


Fig. 4.9. Summarization with a commercial-use requirement.

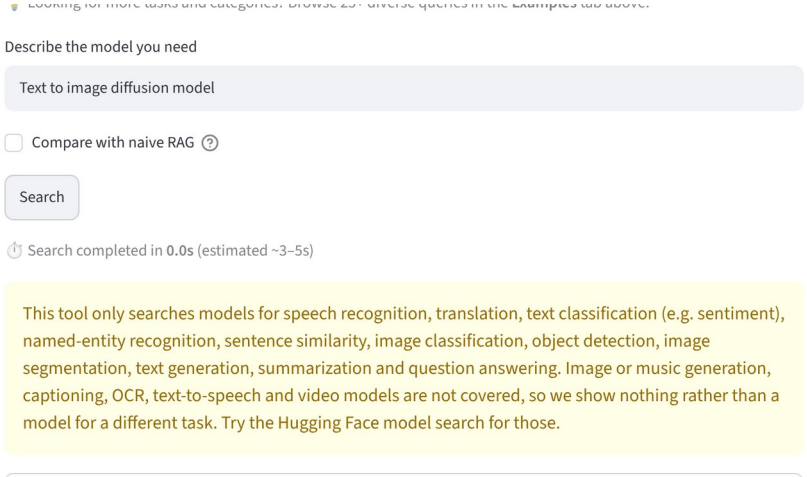


Fig. 4.10. An out-of-scope request is refused, with nothing shown rather than a model for a different task.

4.1.7 Editing the reading, glossary, examples and check-a-model

Any field of the reading can be edited and the search repeated (Fig. 4.11); a glossary defines licence, commercial, modifiable, NER, parameters, size, LoRA and unknown in plain words; and the Examples tab offers more than 25 queries grouped by task (Fig. 4.12).

The Check a model tab (Fig. 4.13) looks a model up in the snapshot, shows its facts (task, languages, size, parameters, rights, base model, documentation) and optionally checks it against requirements, which serves a reviewer who wants to know whether a model already in use is acceptable.

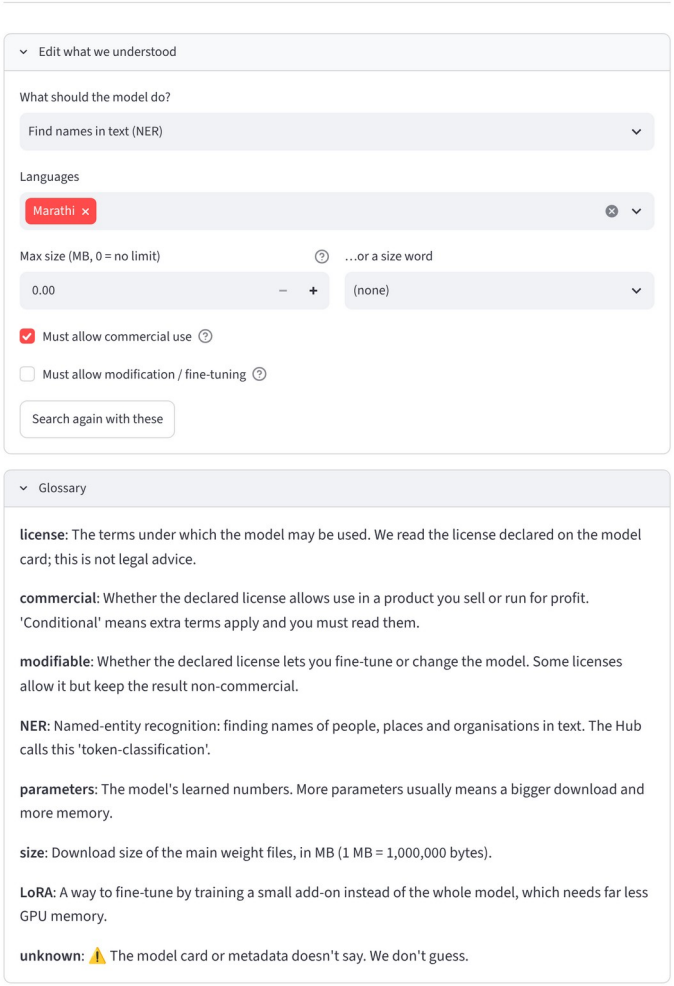


Fig. 4.11. The editable reading ("Edit what we understood") and the glossary.

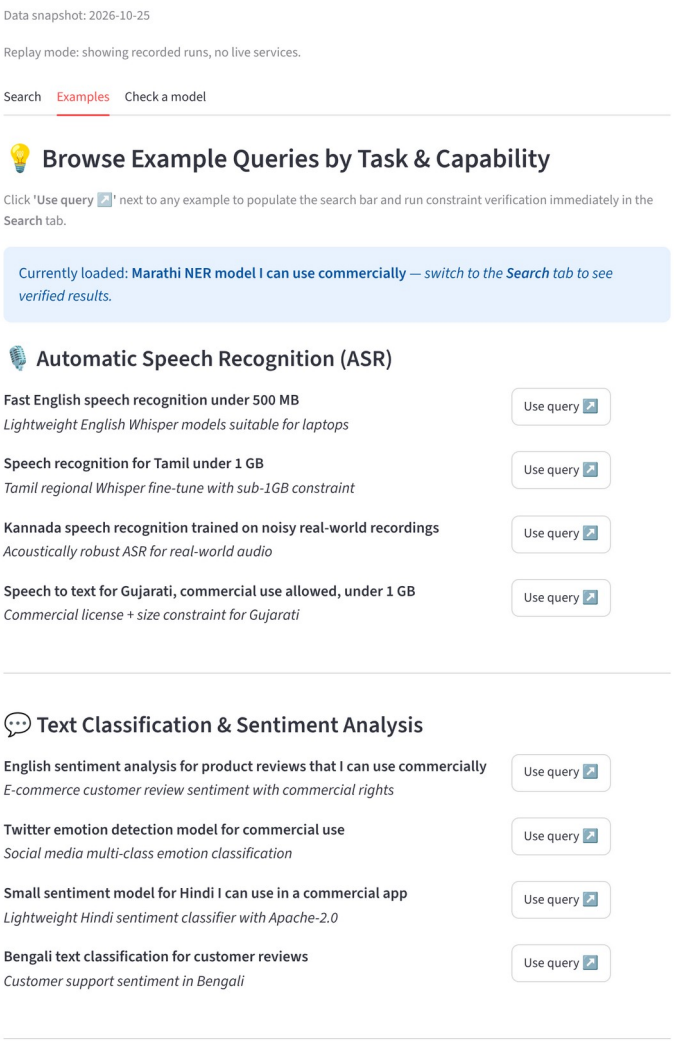


Fig. 4.12. The Examples tab: more than 25 queries grouped by task.

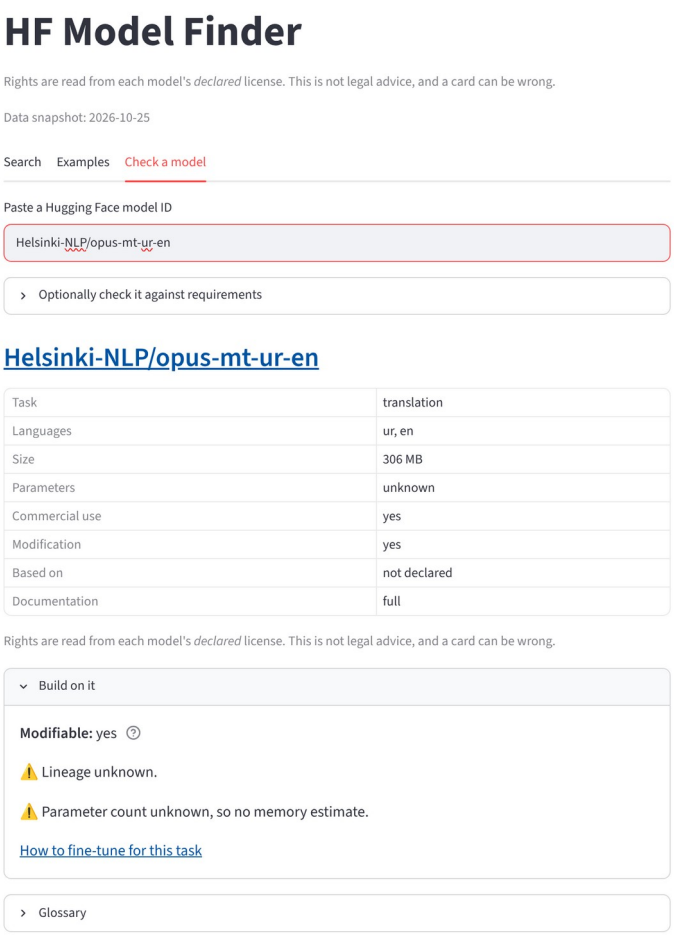


Fig. 4.13. Check-a-model mode for a model in the snapshot.

4.2 Corpus statistics

The index covers 15,991 models over eleven tasks (Fig. 4.14). The proportion of models whose size cannot be determined varies strongly by task: 6% for image classification, where weights follow the standard file names, but 75% for object detection, where many repositories keep weights in sub-folders or in formats the size rule does not count. Table 4.1 gives the data-quality figures for the whole snapshot and for the original five tasks on which the evaluation was run.

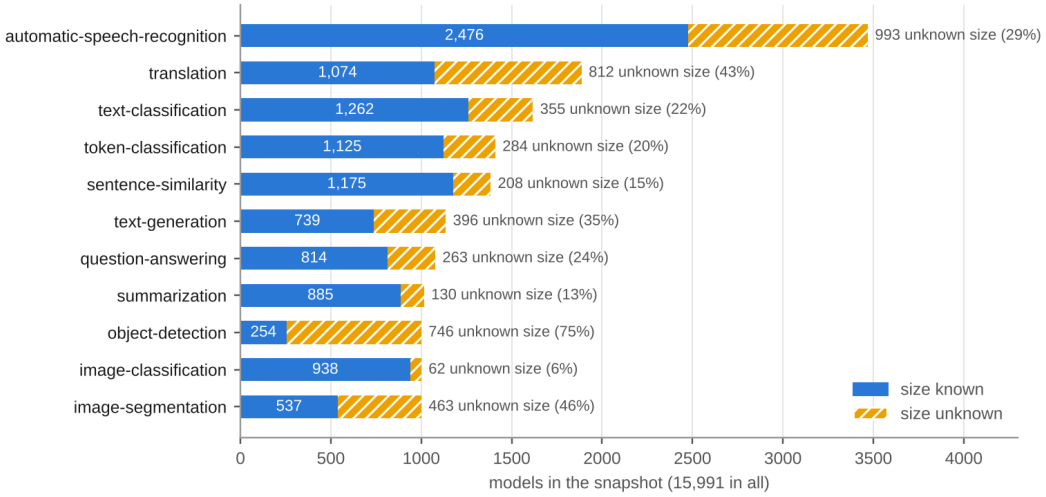


Fig. 4.14. Models per task in the snapshot, split by whether the size is known.

Table 4.1. Data-quality figures of the snapshot (percentages of models).

Measure	**All 11 tasks (15,991)**	**Original 5 tasks (9,764)**
Size unknown	29.5%	27.2%
Commercial and modifiable right unknown	20.1%	15.3%
Commercial use *yes* / *conditional* / *no*	70.3% / 6.5% / 3.2%	–
Language status unknown	32.4% (mostly vision, where language does not apply)	12.9%
Declared multilingual	4.9%	7.6%
Documentation minimal	6.5%	5.8%
Base model declared	47.5%	51.9%
Parameter count available	50.6%	51.4%
Author-reported evaluation results (`model-index`)	23.8%	26.6%
Translation direction stated by the model name	380 models	380 models

4.3 Constraint extraction

Extraction was tested on a set of hand-written requests with the expected constraint record, scored by exact match, and by the number of *silent drops* (a requested constraint that the extractor omitted, which would make a violation rate look better than it is). Table 4.2 shows the effect of the deterministic backstop. These are tuning-set figures, used to choose the model and to design the backstop, and they are not an estimate of how well the extractor generalises.

Table 4.2. Constraint-extraction accuracy on the tuning sets.

Configuration	Cases	Exact match	Silent drops
`gpt-oss-20b` alone	20	100%	0
`gpt-oss-120b` (fallback model) alone	20	85%	3
`gpt-oss-20b` alone, after three cases were added	23	87%	–
`gpt-oss-20b` alone, after six cases for the new tasks	29	90%	5
`gpt-oss-20b` with the deterministic backstop	29	100%	0

Re-parsing all 100 evaluation queries with the final prompt and backstop changed only seven parses, all intended (six for the permissive-licence rule and one for a language name), and when the prompt later gained hints for the six new tasks, four parses changed again (for example "not too big" now reads as a small model). The evaluation therefore uses the *pinned* parses of the frozen run, so later prompt changes cannot move its results.

4.4 Comparison with the baselines on normal queries

Table 4.3 gives the main result on the 75 normal queries. A *recommendation* is one of the top five models returned for a query; 365 to 375 recommendations were made by each system. The verdict of every recommendation was computed by the same verification rules.

Table 4.3. Main results on the 75 normal queries (top five per query, one set of rules for all systems).

System	Answered %	Recs	Violation %	Unknown %	Fully verified %	Median s	p95 s
Ours (constraint-verified RAG)	97.3	365	**0.0**	16.7	**83.3**	0.22	0.36
Naive RAG	100.0	375	50.4	20.5	29.1	0.07	0.64
Keyword (BM25) + filters	100.0	368	0.0	62.0	38.0	0.05	0.08
HF Hub, basic filters	100.0	375	32.8	13.6	53.6	0.10	0.15
HF Hub, licensed filters	100.0	375	25.9	14.7	59.5	0.09	0.14

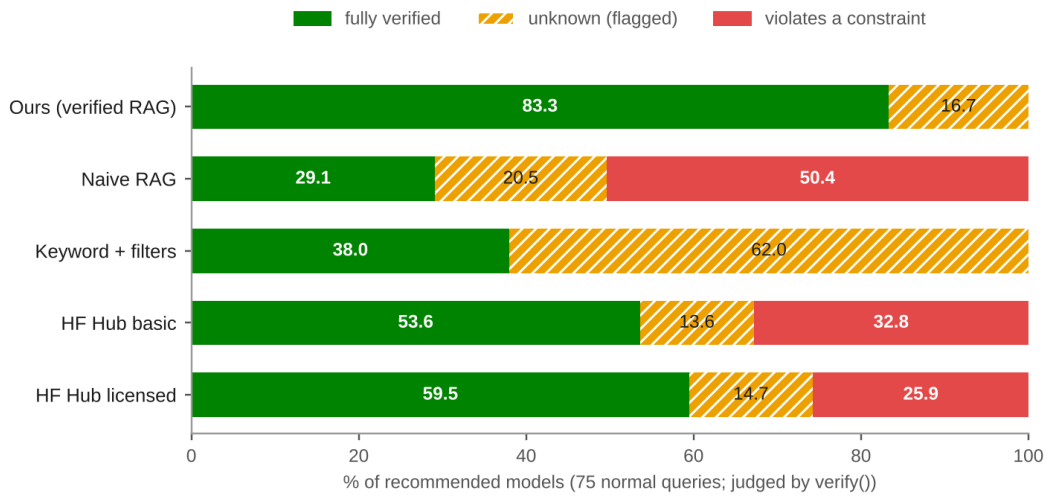


Fig. 4.15. Share of recommended models that are fully verified, unknown or violating, by system.

The full system recommended no model that fails a requested constraint, and 83.3% of its recommendations were verified on every requested constraint, against 29.1% for naive RAG and 59.5% for the best Hub filter. It answered 73 of 75 queries: on the other two (a request naming no single language and a request for language identification, for which no task is mapped) it asked a clarifying question. The latencies in Table 4.3 are for replayed, cached runs and are discussed in Section 4.7.

Fig. 4.16 shows *which* constraints the baselines break. Naive RAG fails the size limit on half of its recommendations, returns a model for the wrong task on 24.1% (for example speech-recognition models for a text request) and in the wrong language on 19.4%. The Hub filters never return the wrong task or language because those are the filters they have, but they cannot filter by size, since the Hub has no size filter, and fail it on 80.0% (basic) and 71.9% (licensed) of recommendations. The licensed variant removes commercial-use failures completely because it filters on one permissive licence string, which also excludes every model under other permissive licences (MIT, BSD, CC-BY-4.0), a loss of recall that the violation rate does not show. Keyword search with the same filters never violates a requested constraint, because the filters are the same as ours, but 62% of its recommendations are unknown: BM25 matches words in card text, which say nothing about size or licence, so it surfaces unknown-metadata models that merely contain the right words, and nothing ranks verified models first.

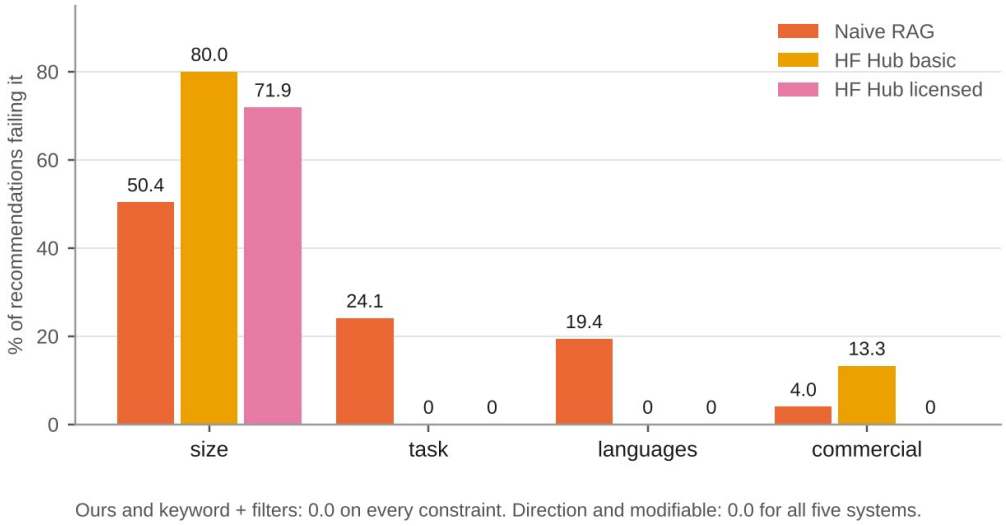


Fig. 4.16. Recommendations that fail each constraint, by baseline. Ours and keyword search: 0.0 on every constraint.

Table 4.4 breaks the violation rate down by task. Naive RAG and the Hub filters fail on every task, with rates between 16.0% and 62.7%. For the full system the violation rate is 0.0% on every task, and it answered 100% of the queries on each task except translation (93.3%), where the one query that asked for "Indian languages" without naming one was answered with a question.

Table 4.4. Violation rate (%) by task.

Task	Ours	Naive RAG	HF basic	HF licensed
Speech recognition	0.0	34.7	38.7	28.0
Translation	0.0	45.3	45.3	30.7
Sentence similarity	0.0	53.3	16.0	16.0
Text classification	0.0	60.0	35.7	32.9
Token classification	0.0	62.7	30.7	24.0
Indic-language queries	0.0	52.8	32.5	24.9
Non-Indic queries	0.0	40.0	34.3	30.0

The effect of the verify pool is itself a result. With a pool of 25 models, the first version of the system recommended 72.3% fully verified models; with the pool of 100 it recommended 83.3%. The translation queries are the clearest case: before the change, six queries had no fully verified model in their top five although a model such as `Helsinki-NLP/opus-mt-en-hi` existed in the corpus. The comparison against the Pinecone index and the exact local index gave the same aggregates for ours, naive RAG and keyword search; the top-five lists were identical on 72 of 100 queries for ours and for naive RAG and on all 100 for keyword search, the differences being ties and approximate-search ordering.

4.5 Vague queries, impossible queries and the no-verified slice

Vague queries. On the 15 beginner requests the full system asked a clarifying question on 100%. The baselines have no such behaviour and answered all of them, with unverified models (violation rates of 5.3% for naive RAG and both Hub variants, and 16.0% unknown for keyword search).

Impossible queries. Ten requests were written to be impossible (for example "Marathi speech recognition under 10 MB" or "Santali speech recognition model"). Because a model with unknown size or language can never fail a constraint, none of the ten could be proved impossible: every one left at least 31 models that were not failing (Table 4.5). The abstain and relax paths therefore never fired and the system answered all ten. For three of the ten it showed at least one fully verified model; the corpus holds at least one fully passing model for five of the ten (up to 33 for q099), so those requests were not impossible after all, and for the remaining seven the system showed unknown models with the banner.

Table 4.5. Models in the corpus that could answer each intended-impossible query.

Query	Fully pass	Not failing (pass or unknown)
q091	7	360
q092	0	105
q093	0	284
q094	1	127
q095	0	117
q096	7	342
q097	0	31
q098	0	308
q099	33	319
q100	6	192

The no-verified slice. Because the original abstention metric could not be satisfied, the measure was redefined on the 12 queries (normal and impossible) for which no model in the corpus fully passes. On this slice the right behaviour is *not to claim verification*. Fig. 4.17 shows that the full system and keyword search recommended only unknown models (no violation), and showed the banner on all 12 queries and claimed full verification on none; naive RAG and the Hub filters recommended violating models for 63.3% to 75.0% of their recommendations and do not warn the user.

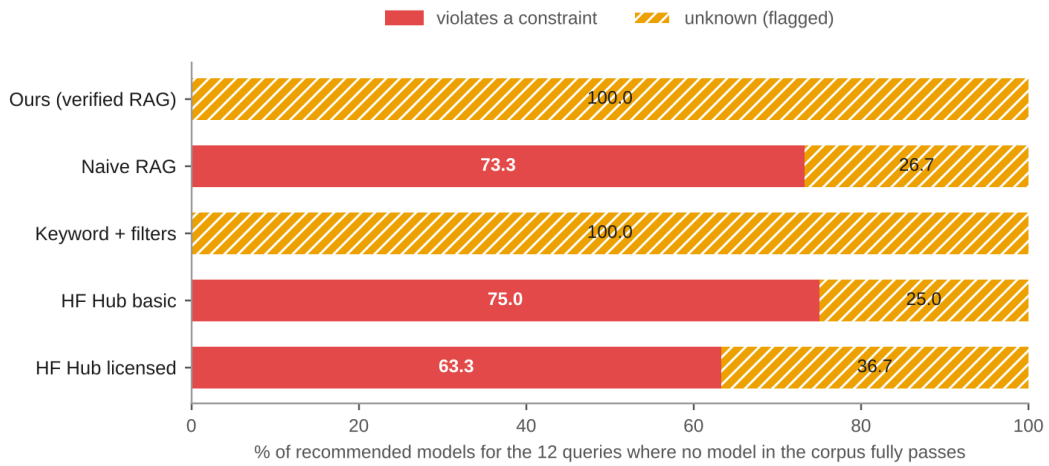


Fig. 4.17. Recommendations on the 12 queries for which no model in the corpus fully passes.

4.6 Relevance

Not violating a constraint is necessary, but a recommendation must also suit the purpose. Table 4.6 and Fig. 4.18 give the blind relevance study. The sheet had 686 pairs over 40 queries, made of the union of the top-five models of ours, naive RAG, keyword search and the licensed Hub baseline. **The grades were produced by independent language-model judges, not humans.** The full system covers 38 queries because it asked a clarifying question on two of the 40.

Table 4.6. Relevance at five, graded 0 to 2 by language-model judges (blind).

System	Queries	Models graded	Mean grade	Queries with a grade-2 model	Models graded 1 or 2
Ours	38	190	**1.28**	**81.6%**	**76.8%**
Keyword + filters	40	197	1.06	65.0%	70.1%
Naive RAG	40	200	0.87	67.5%	51.5%
HF Hub, licensed filters	40	200	0.69	22.5%	61.5%

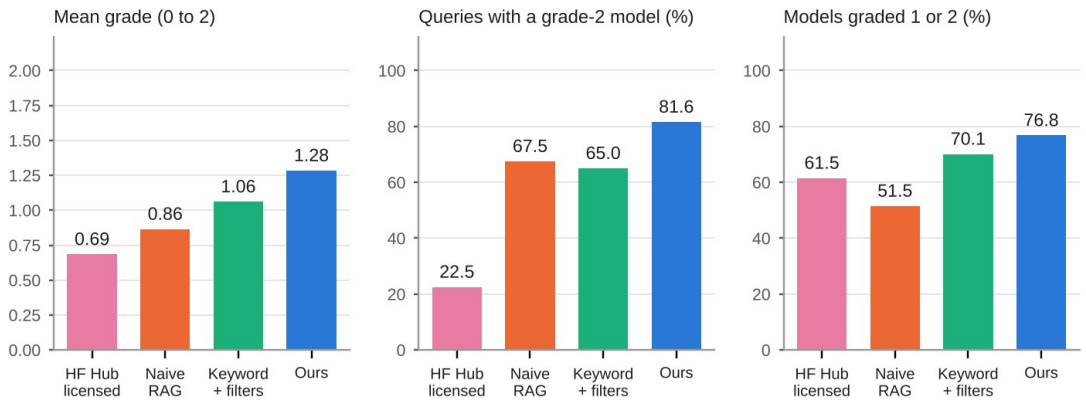


Fig. 4.18. Relevance results from the blind study (language-model judges).

The full system has the highest mean grade and the highest share of useful models, so verification did not cost relevance: filtering on facts removed unsuitable candidates and the verify-wide design kept the relevant ones. Naive RAG retrieves topically related models (67.5% of its queries have a grade-2 model, close to keyword search) but half of those fail a constraint, and its share of useful models is lowest among the semantic systems. The licensed Hub filter performs worst on relevance because the Hub orders by downloads and cannot express the soft part of a request.

Agreement. A repeat pass by three different judges graded 15 of the queries (266 pairs) from scratch, without seeing the first pass. The two passes gave an identical grade on 92.1% of the pairs, Cohen's kappa [36] was **0.874** (unweighted) and **0.945** (quadratic-weighted), and there were no disagreements between a grade of 0 and a grade of 2. In the bands of Landis and Koch [37], both values are "almost perfect". This measures the consistency of language-model judging; it is not agreement between independent humans and it says nothing about agreement with humans. The judges share a model family with the assistant that helped to build the system, so shared blind spots are possible, and a human spot check on a sample (for example one query per task) is the recommended strengthening.

4.7 Explanations, security, usability and performance

Faithfulness of explanations. Two judges counted the factual claims in 28 explanations (one explained model per query, chosen at random) and how many of them the card passages shown to the model support. Of 98 claims, 80 (81.6%) were supported, and 14 of 28 explanations (50.0%) had every claim supported. The unsupported claims were mostly statements the passages do not make: that a model is suitable for commercial use or fits a size limit (these come from the request, not the card), that a generic multilingual model supports a named language, and in one case a wrong translation direction. The verdict tables are built from metadata, not from the explanation, so these errors do not change a verdict, but they confirm the design rule that explanation text must not be used as a source of constraint facts (and the explanation prompt now forbids stating them).

Poisoned cards. With three hostile cards in a separate namespace and with deliberately unknown metadata, verdict tables came from metadata only in 5 of 5 runs, hostile cards ranked below honest ones, citations to identifiers outside the candidate set were dropped, and 0 of 5 explanations repeated an injected claim for each hostile card.

Try-it snippets. The snippet template was run on 32 small models, with the Transformers library plus the image libraries (Fig. 4.19). Twenty ran: all three of the image-classification, text-generation, question-answering and token-classification models, two of three object-detection, image-segmentation and summarization models, one of three text-classification and sentence-similarity models, and none of the translation and speech-recognition models. The failures were MLX or 4-bit quantised weights, a corrupt weights file, custom code, a missing tokenizer package, and decoder-only models used as translation pipelines. The interface therefore says that the snippet is a standard template, suitable for ordinary Hugging Face weights, and not tested on the model shown.

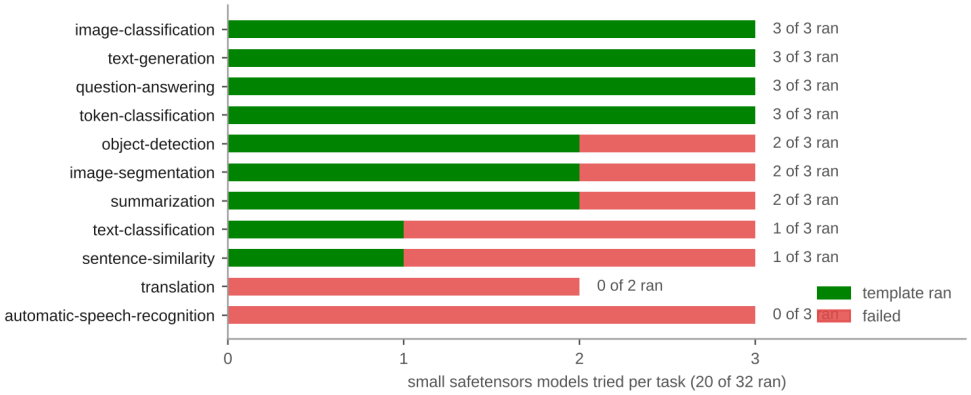


Fig. 4.19. Try-it smoke test: small models for which the generated snippet ran, by task.

Latency. Table 4.7 gives the timings. The evaluation timings are for cached embeddings, re-ranking and language-model calls, so they measure the code path and not the network. A live query with warm caches took a median of 1.3 s and a 95th percentile of 2.0 s without the explanation, which meets NFR-3; the explanation adds 7 to 17 s because of the language-model call and the throttle that keeps the call inside the free token limit. Removing the re-ranker leaves every verification metric unchanged (violation 0.0%, unknown 16.7%, fully verified 83.3%), because verification, not ordering, decides the verdicts; its effect on *relevance* was not measured, since the ablation outputs are not in the graded sheet.

Table 4.7. Latency of the pipeline.

Measurement	Value
Live query, warm caches, no explanation (hosted embedding and re-ranking, local index)	median 1.3 s, p95 2.0 s
Live query with explanation	7 to 17 s
Evaluation harness, cached runs: ours	median 0.22 s, p95 0.36 s
Evaluation harness, cached runs, ours without the re-ranker	median 0.19 s, p95 0.29 s
Evaluation harness, cached runs: naive, keyword, Hub basic, Hub licensed	median 0.07, 0.05, 0.10, 0.09 s
Local re-ranker fallback (CPU, 40 chunks)	about 20 s per query
Replay mode	no network; effectively instant

Scope extension. The six tasks added on 25 October are not part of the 100-query evaluation. They were checked in three ways: smoke queries (for example "object detection model for traffic cameras", where the unranked results were thermal, drone and stamp detectors and the re-ranked results were YOLO and VisDrone vehicle detectors and a traffic-accident detector), the Try-it smoke test above, and a regression check showing that the frozen evaluation is reproduced exactly on the five-task corpus and that ours is unchanged on the full index (the task filter excludes the new models), whereas naive RAG changed on 68 queries and keyword search on 46, because they have no task filter.

4.8 Discussion

Does verification reduce violations? Yes, and by construction: on the 75 normal queries the full system recommended no failing model, against 50.4% for naive RAG and 25.9% to 32.8% for the Hub filters. The more informative number is the share of *fully verified* recommendations (83.3%), because it shows what the user can rely on without reading anything. The remaining 16.7% are unknown, not wrong, and they are marked.

Does it cost relevance? No evidence of a cost was found. In the judged study the full system had the highest mean grade (1.28) and the highest share of queries with a clearly suitable model (81.6%). The explanation is that retrieval and verification do different jobs: retrieval ranks by meaning, verification removes what cannot suit, and verifying a wider pool than is shown means the removal does not empty the list.

Does it ask and abstain when it should? It asked on all 15 vague queries, and it never claimed verification when no model fully passes. It did not abstain, because abstention requires a known failure for every candidate and the corpus always contains unknown-metadata models; the honest behaviour on this data is to show the unknowns with a banner, which is also what the user chose.

Are explanations grounded? Mostly: 81.6% of claims are supported, but only half of the explanations are entirely supported, so the text should be read as a convenience and not as evidence. Safety-relevant facts come from the table.

Is it better than the Hub's filters? On constraint satisfaction, yes: the best Hub variant violated a requested constraint for 25.9% of recommendations, mostly size, which the Hub cannot filter. On relevance the licensed Hub variant graded lowest. The Hub is better in one respect, coverage: it searches over a million models and this system searches a sample of 15,991.

What limits the result? The main limit is the metadata itself. Where 27% to 30% of models have no determinable size, no method can verify a size limit for them; the system can only say so. The same holds for the 20% of models with no usable licence and, for translation, the direction of all but 380 models. The unknown rate of 16.7% is therefore close to a floor for this data and this set of queries, not a failure of the method.

4.9 Threats to validity and limitations

Table 4.8 lists the limits of the evidence and of the system. They are repeated here because they affect how far the results can be trusted.

Table 4.8. Threats to validity and known limitations.

Limit	Detail
Language-model judges	Relevance and faithfulness grades come from independent language-model agents, not humans; they share a model family with the assistant that helped build the system; the agreement figures are between two language-model passes
One author	The same person wrote the 100 queries and built the system; there are no classmate queries; mitigations are committing the queries before tuning, freezing outputs before grading and listing debugging queries (Table 4.9)
Evaluation covers five tasks	The 100 queries cover the original five tasks; the six added tasks were checked by smoke queries, the Try-it test and a regression check
Declared metadata	Verdicts are only as accurate as the declared metadata; the system cannot detect a false card or a wrong licence, and says rights are not legal advice
A sample of the Hub	15,991 models, as of 25 October 2026, not the Hub
Unknown size, language and direction	29.5% of sizes, 20.1% of rights and most translation directions are unknown and shown as such
Abstain and relax unreachable	Models with unknown fields never fail, so these paths are tested but do not fire on this corpus
Re-ranker quota	The hosted re-ranker's monthly quota is exhausted; uncached searches are not re-ranked (the page says so)
Permissive-licence approximation	"Apache", "MIT" or "permissive" in a request becomes commercial and modifiable; there is no constraint on the licence identifier itself
Try-it	Ran on 20 of 32 small models; MLX, GGUF and custom-code models need other tooling
Memory estimates	Lower bounds that exclude activations
Small sample of relevance	40 queries, 686 pairs; differences of a few points are noise

Table 4.9. Evaluation queries inspected while debugging, and what changed.

Queries	What was looked at	Change it led to
q016, q017, q018, q020, q025, q043	Translation queries had no fully verified model although one existed	Verify pool widened from 25 to 100 models
q099, q100	Santali was not a known language name, so the constraint was dropped	Indic language names added to the language table
q004, q038, q050, q066, q093	A named permissive licence never became a constraint	Permissive-licence rule in the backstop, with three new extraction cases written in different words
q021	Its parse changed when the permissive-licence rule was added	None beyond that rule
q046	Candidate pool inspected for a Marathi NER example: relevant models are over the size limit and correctly fail	None
q024, q045	Read only: both ask a clarifying question	None; reported as limitations

Chapter 5
Source Code

This chapter lists the parts of the source code that carry the main ideas of the work. The full repository (about 3,600 lines in `src/` and `app.py`, 900 in `eval/`, 700 in `scripts/` and 3,300 in `tests/`) is at <https://github.com/Adityarane012/RAG-Project>. The listings are copied directly from the files named in their captions, with the line ranges shown, so they match the code that produced the results in Chapter 4. Comments in the listings use the decision identifiers (D2, D12, D18 and so on) of the design log in `docs/decisions.md`.

5.1 Repository layout

Table 5.1 gives the role of each module. The module boundaries follow the design principle of Chapter 3: everything that decides a verdict is a plain module with no network access (`verify`, `normalize`, `retrieve`, `adapt`, `guide`), and everything that touches the network is injected into the graph through one dependency object so that the graph can be tested with stubs.

Table 5.1. Modules of the system.

File	Role	Lines
`src/verify.py`	Pass, fail or unknown for each requested constraint; overall verdict; rank key	123
`src/normalize.py`	Raw Hub record to metadata schema: size rule, licence map, languages, direction	317
`src/retrieve.py`	Metadata filter, chunk-to-model grouping, widening search	96
`src/graph.py`	LangGraph query pipeline and routing	208
`src/llm.py`	Provider access, cache, token throttle, extraction, backstop, explanation, injection defence	404
`src/evidence.py`	Selection and trimming of card passages for the explanation prompt	126
`src/guide.py`	Plain-language intent, clarifying questions, relax offers, snippets, out-of-scope check	223
`src/adapt.py`	Rights notes, lineage, fine-tuning memory estimate, model report	125
`src/index.py`, `src/embedder.py`, `src/localindex.py`	Chunking, embedding with cache, Pinecone and exact local index	611
`src/rerank.py`, `src/replay.py`	Hosted and local re-rankers; recorded-run replay	217
`src/ingest.py`, `src/snapshot.py`, `src/clean.py`	Hub download, snapshot with checksum, card cleaning	408
`src/baselines.py`, `src/checker.py`, `src/pipeline.py`	BM25 baseline and Hub filters; check-a-model; assembly of the live graph	209
`app.py`	Streamlit interface	396
`eval/`	Evaluation harness, metrics, labelling tools, reports	911
`scripts/`	Pinecone semantics check, poisoned-card test, Try-it smoke test, demo recording and rehearsal	668
`tests/`	335 offline tests in 31 files	3,349

5.2 Verification

Listing 5.1 is the whole verification module. It reads only the metadata dictionary and the constraint dictionary; it has no imports from the language-model or network code.

Listing 5.1. src/verify.py (lines 1 to 123): the single place where pass, fail or unknown is decided.

```
"""Re-verify every hard constraint in code: pass / fail / unknown (D2, D3, D18).

The LLM never decides whether a model satisfies a constraint, and card text never
changes a result (D17). `verify` returns a row only for constraints the user asked
for, so the UI never shows a ✅ for something nobody requested.

Constraints dict (output of llm.extract_constraints, editable in the UI):
    task, langs, direction, max_size_mb, size_word, commercial, modifiable, soft_query
"""

from __future__ import annotations

from src.config import CFG

PASS, FAIL, UNKNOWN = "pass", "fail", "unknown"
ICON = {PASS: "✅", FAIL: "❌", UNKNOWN: "⚠️"}

# D18: vague size words -> thresholds. The LLM only picks a key; code supplies the number.
SIZE_WORDS_MB: dict[str, float] = dict(CFG["size_words_mb"])


def resolve_max_size_mb(max_size_mb: float | None, size_word: str | None) -> float | None:
    """An explicit number from the user wins; otherwise look the word up in the table."""
    if max_size_mb is not None:
        return max_size_mb
    if size_word:
        return SIZE_WORDS_MB.get(size_word.strip().lower())
    return None


def check_task(meta: dict, task: str | None) -> str:
    if not meta.get("task"):
        return UNKNOWN
    return PASS if meta["task"] == task else FAIL


def check_languages(meta: dict, langs: list[str]) -> str:
    """All requested languages must be supported (e.g. hi AND mr)."""
    have = meta.get("languages") or []
    if not have or meta.get("language_status") == "unknown":
        return UNKNOWN
    if all(lang in have for lang in langs):
        return PASS
    # A multilingual model's list may be partial (the card says "multilingual"), so absence is
    not proof.
    return UNKNOWN if meta.get("multilingual") else FAIL


def check_direction(meta: dict, direction: list[str]) -> str:
    """Translation direction [source, target]. Never `fail`, never claimed from the
    language list alone: a model listing hi and mr may not translate both ways, so
    direction is unknown unless the card states it (`directions`, written only then)."""
    stated = meta.get("directions") or []
    return PASS if f"{direction[0]}-{direction[1]}" in stated else UNKNOWN


def check_size(meta: dict, max_size_mb: float) -> str:
    if meta.get("size_status") != "known" or "size_mb" not in meta:
        return UNKNOWN
    return PASS if meta["size_mb"] <= max_size_mb else FAIL


def check_right(meta: dict, field: str) -> str:
    """`commercial` or `modifiable`. "conditional" needs a human to read the terms,
    so it is never a clean pass."""
    status = meta.get(field, "unknown")
    if status == "yes":
        return PASS
    if status == "no":
```

```
        return FAIL
    return UNKNOWN


def verify(meta: dict, constraints: dict) -> dict[str, str]:
    """Per-constraint result for one model, for the constraints that were requested."""
    results: dict[str, str] = {}
    if constraints.get("task"):
        results["task"] = check_task(meta, constraints["task"])
    if constraints.get("langs"):
        results["languages"] = check_languages(meta, constraints["langs"])
    if constraints.get("direction"):
        results["direction"] = check_direction(meta, constraints["direction"])
    size_limit = resolve_max_size_mb(constraints.get("max_size_mb"),
    constraints.get("size_word"))
    if size_limit is not None:
        results["size"] = check_size(meta, size_limit)
    if constraints.get("commercial"):
        results["commercial"] = check_right(meta, "commercial")
    if constraints.get("modifiable"):
        results["modifiable"] = check_right(meta, "modifiable")
    return results


def notes(meta: dict, constraints: dict) -> list[str]:
    """Caveats shown next to a ✅ (D18, D20). They never change a result."""
    out: list[str] = []
    if constraints.get("langs") and meta.get("multilingual"):
        wanted = ", ".join(constraints["langs"])
        out.append(f"multilingual, not specific to {wanted}")
    direction = constraints.get("direction")
    if direction and meta.get("direction_source") == "model_name" and f"{direction[0]}-
    {direction[1]}" in (meta.get("directions") or []):
        out.append("direction from model name")
    if meta.get("doc_quality") == "minimal":
        out.append("minimal documentation")
    return out


def overall(results: dict[str, str]) -> str:
    """fail if anything fails, else unknown if anything is unknown, else pass."""
    values = results.values()
    if FAIL in values:
        return FAIL
    if UNKNOWN in values:
        return UNKNOWN
    return PASS


_RANK = {PASS: 0, UNKNOWN: 1, FAIL: 2}


def rank_key(results: dict[str, str]) -> tuple[int, int]:
    """Sort key: full passes first, then fewest unknowns (D12). Python's sort is
    stable, so the reranker's order is kept within each group."""
    n_unknown = sum(1 for v in results.values() if v == UNKNOWN)
    return (_RANK[overall(results)], n_unknown)
```

5.3 Normalisation: rights and size

Listing 5.2 is the licence-to-rights lookup, an unmapped licence giving *unknown* and never a guess. Listing 5.3 holds the translation-direction parser, the size rule and the reason a size is unknown. The size rule sums the top-level weight files and returns nothing, not zero, when it cannot be sure.

Listing 5.2. src/normalize.py (lines 130 to 135): licence identifier to rights.

```
def license_rights(license_id: str | None) -> tuple[str, str]:
    """(commercial, modifiable), each "yes" / "no" / "conditional" / "unknown"."""
    if not license_id:
        return ("unknown", "unknown")
    return LICENSE_RIGHTS.get(license_id.strip().lower(), ("unknown", "unknown"))
```

Listing 5.3. src/normalize.py (lines 200 to 262): translation direction, size rule and the reason a size is unknown.

```
def directions_from_name(model_id: str) -> list[str]:
    """["src-tgt"] for an opus-mt-<src>-<tgt> model id, else []."""
    m = _OPUS_MT.search(model_id.lower())
    if not m:
        return []
    src, tgt = (LANG_ALIASES.get(c, c) for c in m.groups())
    return [f"{src}-{tgt}"]


def _is_main_weight(name: str, suffix: str) -> bool:
    return "/" not in name and name.endswith(suffix) and name.startswith(_WEIGHT_PREFIXES)


def size_mb(files: list[tuple[str, int | None]]) -> float | None:
    """Model size in MB under the D20 rule, or None when it can't be determined.

    `files` is a list of (path, size_in_bytes) from the repo listing. Sums the
    top-level safetensors weights; if there are none, the top-level .bin weights.
    A weight file with no recorded size makes the whole answer unknown.
    """
    for suffix in (".safetensors", ".bin"):
        weights = [(n, s) for n, s in files if _is_main_weight(n, suffix)]
        if weights:
            if any(s is None for _, s in weights):
                return None
            return sum(s for _, s in weights) / MB  # exact: no rounding on a hard constraint
    return None


def param_count(safetensors_info: dict | None) -> int | None:
    """Total parameters from the Hub's safetensors metadata, if present."""
    if not safetensors_info:
        return None
    total = safetensors_info.get("total")
    return int(total) if isinstance(total, (int, float)) and total > 0 else None


def base_model(card_data: dict | None) -> str | None:
    """The declared `base_model` (first one if several); None when absent."""
    value = (card_data or {}).get("base_model")
    if isinstance(value, list):
        value = value[0] if value else None
    return value.strip() if isinstance(value, str) and value.strip() else None


def size_unknown_reason(files: list[tuple[str, int | None]]) -> str | None:
    """Why a model's size is unknown, to decide whether the D20 rule needs widening.
    None when the size is known."""
    if size_mb(files) is not None:
        return None
    names = [n for n, _ in files]
    if any(_is_main_weight(n, ".safetensors") or _is_main_weight(n, ".bin") for n in names):
        return "weights_missing_size"
    if any(n.rsplit("/", 1)[-1].startswith("adapter_model") for n in names):
        return "adapter_only"
    if any("/" in n and n.endswith((".safetensors", ".bin")) for n in names):
        return "weights_only_in_subfolder"
    if any(n.endswith(".gguf") for n in names):
        return "gguf_only"
    if any(n.endswith((".safetensors", ".bin")) for n in names):
        return "weights_with_other_name"
    return "no_weight_files"
```

5.4 Retrieval

Listing 5.4 builds the metadata filter, groups chunks by model and widens the search until enough distinct models are covered. Every clause keeps models whose field is unknown.

Listing 5.4. src/retrieve.py (lines 13 to 86): filter with unknowns kept, grouping, widening search.

```
def build_filter(constraints: dict) -> dict:
    """Pinecone metadata filter for the hard constraints in `constraints`."""
    clauses: list[dict] = []
```

```
    if constraints.get("task"):
        clauses.append({"task": {"$eq": constraints["task"]}})

    size_limit = resolve_max_size_mb(constraints.get("max_size_mb"),
    constraints.get("size_word"))
    if size_limit is not None:
        clauses.append({"$or": [
            {"size_mb": {"$lte": size_limit}},
            {"size_status": {"$eq": "unknown"}},
        ]})

    # Exclude only a definite "no"; conditional and unknown go on to verify.py.
    for right in ("commercial", "modifiable"):
        if constraints.get(right):
            clauses.append({right: {"$in": ["yes", "conditional", "unknown"]}})

    # Models from the multilingual map already have their covered languages merged into
    `languages`.
    for lang in constraints.get("langs") or []:
        clauses.append({"$or": [
            {"languages": {"$in": [lang]}},
            {"language_status": {"$eq": "unknown"}},
            {"multilingual": {"$eq": True}},     # list may be partial; verify decides (pass /
    unknown)
        ]})

    if not clauses:
        return {}
    return clauses[0] if len(clauses) == 1 else {"$and": clauses}


def group_by_model(matches: list[dict], top_n: int) -> list[dict]:
    """Collapse chunk matches to models, scored by their best chunk (D18).

    Each match is {"id", "score", "metadata": {...}}. Returns up to `top_n` models,
    best first, each with its best-scoring chunks under "chunks". Models are never
    re-sorted below the order of their best chunk, so a rerank carries through.
    """
    by_model: dict[str, dict] = {}
    for m in sorted(matches, key=lambda m: m["score"], reverse=True):
        model_id = m["metadata"]["model_id"]
        entry = by_model.setdefault(
            model_id, {"model_id": model_id, "score": m["score"], "meta": m["metadata"],
    "chunks": []}
        )
        entry["chunks"].append(m)
    return list(by_model.values())[:top_n]


def fetch_chunks(
    fetch_fn,
    constraints: dict,
    top_n: int | None = None,
    k: int | None = None,
    k_max: int | None = None,
) -> list[dict]:
    """Filtered chunk search, widening k until the chunks cover `top_n` distinct models (D18).

    `fetch_fn(filter, top_k) -> matches` wraps embed + index.query, so this loop is
    testable without Pinecone. Reranking only reorders chunks, so the distinct-model
    count can be checked before it.
    """
    top_n = top_n or CFG["retrieval"]["top_models"]
    k = k or CFG["retrieval"]["k_chunks"]
    k_max = k_max or CFG["retrieval"]["k_chunks_max"]
    flt = build_filter(constraints)
    while True:
        matches = fetch_fn(flt, k)
        distinct = {m["metadata"]["model_id"] for m in matches}
        # Stop when we have enough models, hit the cap, or the index has no more to give.
        if len(distinct) >= top_n or k >= k_max or len(matches) < k:
            return matches
        k = min(k * 2, k_max)
```

5.5 The query graph

Listing 5.5 is the pipeline of Section 3.6: the state, the dependency object, the two routing functions and the construction of the graph for the full system and for the two baselines.

Listing 5.5. src/graph.py (lines 36 to 208): state, routing and graph construction.

```
class State(TypedDict, total=False):
    # Input / carried between turns
    query: str
    overrides: dict           # user edits and answers, applied over the LLM's extraction
    clarifications_asked: int
    relax_count: int
    # Produced
    constraints: dict
    intent: str
    matches: list
    candidates: list
    results: list
    closest: list
    question: dict | None
    relax_offer: dict | None
    reports: list
    fully_verified: bool      # False => the answer holds only models with ⚠; the UI shows a
    banner
    explanations: list
    explain_error: str | None
    outcome: str              # answer | clarify | relax | abstain | out_of_scope
    message: str | None       # why, for out_of_scope
    rerank_skipped: bool      # True when the reranker's quota was exhausted and results keep
    vector order


@dataclass
class Deps:
    """Everything the graph needs from outside."""
    extract: Callable[[str], dict]                  # llm.extract_constraints
    fetch: Callable[[str, dict, int], list]         # (query text, filter, top_k) -> chunk
    matches
    rerank: Callable[[str, list], list]             # (soft_query, matches) -> matches,
    reordered
    explain: Callable[[dict, list], list] | None    # llm.explain; None skips explanations
    (evaluation, rate limits)
    lineage_index: dict = field(default_factory=dict)
    card_data: Callable[[str], dict | None] = lambda model_id: None


def viable(result: dict) -> bool:
    """A model worth showing: nothing failed (unknowns are shown, flagged ⚠)."""
    return result["overall"] != FAIL


def route_after_extract(state: State) -> str:
    if state.get("outcome") == "out_of_scope":
        return "out_of_scope"
    if clarifying_question(state["constraints"]) and state.get("clarifications_asked", 0) <
    MAX_CLARIFY:
        return "clarify"
    return "retrieve"


def route_after_verify(state: State) -> str:
    if any(viable(r) for r in state["results"]):
        return "enrich"
    if state.get("relax_count", 0) < MAX_RELAX and relaxation_offer(state["constraints"]):
        return "relax"
    return "abstain"


VARIANTS = ("ours", "naive", "keyword")


def build_graph(deps: Deps, variant: str = "ours"):
    """Compile the pipeline. `variant` selects what the evaluation compares (D19):

    * ours    : the full graph.
    * naive   : naive RAG. Same index, embeddings, reranker and LLM, but NO metadata filters
    (not even the
                task) and NO verification-driven behaviour: no clarifying, relaxing, abstaining,
    re-ranking
                by verdict or explanations. Verdict tables are still computed so the harness can
    judge it
                with the same rules as every other system.
    * keyword : like naive, but retrieval is whatever `deps.fetch` does (BM25 + the same
    metadata filters in
                the harness) and the reranker is skipped.
    """
    if variant not in VARIANTS:
```

```
    raise ValueError(f"variant must be one of {VARIANTS}, got {variant!r}")
plain = variant != "ours"
# The full system verifies a wider pool than it shows, so a verified model ranked 6th by the
reranker can
# still outrank an unknown one (D12); the baselines keep retrieval order and show exactly
the top models.
pool_size = CFG["retrieval"]["top_models"] if plain else CFG["retrieval"]["verify_pool"]

def extract(state: State) -> State:
    scope_message = out_of_scope_message(state["query"])
    if scope_message:        # image / vision / OCR requests: say so, and do not spend an
LLM call or show wrong models
        return {"constraints": clean_constraints({}, state["query"]), "intent": "",
"outcome": "out_of_scope", "message": scope_message}
    # User edits go through the same validator as the LLM's output: an override is untrusted
input too
    # (a bad task or size would otherwise make every model fail silently). Key presence
counts, so None clears.
    constraints = clean_constraints({**deps.extract(state["query"]),
**(state.get("overrides") or {})}, state["query"])
    return {"constraints": constraints, "intent": describe_intent(constraints)}

def clarify(state: State) -> State:
    return {"outcome": "clarify", "question": clarifying_question(state["constraints"]),
            "clarifications_asked": state.get("clarifications_asked", 0) + 1}

def retrieve(state: State) -> State:
    # naive RAG searches with no filter at all; keyword relies on deps.fetch applying the
filter itself
    # Every system searches with the user's own words. The extractor's `soft_query` is a
fragile LLM field: on
    # "small Gujarati classifier with a permissive license" it came back as "permissive
license" and retrieval
    # returned only licence sections. It is still used to pick explanation evidence.
    text = state["query"]
    return {"matches": fetch_chunks(lambda flt, k: deps.fetch(text, flt, k),
                                    {} if variant == "naive" else state["constraints"],
top_n=pool_size)}

def rerank(state: State) -> State:
    matches, skipped = state["matches"], False
    if variant != "keyword":
        try:
            matches = deps.rerank(state["query"], matches)
        except Exception as e:
            from src.rerank import RerankQuotaExceeded

            if not isinstance(e, RerankQuotaExceeded):
                raise
            skipped = True        # the hosted reranker's monthly quota is used up: keep
vector order, and say so
    return {"candidates": group_by_model(matches, pool_size), "rerank_skipped": skipped}

def verify_node(state: State) -> State:
    c = state["constraints"]
    results = []
    for cand in state["candidates"]:
        table = verify(cand["meta"], c)
        results.append({"model_id": cand["model_id"], "meta": cand["meta"], "chunks":
cand["chunks"],
                        "table": table, "overall": overall(table), "notes":
notes(cand["meta"], c)})
    if not plain:   # the full system ranks passes above unknowns above fails; the baselines
keep retrieval order
        results.sort(key=lambda r: rank_key(r["table"]))  # stable: reranker order within a
group
    results = results[:CFG["retrieval"]["top_models"]]
    return {"results": results, "closest": results[:3]}

def relax(state: State) -> State:
    return {"outcome": "relax", "relax_offer": relaxation_offer(state["constraints"]),
            "relax_count": state.get("relax_count", 0) + 1}

def finish(state: State) -> State:     # baselines: whatever retrieval returned, unfiltered
by verdict
    return {"outcome": "answer", "fully_verified": any(r["overall"] == PASS for r in
state["results"])}

def abstain(state: State) -> State:
    return {"outcome": "abstain"}

def enrich(state: State) -> State:
    c = state["constraints"]
```

```
        reports = []
        for r in filter(viable, state["results"]):
            report = model_report(r["meta"], deps.lineage_index, c.get("langs"),
    deps.card_data(r["model_id"]))
            report["try_it"] = try_it_snippet(r["model_id"], r["meta"]["task"]) if
    r["meta"].get("task") else None
            reports.append(report)
        return {"reports": reports}

    def explain(state: State) -> State:
        shown = [r for r in state["results"] if viable(r)]
        try:
            # the explanation relates each card to what the user asked, in the user's own words
    (soft_query can lose the topic)
            explanations, error = (deps.explain({**state["constraints"], "soft_query":
    state["query"]}, shown) if deps.explain else []), None
        except Exception as e:  # LLM down or bad JSON: the verified table must still render
            explanations, error = [], f"{type(e).__name__}: {e}"
        return {"outcome": "answer", "explanations": explanations, "explain_error": error,
                "fully_verified": any(r["overall"] == PASS for r in shown)}

    g = StateGraph(State)
    for name, fn in (("extract", extract), ("clarify", clarify), ("retrieve", retrieve),
                     ("rerank", rerank), ("verify", verify_node), ("relax", relax), ("finish",
    finish),
                     ("abstain", abstain), ("enrich", enrich), ("explain", explain)):
        g.add_node(name, fn)
    g.add_edge(START, "extract")
    g.add_conditional_edges("extract", ((lambda st: "out_of_scope" if st.get("outcome") ==
    "out_of_scope" else "retrieve") if plain
                                        else route_after_extract),
                            {"clarify": "clarify", "retrieve": "retrieve", "out_of_scope": END})
    g.add_edge("clarify", END)
    g.add_edge("retrieve", "rerank")
    g.add_edge("rerank", "verify")
    g.add_conditional_edges("verify", (lambda st: "finish") if plain else route_after_verify,
                            {"enrich": "enrich", "relax": "relax", "abstain": "abstain",
    "finish": "finish"})
    g.add_edge("finish", END)
    g.add_edge("relax", END)
    g.add_edge("abstain", END)
    g.add_edge("enrich", "explain")
    g.add_edge("explain", END)
    return g.compile()
```

5.6 Constraint extraction and the deterministic backstop

Listing 5.6 validates the language model's output and fills in only what the request plainly states.

Listing 5.6. src/llm.py (lines 248 to 358): validation, instruction guard, backstop and extraction.

```
def _lang_list(value) -> list[str]:
    if not isinstance(value, list):
        return []
    return [v.strip().lower() for v in value if isinstance(v, str) and v.strip()]


def clean_constraints(data: dict, query: str) -> dict:
    """Validate the LLM's JSON. Model output is untrusted: anything off-schema becomes null."""
    task = data.get("task")
    size = data.get("max_size_mb")
    word = data.get("size_word")
    direction = _lang_list(data.get("direction"))
    direction = direction if len(direction) == 2 else None
    langs = _lang_list(data.get("langs"))
    # A stated direction names both languages even if the model left `langs` empty.
    langs += [x for x in direction or [] if x not in langs]
    return {
        "task": task if task in TASKS else None,
        "langs": langs,
        "direction": direction,
        "max_size_mb": size if isinstance(size, (int, float)) and not isinstance(size, bool) and
    size > 0 else None,
        "size_word": word.strip().lower() if isinstance(word, str) and word.strip().lower() in
    SIZE_WORDS_MB else None,
        "commercial": True if data.get("commercial") is True else None,
```

```
        "modifiable": True if data.get("modifiable") is True else None,
        "soft_query": data.get("soft_query") if isinstance(data.get("soft_query"), str) and
    data["soft_query"].strip() else query,
    }


_INJECTION = re.compile(r"\bignore\b|disregard|previous instructions|system prompt|\
    binstructions?\b|\bset\b.{0,40}\bto\b\s+(true|false|null)", re.I)
_SIZE = re.compile(r"(\d+(?:[.,]\d+)?)\s*(gb|gib|mb|mib)\b", re.I)
_RIGHTS_COMMERCIAL = re.compile(r"\bcommercial(?:ly)?\b|\bsell\b|\bpaid product\b|\bin (?:my |
    our )?product\b", re.I)
_RIGHTS_MODIFY = re.compile(r"fine-?tun|\bmodif(?:y|iable|ied)\b|\bredistribut|\bbuild on\b|\
    badapt\b", re.I)
# "Apache-2.0 license", "MIT licensed", "Apache or MIT licensed", "permissive license",
    "licensed under BSD".
# There is no licence-id constraint; a permissive licence means the model may be used
    commercially and modified,
# so that is what we search for (an approximation, shown to the user in "We understood").
_PERMISSIVE = re.compile(r"\b(?:apache|mit|bsd|permissive(?:ly)?)\b\W+(?:\S+\W+){0,3}?licen[sc]|
    licen[sc]ed?\W+(?:under\W+)?(?:apache|mit|bsd)\b", re.I)
_TASK_WORDS = {
    "translation": re.compile(r"translat", re.I),
    "automatic-speech-recognition": re.compile(r"speech|transcri|\basr\b|\bvoice\b|\baudio\b",
    re.I),
    "token-classification": re.compile(r"\bner\b|named[- ]entit|entity|entities|
    part[- ]of[- ]speech|\bpos\b|find names", re.I),
    "text-classification": re.compile(r"sentiment|classif|\bspam\b|toxic|emotion|hate speech|
    topic", re.I),
    "sentence-similarity": re.compile(r"embedding|semantic search|sentence similarity|
    paraphrase|\bsimilarity\b", re.I),
    "image-classification": re.compile(r"image classif|classify (?:images?|photos?|pictures?)|
    (?:image|photo|picture) (?:recogni|categor)|\bimagenet\b", re.I),
    "object-detection": re.compile(r"object detect|detect (?:objects?|cars?|people|faces)|
    bounding box|\byolo\b", re.I),
    "image-segmentation": re.compile(r"(?:image|semantic|instance|panoptic|medical|satellite)
    segmentation|segment(?:ing)? (?:\w+ ){0,2}(?:images?|photos?|pictures?|scenes?)", re.I),
    "text-generation": re.compile(r"text generation|generate text|language model|\bllm\b|chat ?
    bot|chat model|instruction[- ]tuned|write (?:stories|code|text)", re.I),
    "summarization": re.compile(r"summari[sz]", re.I),
    "question-answering": re.compile(r"question[- ]answering|answer(?:ing)? questions|\bqa\b|
    reading comprehension", re.I),
}
_GENERATION_CUE = re.compile(r"\bgenerat|\bwrite\b|\bwriting\b|\bchat|assistant|\bgpt\b|
    completion|\bstory|\bstories|\bpoem|\bcopilot\b|\bllama\b|\bmistral\b|\bqwen\b|\bgemma\b",
    re.I)
_VISION = {"image-classification", "object-detection", "image-segmentation"}
_TEXT_LABELLING = {"text-classification", "token-classification", "sentence-similarity"}


def backstop_constraints(query: str, c: dict) -> dict:
    """Fill in ONLY what the LLM left empty and the request plainly states (D2/D18: facts come
    from
    code, parsed from the user's own words). It never overrides the LLM and never runs on
    instruction-like queries, so a request to "set commercial to false" cannot trick it into a
    value.
    Applied to the already-validated constraints; returns a new dict."""
    from src.guide import LANG_BY_NAME

    out = dict(c)
    if _INJECTION.search(query):
        return out
    # "text-generation" is the catch-all an LLM reaches for when a vague request only says "a
    model" ("I want a Hindi model" came
    # back as text-generation after the new tasks were added). Accept it only when the request
    has a generation cue; otherwise
    # leave the task empty so the user is asked what the model should do.
    if out["task"] == "text-generation" and not (_TASK_WORDS["text-generation"].search(query) or
    _GENERATION_CUE.search(query)):
        out["task"] = None
    if out["max_size_mb"] is None:
        m = _SIZE.search(query)
        if m:
            n = float(m.group(1).replace(",", "."))
            out["max_size_mb"] = n * 1000 if m.group(2).lower().startswith("g") else n     # 1
    GB = 1000 MB
    if out["max_size_mb"] is None and out["size_word"] is None:
        found = [w for w in SIZE_WORDS_MB if re.search(rf"\b{w}\b", query, re.I)]
        if len(found) == 1:
            out["size_word"] = found[0]
    if out["commercial"] is None and _RIGHTS_COMMERCIAL.search(query):
        out["commercial"] = True
    if out["modifiable"] is None and _RIGHTS_MODIFY.search(query):
        out["modifiable"] = True
```

```
    if _PERMISSIVE.search(query):
        out["commercial"] = out["commercial"] or True
        out["modifiable"] = out["modifiable"] or True
    if out["task"] is None:
        hits = [t for t, rx in _TASK_WORDS.items() if rx.search(query)]
        if any(h in _VISION for h in hits):           # "image classifier" is not text
   classification
            hits = [h for h in hits if h not in _TEXT_LABELLING]
        if len(hits) == 1:                        # two or more categories: ambiguous, so ask the
   user instead
            out["task"] = hits[0]
    words = [w.strip(".,;:!?()\"'") .lower() for w in query.split()]
    mentioned = [LANG_BY_NAME[w] for w in words if w in LANG_BY_NAME]
    out["langs"] = out["langs"] + [x for x in dict.fromkeys(mentioned) if x not in out["langs"]]
    return out


def extract_constraints(query: str, backstop: bool = True) -> dict:
    """Natural-language request -> constraints dict. Size words stay words here; code
    turns them into numbers (verify.resolve_max_size_mb)."""
    prompt = EXTRACT_PROMPT.format(
        tasks=", ".join(f'"{t}"' for t in TASKS),
        size_words=", ".join(f'"{w}"' for w in SIZE_WORDS_MB),
        query=query,
    )
    cleaned = clean_constraints(_parse_json(complete(prompt, reserve=CFG["llm"]
   ["extract_output_reserve_tokens"])), query)
    return backstop_constraints(query, cleaned) if backstop else cleaned


# --- explanations over untrusted card text (D17) -----------------------------------
```

5.7 Defence against hostile card text, and explanation

Listing 5.7 wraps card text as data, neutralises delimiters, filters citations and builds the explanation prompt from budgeted passages.

Listing 5.7. src/llm.py (lines 359 to 404): untrusted-text handling and the explanation call.

```
_TAG_LIKE = re.compile(r"<(\s*/?\s*card_section)", re.I)


def escape_card_text(text: str) -> str:
    """Neutralise any literal <card_section> / </card_section> so a card can't close
    the tag itself and smuggle text outside the data block."""
    return _TAG_LIKE.sub(r"&lt;\1", text)


def wrap_card_section(model_id: str, section: str, text: str) -> str:
    return (
        f'<card_section model_id="{html.escape(model_id, quote=True)}" '
        f'section="{html.escape(section, quote=True)}">\n'
        f"{escape_card_text(text)}\n</card_section>"
    )


def filter_citations(explanations: list[dict], candidate_ids: set[str]) -> list[dict]:
    """Drop any explanation citing a model that isn't in the candidate set (D17)."""
    return [e for e in explanations
            if isinstance(e, dict) and e.get("model_id") in candidate_ids]


def explain(constraints: dict, candidates: list[dict]) -> list[dict]:
    """Cited explanations for the verified candidates.

    Each candidate is {"model_id", "chunks": [{"score", "metadata": {"section", "text"}}]}.
    Only a budgeted selection of passages is sent (evidence.select_evidence): up to
    `explain.chunks_per_model` per model, trimmed to fit `explain.max_prompt_tokens`, so the
    prompt fits the free tier's tokens-per-minute limit. The output is untrusted too: only
    text, and only for models we passed in.
    """
    evidence = select_evidence(candidates, constraints.get("soft_query") or "")
    blocks = [wrap_card_section(e["model_id"], p["section"], p["text"])
              for e in evidence for p in e["passages"]]
    if not blocks:
        return []
    prompt = (
```

```
        f"User request: {constraints.get('soft_query', '')}\n\n"
        "Explain, per model, how its card relates to the request.\n\n" + "\n\n".join(blocks)
    )
    data = _parse_json(complete(prompt, system=EXPLAIN_SYSTEM))
    items = data.get("explanations")
    if not isinstance(items, list):
        return []
    return filter_citations(items, {c["model_id"] for c in candidates})
```

5.8 Token-budget throttle

Listing 5.8 is the sliding-window throttle that keeps calls inside the free tier's tokens-per-minute limit (Fig. A.5).

Listing 5.8. src/llm.py (lines 133 to 175): sliding-window token budget.

```
class TokenBudget:
    """Sliding 60-second window over tokens sent to one model. `acquire` blocks (sleeps)
    until the call fits, so two calls in the same minute can't exceed the provider's
    tokens-per-minute limit. Clock and sleep are injectable for tests."""

    def __init__(self, tpm: int, clock=time.monotonic, sleep=time.sleep):
        self.tpm, self._clock, self._sleep = tpm, clock, sleep
        self._events: deque[list] = deque()          # [time, tokens]
        self.last: list | None = None

    def acquire(self, tokens: int) -> float:
        """Reserve `tokens`; returns the seconds waited. A request bigger than the whole
        budget waits for an empty window and uses all of it (the provider may still reject
    it)."""
        if self.tpm <= 0:
            return 0.0
        tokens, waited = min(tokens, self.tpm), 0.0
        while True:
            now = self._clock()
            while self._events and now - self._events[0][0] >= 60:
                self._events.popleft()
            if sum(e[1] for e in self._events) + tokens <= self.tpm:
                self.last = [now, tokens]
                self._events.append(self.last)
                return waited
            pause = max(0.05, self._events[0][0] + 60 - now)
            self._sleep(pause)
            waited += pause

    def settle(self, actual: int) -> None:
        """Replace the estimate of the most recent `acquire` with the tokens the provider really
    counted."""
        if self.last is not None and self.tpm > 0:
            self.last[1] = min(int(actual), self.tpm)


_sleep = time.sleep
_budgets: dict[str, TokenBudget] = {}


def _budget_for(model: str) -> TokenBudget:
    if model not in _budgets:
        _budgets[model] = TokenBudget(CFG["llm"].get("tpm", {}).get(model, 0))
    return _budgets[model]
```

5.9 Build-on information

Listing 5.9 computes the plain-language rights notes and the fine-tuning memory estimate (Eq. 3.1).

Listing 5.9. src/adapt.py (lines 22 to 60): rights notes and memory estimate.

```
def rights_notes(meta: dict) -> list[str]:
    """Plain-language caveats on `commercial` / `modifiable` (D14)."""
```

```
    commercial, modifiable = meta.get("commercial", "unknown"), meta.get("modifiable",
    "unknown")
    notes = []
    if modifiable == "yes" and commercial == "no":
        notes.append("You may modify it, but the result stays non-commercial.")
    if modifiable == "no":
        notes.append("No derivatives allowed: you may not fine-tune and redistribute it.")
    if "conditional" in (commercial, modifiable):
        notes.append("Extra terms apply (e.g. use restrictions or share-alike conditions that "
                     "carry over to a fine-tune). Read the license before relying on it.")
    if "unknown" in (commercial, modifiable):
        notes.append("The license is missing or not one we can interpret.")
    return notes


# --- fine-tuning memory (lower bounds, excluding activations) ------------------

def memory_estimate(param_count: int | None) -> dict:
    """GB needed per method, compared with the reference GPU (a free Colab T4).

    Lower bounds that exclude activations; label them as estimates in the UI.
    """
    gpu = _A["reference_gpu_gb"]
    if not param_count:
        return {"status": UNKNOWN, "reference_gpu_gb": gpu, "methods": {}}
    inference = param_count * _A["bytes_per_param_inference"]
    # LoRA: frozen base weights at inference precision + a small trainable adapter
    # trained at full-fine-tune cost per parameter.
    adapter = param_count * _A["lora_adapter_fraction"] * _A["bytes_per_param_full_finetune"]
    full = param_count * _A["bytes_per_param_full_finetune"]
    methods = {}
    for name, nbytes in (("inference", inference), ("lora", inference + adapter), ("full",
    full)):
        gb = nbytes / GB
        methods[name] = {"gb": round(gb, 1), "fits": PASS if gb <= gpu else FAIL}
    return {"status": PASS, "reference_gpu_gb": gpu, "methods": methods}


# --- lineage -------------------------------------------------------------------
```

5.10 Evaluation metrics

Listing 5.10 defines the automatic metrics (Eq. 3.2) and Cohen's kappa (Eq. 3.3). The same functions score every system.

Listing 5.10. eval/metrics.py (lines 21 to 64): violation, unknown and clarification rates.

```
def _rates(tables_per_query: list[list[dict]], k: int) -> tuple[int, int, int]:
    total = fails = unknowns = 0
    for tables in tables_per_query:
        for table in tables[:k]:
            total += 1
            verdict = overall(table)
            fails += verdict == FAIL
            unknowns += verdict == UNKNOWN
    return total, fails, unknowns


def violation_rate_at_k(tables_per_query: list[list[dict]], k: int = 5) -> float:
    """Share of recommended models (top k per query) that break a hard constraint."""
    total, fails, _ = _rates(tables_per_query, k)
    return fails / total if total else 0.0


def unknown_rate_at_k(tables_per_query: list[list[dict]], k: int = 5) -> float:
    """Share of recommended models with ⚠️ on some constraint and no ❌."""
    total, _, unknowns = _rates(tables_per_query, k)
    return unknowns / total if total else 0.0


def abstention_accuracy(outcomes: list[str]) -> float:
    """On impossible queries: share where the system said no match. A relax offer
    ("no model meets all constraints; relax X?") counts as saying so."""
    return sum(o in ("abstain", "relax") for o in outcomes) / len(outcomes) if outcomes else 0.0


def clarification_rate(outcomes: list[str]) -> float:
```

```
    """On vague queries: share where it asked instead of guessing."""
    return sum(o == "clarify" for o in outcomes) / len(outcomes) if outcomes else 0.0


def modifiable_correctness(pairs: list[tuple[str | None, str]]) -> float:
    """Share of (license_id, shown `modifiable`) pairs that agree with the license map."""
    from src.normalize import modifiable_status

    return sum(modifiable_status(lic) == shown for lic, shown in pairs) / len(pairs) if pairs
   else 0.0


def latency_summary(seconds: list[float]) -> dict:
    return {"median": statistics.median(seconds), "worst": max(seconds)} if seconds else {}
```

Listing 5.11. eval/metrics.py (lines 90 to 119): Cohen's kappa, unweighted and quadratic-weighted.

```
def cohens_kappa(a: list[int], b: list[int], weights: str | None = None) -> float:
    """Agreement between two labellers on the same items, corrected for chance.

    `weights`: None (nominal), "linear" or "quadratic" (for ordinal grades like 0/1/2,
    where 0-vs-2 is a worse disagreement than 0-vs-1). Returns 1.0 for perfect
    agreement; when chance agreement is already perfect (one label only) returns 1.0 if
    the labellers agree and 0.0 otherwise.
    """
    if len(a) != len(b) or not a:
        raise ValueError("need two equal-length, non-empty label lists")
    cats = sorted(set(a) | set(b))
    idx = {c: i for i, c in enumerate(cats)}
    k, n = len(cats), len(a)
    obs = [[0.0] * k for _ in range(k)]
    for x, y in zip(a, b):
        obs[idx[x]][idx[y]] += 1 / n
    row = [sum(r) for r in obs]
    col = [sum(obs[i][j] for i in range(k)) for j in range(k)]

    def w(i: int, j: int) -> float:  # disagreement weight
        if weights is None:
            return float(i != j)
        d = abs(i - j) / (k - 1 if k > 1 else 1)
        return d if weights == "linear" else d * d

    observed = sum(w(i, j) * obs[i][j] for i in range(k) for j in range(k))
    expected = sum(w(i, j) * row[i] * col[j] for i in range(k) for j in range(k))
    if expected == 0:
        return 1.0 if observed == 0 else 0.0
    return 1 - observed / expected
```

5.11 Rendering a result

Listing 5.12 is the function of the interface that renders one run of the graph in every outcome: out of scope, clarify, relax or abstain, and answer with the banner, the verdict tables, the explanations and the Try it and Build on it panels.

Listing 5.12. app.py (lines 114 to 181): rendering the outcome of one graph run.

```
def render_state(state: dict) -> None:
    """Render one graph run's output. Handles every outcome."""
    outcome = state["outcome"]
    if outcome == "out_of_scope":
        st.warning(state.get("message") or "This request is outside what this tool searches.")
        return
    st.info(state["intent"])  # D13: parsed intent in plain words
    if state.get("rerank_skipped"):
        st.caption("Reranking was unavailable for this search (the hosted reranker's monthly
   quota is used up), so results are ordered "
                   "by text similarity only. The pass / fail / unknown checks are not
   affected.")

    if outcome == "clarify":
        q = state["question"]
        st.subheader(q["question"])
        for value, label in q["options"].items():
            if st.button(label, key=f"clarify-{value}"):
```

```
                _follow_up({q["field"]: value})
        if not q["options"]:  # free-form question (translation languages): offer a picker
            codes = st.multiselect("Languages", list(LANG_NAMES), format_func=LANG_NAMES.get,
key="clarify-langs")
            if st.button("Continue", disabled=len(codes) < 2):
                _follow_up({"langs": codes})
        return

    if outcome in ("relax", "abstain"):
        offer = state.get("relax_offer")
        st.warning(offer["message"] if offer else "No model meets all constraints.")
        if offer and st.button("Yes, relax it"):
            _follow_up({offer["field"]: offer["new_value"]})
        if state.get("closest"):
            st.caption("Closest matches (they fail at least one constraint):")
            for r in state["closest"]:
                st.markdown(f"**{r['model_id']}**")
                render_table(r)
        return

    reports = {r["model_id"]: r for r in state.get("reports", [])}
    explanations = {e["model_id"]: e for e in state.get("explanations", [])}
    if not state.get("fully_verified", True):
        st.warning("No model fully verified: every result below has at least one ⚠️ unknown.")
    if state.get("explain_error"):
        st.warning("Explanations are unavailable right now; the verified table below is
unaffected.")
    st.caption(LEGEND)
    models_to_show = [r for r in state["results"] if r["overall"] != "fail"]
    for i, result in enumerate(models_to_show):
        if i > 0:
            st.divider()
        mid = result["model_id"]
        st.markdown(f"### [{mid}](https://huggingface.co/{mid})")
        if mid in explanations:
            col_table, col_desc = st.columns([1, 1], gap="large")
            with col_table:
                render_table(result)
            with col_desc:
                st.markdown("##### 📝 Card Summary & Evidence")
                st.write(explanations[mid]["text"])
        else:
            render_table(result)
        report = reports.get(mid)
        if report:
            with st.expander("Try it"):
                st.code(report["try_it"] or "", language="python")
                st.caption("A standard template, not tested on this model. It suits ordinary
Hugging Face weights with a recent transformers "
                           "(image models also need timm or torchvision); MLX, GGUF, quantized
or custom-code models need their own "
                           "runtime. The model page says which.")
                st.markdown(f"[Open the model page]({report['model_page']})")
            with st.expander("Build on it"):
                render_build_on(report)
```

Chapter 6
Conclusion and Future Scope

6.1 Work carried out

This project designed, built and evaluated **HF Model Finder**, a constraint-verified retrieval-augmented system for finding Hugging Face models. A data pipeline turned the Hub into a checksummed snapshot of 15,991 models over eleven tasks, with size, licence rights, languages and translation direction normalised and every missing value marked unknown; 48,179 card chunks were embedded with a multilingual model and indexed in Pinecone and in an exact local index with the same filter semantics. A LangGraph pipeline parses a request with a language model and a deterministic backstop, asks clarifying questions when needed, retrieves with filters that keep unknowns, re-ranks, verifies a pool of 100 models in code, shows five, and explains them with cited, budgeted card excerpts under a prompt-injection defence. A Streamlit interface adds the plain-language reading of the request, the side-by-side comparison with naive RAG, starter snippets, adaptation information and a check-a-model mode. The system was tested by 335 offline tests and evaluated against four baselines on 100 queries fixed before tuning.

6.2 Conclusions

The results of Chapter 4 support the following conclusions, each with the limits stated there.

1. **Verification in code removes constraint violations that retrieval alone cannot.** On the 75 normal queries the full system recommended no model that fails a requested constraint, while naive RAG failed one on 50.4% of its recommendations and the Hub's own filters on 25.9% to 32.8%. Because verdicts are computed from metadata and never from model output, the property holds regardless of what the language model or a model card says.
2. **Honest unknowns are better than guesses and than silence.** 83.3% of the system's recommendations were fully verified and the remaining 16.7% were shown with an explicit unknown. On the 12 queries for which no model in the corpus can fully pass, the system never claimed verification and said so in a banner, while the baselines recommended violating models for 63.3% to 75.0% of their recommendations.

3. **Verifying wide and showing narrow matters.** Semantic search cannot see size, licence or direction; verifying 100 models instead of five raised fully verified answers from 72.3% to 83.3% and repaired the translation queries.
4. **Verification did not cost relevance.** In a blind study graded by language-model judges (not humans), the full system had the highest mean grade (1.28 of 2) and the highest share of queries with a clearly suitable model (81.6%); agreement between two judging passes was high (kappa 0.874 unweighted, 0.945 quadratic-weighted), which shows consistent judging but not agreement with human raters.
5. **Beginners are served by design, not by documentation.** The system asked a clarifying question on all 15 vague queries, shows how it understood the request, offers editable fields, runnable snippets and a memory estimate for fine-tuning, and refuses out-of-scope requests with nothing shown.
6. **Explanations are useful but not evidence.** 81.6% of the claims in explanations were supported by the card text shown, and half of the explanations were entirely supported; the design therefore keeps all licence, size and language facts in the verified table and forbids the explanation from stating them.

6.3 Limitations

The main limitations are summarised in Table 4.8. In brief: relevance and faithfulness were judged by language models; one person wrote the queries and built the system; the 100-query evaluation covers five of the eleven tasks; verdicts depend on declared metadata and 27% to 30% of sizes, 20% of rights and most translation directions are unknown; the abstain and relax paths do not fire on this corpus; the hosted re-ranker's monthly quota is exhausted, so new searches are not re-ranked; and the snapshot is a sample of the Hub dated 25 October 2026.

6.4 Future scope

The following extensions follow directly from the limitations and are listed in the order in which they would add the most evidence or value.

1. **Human validation of the grades.** Re-grade a sample (for example one query per task, about 40 pairs) with human annotators, report agreement with the language-model judges, and replace the judged figures where they differ. This is the single most useful strengthening of the evidence.

2. **Extend the evaluation to all eleven tasks.** Write queries for vision, text generation, summarization and question answering before tuning, include queries from people other than the builder, and repeat the study with the new tasks included.
3. **Make unknowns smaller without guessing.** Parse GGUF and ONNX sizes as separate, labelled sizes, read the licence text for the common custom licences marked `other`, infer translation direction from card text with a verified extraction step, and keep the unknown verdict wherever the evidence is not explicit.
4. **A stronger abstention policy.** Allow the system to abstain, or to offer a relaxation, when no candidate is fully verified and some constraint is known to fail for most candidates, and measure it on queries that are impossible by a known failure.
5. **Keep the index fresh.** Replace the one-off snapshot by an incremental pipeline that re-fetches changed models, with the checksummed snapshot retained for reproducibility, and add live fetching of a model outside the snapshot to the check-a-model mode.
6. **Wider coverage.** Add tasks that are currently mapped to the nearest task or refused (for example feature-extraction embeddings, audio classification and zero-shot classification), and a sampling strategy for text generation that does not rely on download counts alone.
7. **Benchmark scores as information.** Show author-reported evaluation results labelled as unverified (the code exists but is not shown), and study whether independently re-run benchmark numbers could become a verified signal.
8. **A user study.** Observe developers and students using the tool on their own requests and compare time to a suitable, usable model with the Hub's search; the present evaluation measures correctness of the list, not the user's experience.
9. **Local re-ranking and offline use.** Run the cross-encoder on a GPU or replace it with a smaller model so that re-ranking has no quota, and complete the rehearsal of the fully local mode (local index, local embeddings, local language model).
10. **Publish the evaluation.** Release the snapshot, queries, parses and grades as a small benchmark for constraint-aware model search.

6.5 Closing remark

The project began from a simple observation: a developer's request is a mixture of facts and preferences, and a system that treats facts as preferences will sometimes be wrong in ways the user cannot see. The remedy built here is modest and general. Let the language model do what it is good at, which is reading a request and describing a card; let code do what it must, which is checking facts; and show the user, in the same table, what is known, what is not, and what is false. The measurements in this report suggest that this separation costs little and removes a whole class of errors.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Retrieval augmented language model pre-training," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, PMLR vol. 119, 2020, pp. 3929–3938.

[3] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. 16th Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2021, pp. 874–880.

[4] S. Borgeaud *et al.*, "Improving language models by retrieving from trillions of tokens," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, PMLR vol. 162, 2022, pp. 2206–2240.

[5] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2310.11511.

[6] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, Dec. 2023.

[7] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, Dec. 2009.

[10] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, Jan. 2019.

[11] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 2318–2335.

[12] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," arXiv:2402.05672, Feb. 2024.

[13] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[14] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[15] S. Gollapudi *et al.*, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf. 2023 (WWW '23)*, 2023, pp. 3406–3416.

[16] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American*

Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2019, pp. 4171–4186.

[18] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

[19] L. Gao *et al.*, "PAL: Program-aided language models," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR vol. 202, 2023, pp. 10764–10799.

[20] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. arXiv:2303.17580.

[21] M. Mitchell *et al.*, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency*, 2019, pp. 220–229.

[22] T. Gebru *et al.*, "Datasheets for datasets," *Commun. ACM*, vol. 64, no. 12, pp. 86–92, Dec. 2021.

[23] W. Jiang *et al.*, "An empirical study of pre-trained model reuse in the Hugging Face deep learning model registry," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023, pp. 2463–2475.

[24] S. Longpre *et al.*, "A large-scale audit of dataset licensing and attribution in AI," *Nat. Mach. Intell.*, vol. 6, no. 8, pp. 975–987, Aug. 2024.

[25] D. Kakwani *et al.*, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 4948–4961.

[26] S. Doddapaneni *et al.*, "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in *Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL), Vol. 1: Long Papers*, 2023, pp. 12402–12426.

[27] J. Gala *et al.*, "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," arXiv:2305.16307, May 2023.

[28] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR vol. 202, 2023, pp. 28492–28518.

[29] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8440–8451.

[30] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158.

[31] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023. arXiv:2306.05685.

[32] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogenous benchmark for zero-shot evaluation of information retrieval models," in *Proc. 35th Conf. Neural Information Processing Systems, Datasets and Benchmarks Track*, 2021. arXiv:2104.08663.

[33] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.

[34] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, Mar. 2023.

[35] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, 2023, pp. 79–90.

[36] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, Apr. 1960.

[37] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, Mar. 1977.

[38] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE Standard 29148:2018, 2018.

[39] *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Standard 1016-2009, 2009.

[40] *IEEE Standard for System, Software, and Hardware Verification and Validation*, IEEE Standard 1012-2016, 2016.

[41] *Software and Systems Engineering — Software Testing — Part 3: Test Documentation*, ISO/IEC/IEEE Standard 29119-3:2021, 2021.

[42] *IEEE Standard Model Process for Addressing Ethical Concerns During System Design*, IEEE Standard 7000-2021, 2021.

[43] LangChain, Inc., "LangGraph documentation." [Online]. Available: https://langchain-ai.github.io/langgraph/ (accessed Oct. 27, 2026).

[44] Snowflake Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/ (accessed Oct. 27, 2026).

[45] Pinecone Systems, Inc., "Filter by metadata," Pinecone Documentation. [Online]. Available: https://docs.pinecone.io/guides/search/filter-by-metadata (accessed Oct. 27, 2026).

[46] Pinecone Systems, Inc., "multilingual-e5-large," Pinecone Documentation. [Online]. Available: https://docs.pinecone.io/models/multilingual-e5-large (accessed Oct. 27, 2026).

[47] Pinecone Systems, Inc., "bge-reranker-v2-m3," Pinecone Documentation. [Online]. Available: https://docs.pinecone.io/models/bge-reranker-v2-m3 (accessed Oct. 27, 2026).

[48] Groq, Inc., "Rate limits," GroqDocs. [Online]. Available: https://console.groq.com/docs/rate-limits (accessed Oct. 27, 2026).

[49] Hugging Face, Inc., "Hub API endpoints," Hugging Face Hub Documentation. [Online]. Available: https://huggingface.co/docs/hub/api (accessed Oct. 27, 2026).

[50] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[51] Hugging Face, Inc., "Model Cards," Hugging Face Hub Documentation. [Online]. Available: https://huggingface.co/docs/hub/model-cards (accessed Oct. 27, 2026).

[52] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, Aug. 2025.

[53] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, Dec. 2024.

Appendix A: Soft Code Flowcharts

This appendix gives flowcharts of the algorithms whose code is listed in Chapter 5. The overall query graph is Fig. 3.3 in Chapter 3 and the data pipeline is Fig. 3.2. In every flowchart, blue boxes are deterministic code, orange boxes are language-model calls, green boxes are the user or interface, and the three outcome colours are those of the verdicts (red for fail, amber for unknown, green for pass).

A.1 Verification of one model

Fig. A.1 shows `verify` and `overall` (Listing 5.1). One result is produced for each constraint the user requested, then the results are combined. A requested constraint that cannot be checked yields *unknown*, never *pass*.

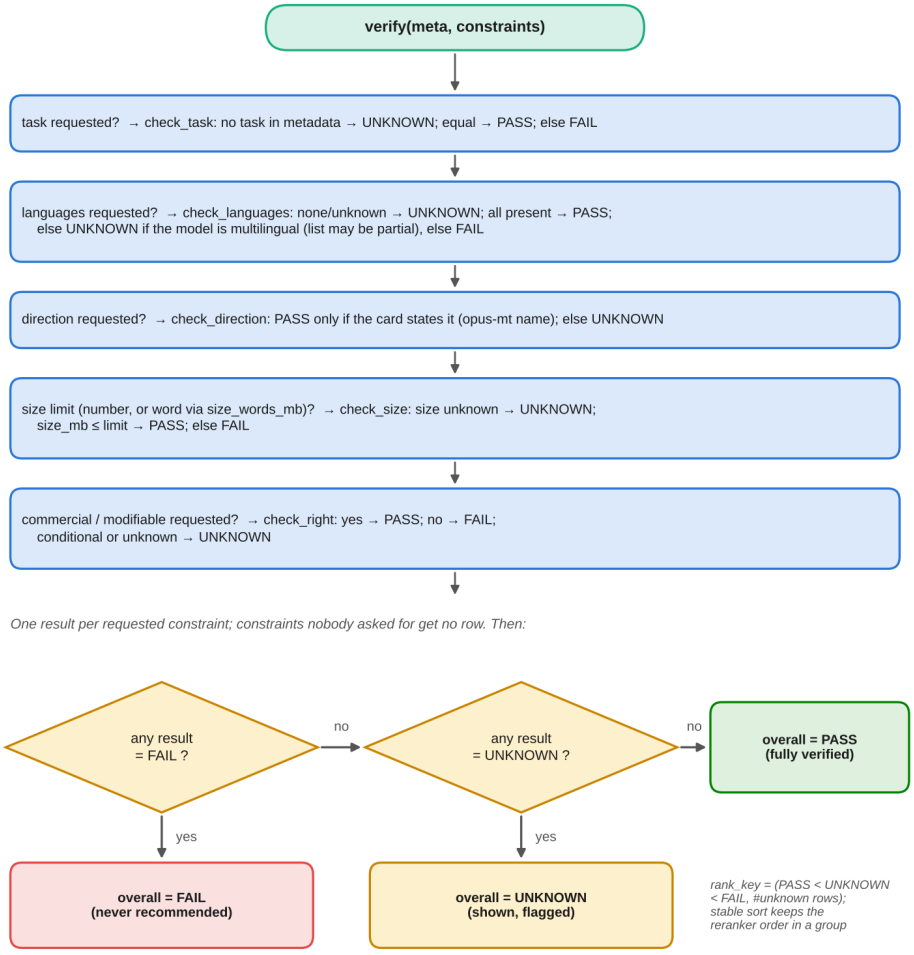


Fig. A.1. Flowchart of `verify`, `overall` and `rank_key`.

A.2 Metadata filter

Fig. A.2 shows `build_filter` (Listing 5.4). Each requested constraint contributes one clause, and each clause keeps models whose field is unknown, so the filter never decides a verdict.

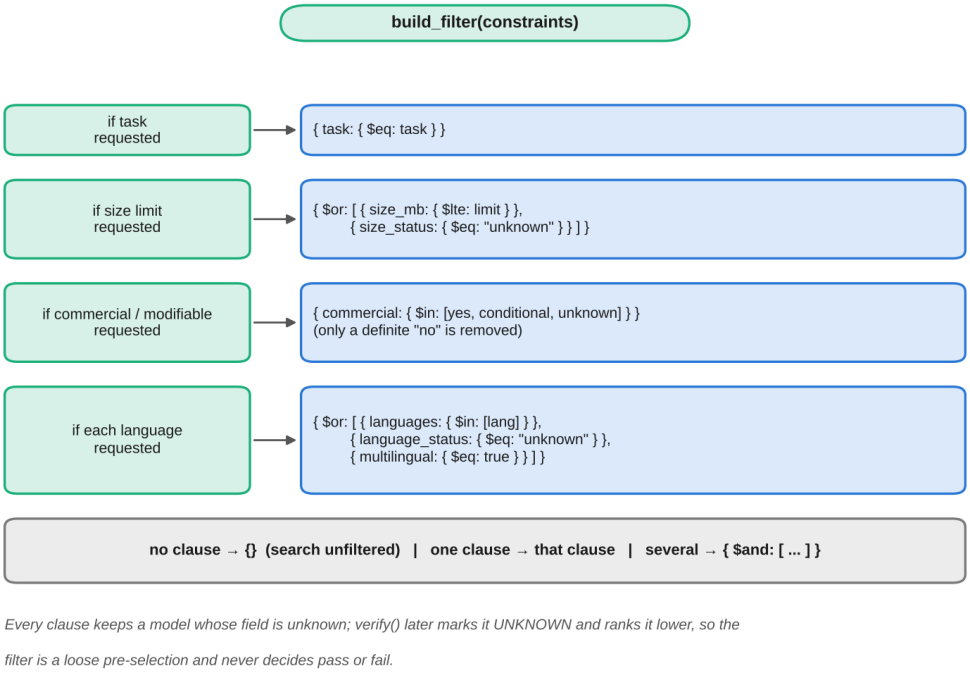


Fig. A.2. Flowchart of the metadata filter construction.

A.3 Size rule

Fig. A.3 shows the rule that gives a model's size (Listing 5.3). The size is the exact sum of the main weight files, and any doubt makes it unknown.

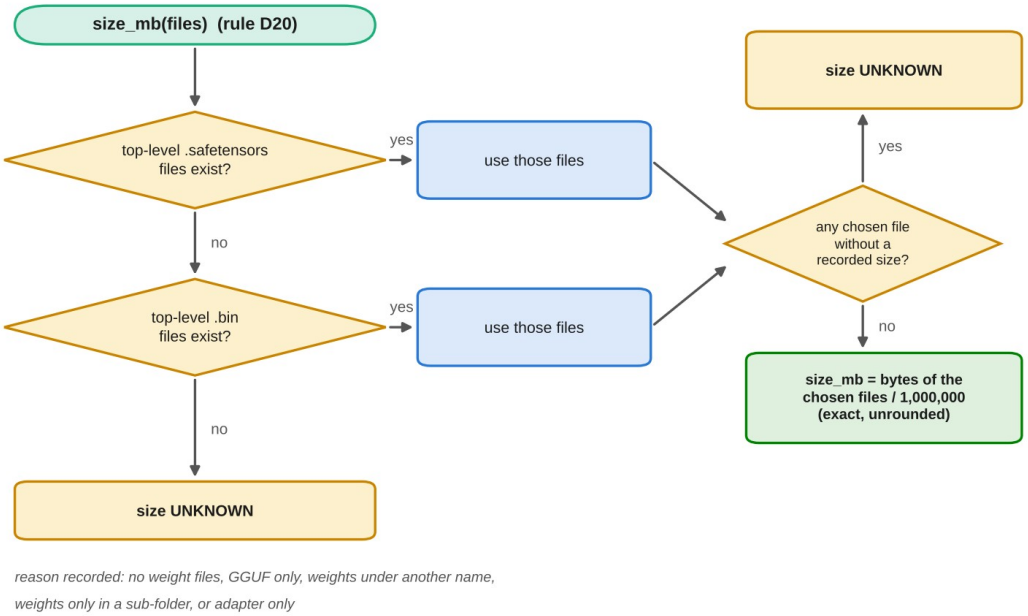


Fig. A.3. Flowchart of the size rule (D20).

A.4 Constraint extraction

Fig. A.4 shows how a request becomes a validated constraint record (Listing 5.6): refusal of out-of-scope requests, the language-model call, schema validation, the deterministic backstop and the user's overrides.

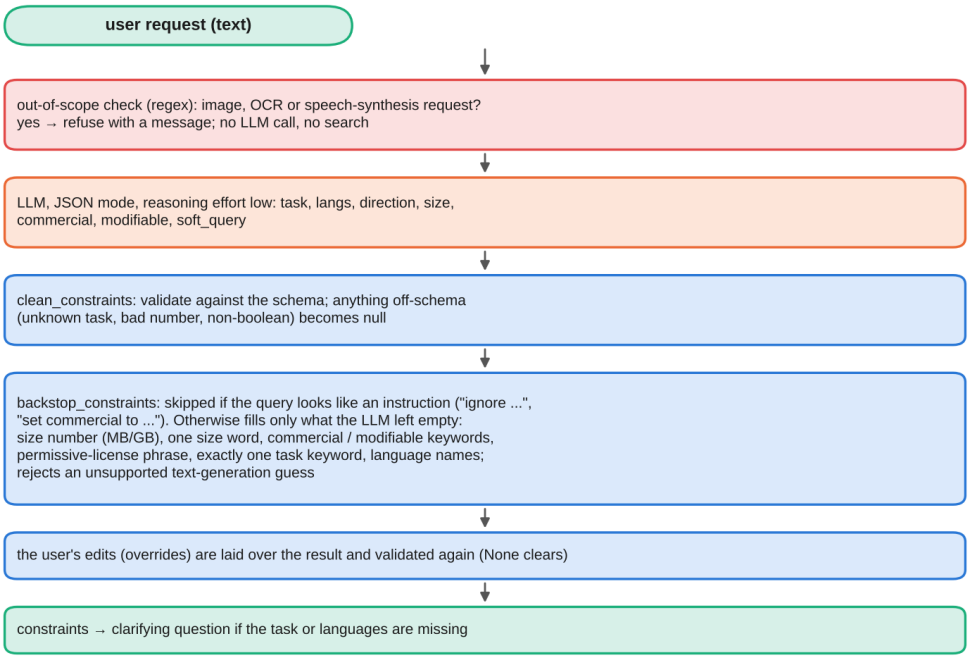


Fig. A.4. Flowchart of constraint extraction and the deterministic backstop.

A.5 Language-model call with token throttle

Fig. A.5 shows `complete` (Listing 5.8): the cache lookup, the reservation of tokens in the sliding window, the call, and the handling of rate-limit errors with a retry and a fallback model.

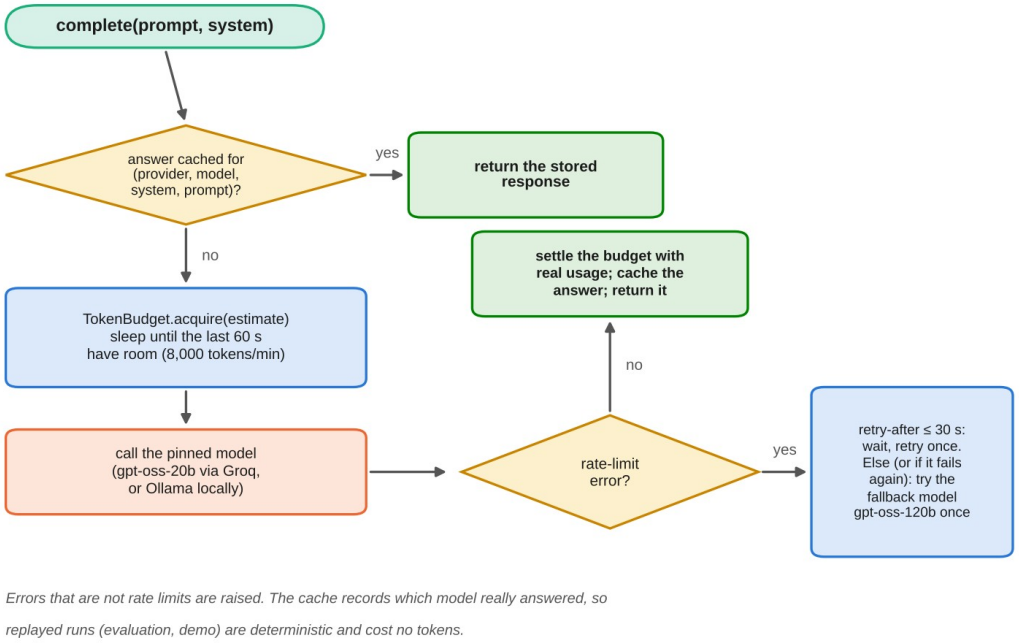


Fig. A.5. Flowchart of a language-model call with cache, token throttle and fallback.

Appendix B: Data Sheets

This appendix documents the data and the services that the system depends on, in the manner of a datasheet [22].

B.1 Datasheet of the model snapshot

Motivation. The snapshot exists so that models can be verified against requirements without calling the Hub at query time, and so that results can be reproduced. It was created for this project by the author.

Composition. Each record describes one model repository on the Hugging Face Hub: identifier, task, downloads, languages and language status, size and size status, parameter count, commercial and modification rights, base model and lineage status, translation directions, documentation quality, and a flag for the presence of author-reported evaluation results. The snapshot holds 15,991 records (SHA-256 `45814b8c7c15f569e2b558e7b0ae56a77f6835223500a69ab8937f2f443b8fa8`, dated 25 October 2026). The 9,764 records of the earlier five-task snapshot (dated 21 October 2026) are byte-identical in it. Card text is not in the snapshot; it is in the raw cache and, in chunked form, in the index (48,179 chunks).

Collection process. For each of eleven task tags the pipeline listed the 1,000 models with the most downloads and, for the language tasks, every model tagged with one of ten Indic languages; it then fetched each model's API record and card (two logged transient failures among about 16,000 fetches). Downloads were used for sampling only.

Preprocessing. Size, rights, languages and direction were computed by the rules of Section 3.5.2, with unknown preserved. Base-model identifiers were canonicalised through the Hub API. Cards were cleaned and chunked as in Section 3.5.3.

Uses. Intended: verifying constraints on candidate models, check-a-model, evaluation. Not intended: ranking models by quality, legal advice, or any claim about models outside the snapshot.

Distribution and maintenance. The snapshot is a local file whose checksum is recorded in the configuration; publishing it as a release asset is pending. It is not updated automatically; a rebuild re-runs the pipeline and produces a new dated snapshot.

Known limitations. The snapshot is a sample of the Hub, taken on one date. Metadata is as declared by model authors. Licence rights are interpreted from licence identifiers and are not legal advice.

B.2 Models per task

Table B.1 gives, for each task, the number of models, the number with unknown size, and the number with commercial use *yes* and *unknown*.

Table B.1. Composition of the snapshot by task.

Task	Models	Size unknown	Commercial *yes*	Commercial *unknown*
automatic-speech-recognition	3,469	993	2,989	372
translation	1,886	812	1,241	290
text-classification	1,617	355	1,096	388
token-classification	1,409	284	1,200	156
sentence-similarity	1,383	208	1,006	284
text-generation	1,135	396	668	295
question-answering	1,077	263	655	355
summarization	1,015	130	571	407
image-classification	1,000	62	793	153
object-detection	1,000	746	479	195
image-segmentation	1,000	463	544	314
All tasks	**15,991**	**4,712**	**11,242**	**3,209**

B.3 Licences in the snapshot

Table B.2 lists the most frequent licence identifiers and the rights assigned to them.

Table B.2. Most frequent licence identifiers.

Identifier	Models	Commercial	Modification
apache-2.0	7,990	yes	yes
mit	2,537	yes	yes
(no licence declared)	2,361	unknown	unknown
other	808	unknown	unknown
cc-by-4.0	573	yes	yes
cc-by-nc-4.0	409	no	yes
agpl-3.0	368	conditional	conditional
gemma	194	conditional	conditional
cc-by-sa-4.0	99	conditional	conditional
gpl-3.0	90	conditional	conditional
cc-by-nc-sa-4.0	77	no	yes
bsd-3-clause	71	yes	yes
llama3.1	65	conditional	conditional
llama3.2	57	conditional	conditional

B.4 Size summary

Among the 11,279 models with a known size, the median is 866 MB and the 90th percentile 6,173 MB. The reasons for an unknown size in the other 4,712 models are no weight files (2,416), GGUF only (1,570), weights under another name (262), weights only in a sub-folder (243) and adapter only (221).

B.5 Vector index

Table B.3 gives the specification of the vector index and the chunks.

Table B.3. Vector index specification.

Item	Value
Service	Pinecone serverless, index `hf-model-finder`, AWS us-east-1
Namespaces	`prod` (production data), `exp-<name>` (experiments), `eval5` (five-task reproduction of the frozen evaluation)
Vectors	48,179 in `prod`; 15,990 models (one card produced no text)
Dimension and metric	1,024; cosine
Embedding model	`multilingual-e5-large`, with the passage and query prefixes the model expects
Chunking	By Markdown heading; at most 450 estimated tokens per chunk; at most 4 chunks per model; boilerplate sections dropped; mean 3.0 chunks per model
Metadata on each vector	The fields of Table 3.4
Filter operators used	`$eq`, `$lte`, `$in`, `$and`, `$or` [45]

B.6 Models and service limits

Table B.4 lists the models used and the limits of the free services, as measured or documented in October 2026.

Table B.4. Models used and free-tier limits.

Item	Value
Embedding [12], [46]	`multilingual-e5-large`, 1,024 dimensions; Pinecone-hosted; 4.05 million tokens used for the whole corpus
Re-ranking [11], [47]	`bge-reranker-v2-m3`; hosted limit of 500 requests per month, reached on 25 October 2026; local fallback on CPU
Primary language model [48], [52]	`openai/gpt-oss-20b` on Groq, temperature 0, JSON mode, low reasoning effort; 8,000 tokens per minute, 1,000 requests and 200,000 tokens per day
Fallback language model	`openai/gpt-oss-120b` on Groq, tried once after a rate-limit error; same limits, own bucket
Offline language model [53]	`qwen2.5:7b-instruct` via Ollama (about 4.7 GB), `qwen2.5:3b` for 8 GB machines; the provider code is tested with stubs only
Cost of a query	About 500 tokens for extraction and 1,300 to 2,800 for the explanation; about 3,300 for a fully explained query, so about 60 explained queries a day

B.7 Configuration values that affect results

Table B.5 lists the values in `config.yaml` on which the results depend.

Table B.5. Selected configuration values.

Key	Value	Meaning
`retrieval.k_chunks`, `k_chunks_max`	50, 400	First and maximum number of chunks requested
`retrieval.top_models`	5	Models shown
`retrieval.verify_pool`	100	Models verified before the cut
`embedding.max_section_tokens`	450	Largest chunk
`embedding.max_chunks_per_model`	4	Chunks kept per model
`cleaning.min_card_chars`	200	Below this a card counts as minimal
`size_words_mb`	100 / 500 / 2,000	tiny and mobile / small and lightweight / medium
`graph.max_clarifying_questions`, `max_relax_retries`	2, 1	Loop caps
`explain.max_prompt_tokens`	5,000	Prompt budget for the explanation
`adapt.reference_gpu_gb`	16	Reference GPU for memory estimates

Appendix C: List of Components

Table C.1 to Table C.3 list the software, hardware and service components, and the data artefacts, that make up the project. Versions are those installed in the environment in which the results were produced.

Table C.1. Software components.

Component	Version	Purpose in the project
Python	3.13	Implementation language
LangGraph	1.2	State-graph orchestration of the query pipeline
Streamlit	1.66	Web interface
Pinecone Python client	10.1	Vector store, hosted embeddings, hosted re-ranking
Groq Python client	1.7	Language-model inference
huggingface_hub	2.2	Hub listing and downloads
sentence-transformers	6.2	Optional local embeddings and re-ranker
Transformers	4.57 / 5.20	Try-it smoke test
NumPy	2.5	Vector arithmetic in the exact local index
PyYAML, requests, python-dotenv	–	Configuration, HTTP, environment file
pytest	9.1	335 offline tests
Playwright (Chromium)	–	Screenshots for this report
LibreOffice, python-docx, matplotlib	–	Generation of this document and its figures

Table C.2. Hardware and hosted-service components.

Component	Specification
Development container	4 virtual CPUs, 16 GB RAM, no GPU, Linux 6.x
Pinecone Starter	Serverless vector index (48,179 vectors); hosted `multilingual-e5-large` and `bge-reranker-v2-m3`
Groq free tier	`gpt-oss-20b` and `gpt-oss-120b` inference
Hugging Face Hub	Public API for model metadata and cards
Ollama (optional)	Local language-model runtime, not used for the reported results

Table C.3. Data artefacts in the repository.

Artefact	Content
`data/snapshot-2026-10-25/records.jsonl.gz`	Normalised snapshot, 15,991 records
`eval/queries.jsonl`	The 100 evaluation queries (committed before tuning)
`eval/extraction_test.jsonl`	Constraint-extraction test cases
`eval/results/`	Raw and summarised results of every run, frozen parses, relevance and faithfulness reports, smoke-test outputs
`eval/labels/`	Relevance sheet, grades of both passes, faithfulness grades, one-line reasons from the judges, provenance note
`demo/plan.json`, `demo/cache/`	Ten demonstration scenarios and their recorded runs
`docs/`	Design log (D1 to D24), architecture, data notes, evaluation plan, security and limitations

Appendix D: List of Papers Presented and Published

No paper based on this work has been presented or published to date.

If any paper, poster or demonstration based on this project is presented or published later, list it here in IEEE format, and include reprints after the reference list as the guidelines permit.